MySQL Shell 8.0 Release Notes

Abstract

This document contains release notes for the changes in each release of MySQL Shell 8.0.

For additional MySQL Shell documentation, see http://dev.mysql.com/.

Updates to these notes occur as new product features are added, so that everybody can follow the development process. If a recent version is listed here that you cannot find on the download page (https://dev.mysql.com/downloads/), the version has not yet been released.

The documentation included in source and binary distributions may not be fully up to date with respect to release note entries because integration of the documentation occurs at release build time. For the most up-to-date release notes, please refer to the online documentation instead.

For legal information, see the Legal Notices.

For help with using MySQL, please visit the MySQL Forums, where you can discuss your issues with other MySQL users.

Document generated on: 2025-08-04 (revision: 30411)

Table of Contents

Preface and Legal Notices 2
Changes in MySQL Shell 8.0.43 (2025-07-22, General Availability) 4
Changes in MySQL Shell 8.0.42 (2025-04-15, General Availability) 4
Changes in MySQL Shell 8.0.41 (2025-01-21, General Availability) 5
Changes in MySQL Shell 8.0.40 (2024-10-15, General Availability) 6
Changes in MySQL Shell 8.0.39 (Not released, General Availability) 7
Changes in MySQL Shell 8.0.38 (2024-07-01, General Availability) 7
Changes in MySQL Shell 8.0.37 (2024-04-30, General Availability) 8
Changes in MySQL Shell 8.0.36 (2024-01-16, General Availability) 9
Changes in MySQL Shell 8.0.35 (2023-10-25, General Availability) 10
Changes in MySQL Shell 8.0.34 (2023-07-18, General Availability) 12
Changes in MySQL Shell 8.0.33 (2023-04-18, General Availability) 13
Changes in MySQL Shell 8.0.32 (2023-01-17, General Availability) 20
Changes in MySQL Shell 8.0.31 (2022-10-11, General Availability) 27
Changes in MySQL Shell 8.0.30 (2022-07-26, General Availability) 33
Changes in MySQL Shell 8.0.29 (2022-04-26, General Availability) 39
Changes in MySQL Shell 8.0.28 (2022-01-18, General Availability) 43
Changes in MySQL Shell 8.0.27 (2021-10-19, General Availability) 47
Changes in MySQL Shell 8.0.26 (2021-07-20, General Availability) 54
Changes in MySQL Shell 8.0.25 (2021-05-11, General Availability) 56
Changes in MySQL Shell 8.0.24 (2021-04-20, General Availability) 57
Changes in MySQL Shell 8.0.23 (2021-01-18, General Availability) 61
Changes in MySQL Shell 8.0.22 (2020-10-19, General Availability) 67
Changes in MySQL Shell 8.0.21 (2020-07-13, General Availability) 72
Changes in MySQL Shell 8.0.20 (2020-04-20, General Availability) 75
Changes in MySQL Shell 8.0.19 (2020-01-13, General Availability) 79
Changes in MySQL Shell 8.0.18 (2019-10-14, General Availability) 85

Changes in MySQL Shell 8.0.17 (2019-07-22, General Availability) 89
Changes in MySQL Shell 8.0.16 (2019-04-25, General Availability) 97
Changes in MySQL Shell 8.0.15 (2019-02-01, General Availability) 102
Changes in MySQL Shell 8.0.14 (2019-01-21, General Availability) 102
Changes in MySQL Shell 8.0.13 (2018-10-22, General Availability) 109
Changes in MySQL Shell 8.0.12 (2018-07-27, General Availability) 115
Changes in MySQL Shell 8.0.11 (2018-04-19, General Availability) 120
Changes in MySQL Shell 8.0.5 - 8.0.10 (Skipped version numbers) 126
Changes in MySQL Shell 8.0.4 (2018-01-25, Release Candidate) 126
Changes in MySQL Shell 8.0.3 (2017-09-29, Development Milestone) 133
Changes in MySQL Shell 8.0.2 (Not released) 138
Changes in MySQL Shell 8.0.1 (Not released) 138
Changes in MySQL Shell 8.0.0 (2017-07-14, Development Milestone) 138

Preface and Legal Notices

This document contains release notes for the changes in each release of MySQL Shell 8.0.

Legal Notices

Copyright © 1997, 2025, Oracle and/or its affiliates.

License Restrictions

This software and related documentation are provided under a license agreement containing restrictions on use and disclosure and are protected by intellectual property laws. Except as expressly permitted in your license agreement or allowed by law, you may not use, copy, reproduce, translate, broadcast, modify, license, transmit, distribute, exhibit, perform, publish, or display any part, in any form, or by any means. Reverse engineering, disassembly, or decompilation of this software, unless required by law for interoperability, is prohibited.

Warranty Disclaimer

The information contained herein is subject to change without notice and is not warranted to be error-free. If you find any errors, please report them to us in writing.

Restricted Rights Notice

If this is software, software documentation, data (as defined in the Federal Acquisition Regulation), or related documentation that is delivered to the U.S. Government or anyone licensing it on behalf of the U.S. Government, then the following notice is applicable:

U.S. GOVERNMENT END USERS: Oracle programs (including any operating system, integrated software, any programs embedded, installed, or activated on delivered hardware, and modifications of such programs) and Oracle computer documentation or other Oracle data delivered to or accessed by U.S. Government end users are "commercial computer software," "commercial computer software documentation," or "limited rights data" pursuant to the applicable Federal Acquisition Regulation and agency-specific supplemental regulations. As such, the use, reproduction, duplication, release, display, disclosure, modification, preparation of derivative works, and/or adaptation of i) Oracle programs (including any operating system, integrated software, any programs embedded, installed, or activated on delivered hardware, and modifications of such programs), ii) Oracle computer documentation and/or iii) other Oracle data, is subject to the rights and limitations specified in the license contained in the applicable contract. The terms governing the U.S. Government's use of Oracle cloud services are defined by the applicable contract for such services. No other rights are granted to the U.S. Government.

Hazardous Applications Notice

This software or hardware is developed for general use in a variety of information management applications. It is not developed or intended for use in any inherently dangerous applications, including applications that may create a risk of personal injury. If you use this software or hardware in dangerous applications, then you shall be responsible to take all appropriate fail-safe, backup, redundancy, and other measures to ensure its safe use. Oracle Corporation and its affiliates disclaim any liability for any damages caused by use of this software or hardware in dangerous applications.

Trademark Notice

Oracle, Java, MySQL, and NetSuite are registered trademarks of Oracle and/or its affiliates. Other names may be trademarks of their respective owners.

Intel and Intel Inside are trademarks or registered trademarks of Intel Corporation. All SPARC trademarks are used under license and are trademarks or registered trademarks of SPARC International, Inc. AMD, Epyc, and the AMD logo are trademarks or registered trademarks of Advanced Micro Devices. UNIX is a registered trademark of The Open Group.

Third-Party Content, Products, and Services Disclaimer

This software or hardware and documentation may provide access to or information about content, products, and services from third parties. Oracle Corporation and its affiliates are not responsible for and expressly disclaim all warranties of any kind with respect to third-party content, products, and services unless otherwise set forth in an applicable agreement between you and Oracle. Oracle Corporation and its affiliates will not be responsible for any loss, costs, or damages incurred due to your access to or use of third-party content, products, or services, except as set forth in an applicable agreement between you and Oracle.

Use of This Documentation

This documentation is NOT distributed under a GPL license. Use of this documentation is subject to the following terms:

You may create a printed copy of this documentation solely for your own personal use. Conversion to other formats is allowed as long as the actual content is not altered or edited in any way. You shall not publish or distribute this documentation in any form or on any media, except if you distribute the documentation in a manner similar to how Oracle disseminates it (that is, electronically for download on a Web site with the software) or on a CD-ROM or similar medium, provided however that the documentation is disseminated together with the software on the same medium. Any other use, such as any dissemination of printed copies or use of this documentation, in whole or in part, in another publication, requires the prior written consent from an authorized representative of Oracle. Oracle and/or its affiliates reserve any and all rights to this documentation not expressly granted above.

Documentation Accessibility

For information about Oracle's commitment to accessibility, visit the Oracle Accessibility Program website at
http://www.oracle.com/pls/topic/lookup?ctx=acc&id=docacc.

Access to Oracle Support for Accessibility

Oracle customers that have purchased support have access to electronic support through My Oracle Support. For information, visit
http://www.oracle.com/pls/topic/lookup?ctx=acc&id=info or visit `http://www.oracle.com/pls/topic/lookup?ctx=acc&id=trs` if you are hearing impaired.

Changes in MySQL Shell 8.0.43 (2025-07-22, General Availability)



Note

These release notes were created with the assistance of MySQL HeatWave GenAI.

- Utilities Bugs Fixed
- Bugs Fixed

Utilities Bugs Fixed

- util.dumpInstance returned the following error when run against MySQL HeatWave DB Systems:

```
ERROR: User 'admin'@'%' is granted restricted privilege: OPTION_TRACKER_OBSERVER
(fix this with 'strip_restricted_grants' compatibility option)
```

  The list of privileges has been updated to include `OPTION_TRACKER_OBSERVER`. (Bug #37958876)

- MySQL Shell logging has been improved for `LOAD DATA` warnings generated by util.importTable and util.loadDump. Previously, `LOAD DATA` warnings were printed to the terminal and were difficult to locate in the log. For example:

```
Warning: schema@table@123.tsv.zst error 1062: Duplicate entry
'1234567' for key 'table.PRIMARY'
```

  These messages have been improved, making them easier to find in the log. For example:

```
Warning: An error has been reported while loading data into
`schema`.`table` from 'schema@table@123.tsv.zst' file, error 1062:
Duplicate entry '1234567' for key 'table.PRIMARY'
```

  (Bug #37800574)

- The MySQL REST Service-specific account, `ocirest`, is automatically excluded when dumping with `ocmids:true`, or loading into MySQL HeatWave DB Systems. (Bug #37792183)

Bugs Fixed

- Kerberos authentication is now supported on macOS. The `authentication_kerberos_client` plugin is now bundled on macOS installations of MySQL Shell. (Bug #37777584)

Changes in MySQL Shell 8.0.42 (2025-04-15, General Availability)

- Utilities Added or Changed Functionality
- Utilities Bugs Fixed
- Bugs Fixed

Utilities Added or Changed Functionality

- The `mysql_option_tracker_persister` role is now excluded from the dump when `ocimds: true` and is excluded when loading a dump into a MySQL HeatWave DB System. (Bug #37457569)

Utilities Bugs Fixed

- Dump and load operations failed randomly due to a bug in the bundled Curl package.

Curl is upgraded to 8.12.1 in this release. (Bug #37576066)

Bugs Fixed

- Under certain circumstances MySQL Shell could crash during auto-completion on options such as `shell.options.history.auto`. (Bug #37528585)

Changes in MySQL Shell 8.0.41 (2025-01-21, General Availability)

- Utilities Added or Changed Functionality
- Utilities Bugs Fixed
- Functionality Added or Changed
- Bugs Fixed

Utilities Added or Changed Functionality

- The schema `MYSQL_AUTOPILOT` is excluded by dump and load operations with `ocimds:true`. (Bug #37278169)
- As of this release, the upgrade checker utility writes all compatibility issues and fixes to the log file, instead of only writing to the console. (Bug #37154456)

Utilities Bugs Fixed

- If `convertBsonTypes` was enabled, the JSON import utility failed when importing negative BSON values.

  An error similar to the following was returned:

  ```
  ValueError: Unexpected data, expected to find an integer string processing extended JSON for $nu
  ```

  (Bug #37243264)
- Under certain circumstances, such as a large amount of data chunking, the `maxRate` dump option did not properly limit the throughput due to a gap between the start of the dump and the start of the data dump. As of this release, `maxRate` is used only when data is being dumped. (Bug #37216767)
- MySQL Shell could hang when running a dump with `consistent: true` under an account which lacked privileges to execute `FLUSH TABLES WITH READ LOCK`.

  As of this release, query events are checked only if they contain data, and `GRANT` and `REVOKE` statements are flagged as unsafe. (Bug #37158908)

Functionality Added or Changed

- As of this release, the Google V8 JavaScript engine is replaced by Oracle GraalVM. (Bug #34370637)

Bugs Fixed

- In MySQL Shell 8.0.40, RPM installation failed on Oracle Linux 8 due to a dependency on Python 3.9. As of this release, MySQL Shell bundles Python 3.13. (Bug #37479400)

- It was not possible to start MySQL Shell on Oracle Linux 8 running on ARM platforms if `PAGE_SIZE` was set to 64K. An error similar to the following was displayed:

```
mysqlsh: error while loading shared libraries:
libantlr4-runtime.so.4.10.1: ELF load command alignment
not page-aligned
```

(Bug #36792750)

Changes in MySQL Shell 8.0.40 (2024-10-15, General Availability)

- Utilities Added or Changed Functionality
- Utilities Bugs Fixed
- Bugs Fixed

Utilities Added or Changed Functionality

- As of this release, the `MYSQL_OPTION` schema is excluded by the dump utilities when `ocimds: true` and is automatically excluded when loading a dump into a MySQL HeatWave DB System. (Bug #37023079)

Utilities Bugs Fixed

- Under certain circumstances, using zstd compression, the dump utilities could generate corrupted data files. (Bug #36836188)
- Attempting to run the dump or copy utilities from MySQL Shell 8.0.x against a more recent version of the server, such as 8.4.0, could result in a syntax error.

  Many breaking changes have been made to MySQL syntax and configuration between 8.0.37 and 8.4.x, and higher, such as replacing `SHOW MASTER STATUS` with `SHOW BINARY LOG STATUS`, for example. There were also many removals. See the release notes for those server versions for more information.

  As of this release, the dump and copy utilities raise an error when such incompatibilities are detected and recommend the appropriate MySQL Shell upgrade.

  

  Important

  It is always recommended to use the latest version of MySQL Shell.

  (Bug #36701854)

Bugs Fixed

- MySQL Shell failed to start if installed by MSI on Microsoft Windows 11 with Visual Studio Redistributable version 14.3x or lower. On Windows platforms, MySQL Shell requires Visual Studio Redistributable version 14.4x or higher.

  See Microsoft Visual C++ Redistributable latest supported downloads. (Bug #37049411)

- The MySQL Shell MSI progress dialogs displayed numbered placeholders instead of the installation values. (Bug #37033676)

- When running MySQL Shell over SSH, if a command was entered on the command line, but not executed, closing the SSH session could result in the command being executed without user input. (Bug #36861912)
- The URI parser threw an exception if the URI contained an unescaped @ character. (Bug #36105235)

Changes in MySQL Shell 8.0.39 (Not released, General Availability)

Version 8.0.39 has no release notes, or they have not been published because the product version has not been released.

Changes in MySQL Shell 8.0.38 (2024-07-01, General Availability)

- AdminAPI Bugs Fixed
- Utilities Bugs Fixed
- Bugs Fixed

AdminAPI Bugs Fixed

- MySQL Shell closed unexpectedly when calling certain AdminAPI functions on EL7 platforms. (Bug #36651010)
- `dba.reboot_cluster_from_complete_outage()` disabled `super_read_only` on the primary member of an INVALIDATED Cluster. As a result, clients continued to perform updates and introduce errant transactions.

  As of this release, `dba.reboot_cluster_from_complete_outage()` enables `super_read_only` on the primary member and disables the Group Replication action `mysql_disable_super_read_only_if_primary`. (Bug #36562916)
- If an attempt to create a Replica Cluster failed due to a timeout and the revert also failed due to a timeout, the Replica Cluster could be left in an inconsistent state; ONLINE, but not associated with the ClusterSet's metadata. This specific issue was caused by low values for `wait_timeout` and `interactive_timeout`.

  The following changes were made:

  - `wait_timeout` is checked and, if set to a value lower than the default of 8 hours, is set to 8 hours.
  - *`Cluster`*`.rescan()` is extended with a new option, `repairMetadata` which can be enabled to resolve inconsistencies in the Cluster's metadata.
  - *`Cluster`*`.dissolve()` can now be used on Clusters in this inconsistent state.

  (Bug #36495756)

Utilities Bugs Fixed

- Primary keys defined on an `ENUM` column were reported as missing for dumps with `ocimds:true`. This was caused by a fix in an earlier version which instructed the dump utility to ignore primary keys or unique indexes which contain one or more `ENUM` columns when selecting an index for chunking.

  As of this release, information about the index selected for chunking and whether the table has a primary key is separated. (Bug #36493316)

References: See also: Bug #35180061.

Bugs Fixed

- MySQL Shell did not prompt for a password if `-p` was specified on the command line without an argument. (Bug #36433418)
- Upgrading MySQL Shell 8.0.35, or higher, on Windows platforms, resulted in multiple installations instead of overwriting the existing installation. (Bug #113732, Bug #36259270)

Changes in MySQL Shell 8.0.37 (2024-04-30, General Availability)

- AdminAPI Added or Changed Functionality
- AdminAPI Bugs Fixed
- Utilities Bugs Fixed
- Functionality Added or Changed

AdminAPI Added or Changed Functionality

- Cloning version compatibility checks for donor and recipient instances are relaxed. As of this release, with certain conditions, only the major and minor version numbers need to match, the patch number is now disregarded.

  The following conditions apply:

  - Only version 8.0.17, or higher, can perform cloning.
  - If both versions are 8.0.37, or higher, only the major and minor versions are required to match.
  - If the version is 8.0.17, or higher, and less than 8.0.37, major, minor, and patch numbers must match.

  (Bug #36054489)

AdminAPI Bugs Fixed

- The documentation for Rescanning a Cluster did not make clear that while `group_replication_transaction_size_limit` is set to the maximum value in Replica Clusters, the original value is stored in the metadata schema and is restored by *`Cluster`*`.rescan()` in the event of a switchover or failover. This overwrites any user-defined value set on the Replica Cluster.

  The documentation is updated with this information. (Bug #36494958)

- If the primary instance of a Replica Cluster was changed, attempting to remove that Cluster from the Cluster set failed with the following error:

  ```
  ERROR: Error enabling automatic super_read_only management at secondary:port:
  MySQL Error 3910 (HY000): The function 'group_replication_enable_member_action' failed.
  Member must be the primary or OFFLINE.
  ```

  (Bug #36400360)

- `dba.createReplicaSet` with `adoptFromAR:true` could fail if the host and port values returned were not properly configured on the target instance. The error returned did not provide useful information.

As of this release, if the target instance does not have properly configured host and port values, it is ignored and the user is informed. (Bug #36201015)

Utilities Bugs Fixed

- The dump utilities included the MySQL HeatWave Service-reserved username `oracle-cloud-agent` resulting in the following error:

```
User 'oracle-cloud-agent'@'localhost' is using an unsupported
authentication plugin 'auth_socket' (fix this with 'skip_invalid_accounts' compatibility option)
```

  The following users are now excluded when loading to, or dumping from, an MySQL HeatWave Service instance:

  - ocidbm
  - oracle-cloud-agent
  - rrhhuser

  (Bug #36159820)

- Loading a dump on Windows platforms failed if `sql_mode` was set to `STRICT_ALL_TABLES`. The following error was returned:

```
ERROR 1231 (42000): Variable 'wait_timeout' can't be set to the value of '31536000'
```

  The load utility attempted to set a maximum value for `wait_timeout` which is not permitted on Windows platforms. (Bug #36119568)

- Under certain circumstances, the Upgrade Checker utility's reserved keywords check did not generate warnings for the `FULL` and `INTERSECT` keywords. (Bug #114423, Bug #36424093)

- The upgrade checker utility did not check for all old temporal types. Under certain circumstances, this could result in an upgrade failure. (Bug #112991, Bug #36029331)

Functionality Added or Changed

- The V8 JavaScript engine used by MySQL Shell was updated to version 12.0.267.8. (WL #15948)

Changes in MySQL Shell 8.0.36 (2024-01-16, General Availability)

- AdminAPI Bugs Fixed
- Utilities Added or Changed Functionality
- Utilities Bugs Fixed

AdminAPI Bugs Fixed

- *`Cluster`*`.rescan()` did not correctly handle missing recovery account users. If it encountered a user with an unexpected format, and the correct format was missing, it did not attempt to create the correct user.

  Errors similar to the following were logged by the *`Cluster`*`.status()` command:

```
WARNING: Incorrect recovery account (mysql_innodb_cluster_3337079193) being used. Use Cluster.rescan() to re
```

As of this release, `Cluster.rescan()` creates the missing user. (Bug #35828910)

- Under certain circumstances, `dba.rebootClusterFromCompleteOutage()` failed with malformed GTID errors relating to `GROUP_CONCAT`. `dba.rebootClusterFromCompleteOutage()` must query the complete GTID set of the channel and this query failed if the default `GROUP_CONCAT_MAX_LEN` value was too low.

  As of this release, queries which do not require the `GROUP_CONCAT` function, do not use it and queries which require it, use a `GROUP_CONCAT_MAX_LEN` value of 1GB. (Bug #35356006)

Utilities Added or Changed Functionality

- As of this release, `util.loadDump()` and `util.copyInstance()` automatically exclude the `mysql_audit` and `mysql_firewall` schemas if the target is a MySQL HeatWave Service DB System. (Bug #35830920)

Utilities Bugs Fixed

- `util.loadDump()` ignored leading zeroes (0) in S3 bucket prefix names. (Bug #36041691)
- It was not possible to migrate from a compatible database using the `copyInstance()` or `dumpInstance()` utilities. Errors similar to the following error were returned:

```
Util.copyInstance: Unknown system variable 'activate_all_roles_on_login' (MYSQLSH 1193)
```

(Bug #35963431)

Changes in MySQL Shell 8.0.35 (2023-10-25, General Availability)

- AdminAPI Bugs Fixed
- Utilities Added or Changed Functionality
- Utilities Bugs Fixed
- Bugs Fixed

AdminAPI Bugs Fixed

- Using `-- clusterset listRouters` on the command line, without providing a parameter for `listRouters`, resulted in the following error in MySQL Shell 8.0.34:

```
        ERROR: Argument #1 is expected to be a string
```

(Bug #35747208)

References: This issue is a regression of: Bug #35068427.

Utilities Added or Changed Functionality

- Instance dump utility now excludes the `mysql_firewall` schema if `ocimds: true`. (Bug #35805866)

- MySQL Shell was updated for compatibility with the privilege changes made in MySQL HeatWave Service.

  The following privileges were added to MySQL HeatWave Service:

  - `AUDIT_ADMIN`
  - `BACKUP_ADMIN`
  - `FLUSH_OPTIMIZER_COSTS`
  - `FLUSH_STATUS`
  - `FLUSH_TABLES`
  - `FLUSH_USER_RESOURCES`
  - `ROLE_ADMIN`

  The following privileges were removed from MySQL HeatWave Service

  - `RESOURCE_GROUP_ADMIN`
  - `RESOURCE_GROUP_USER`

  For more information on MySQL HeatWave Service privileges, see Default MySQL Privileges. (Bug #35668544)

- Operations resulting in the curl errors `52: CURLE_GOT_NOTHING` and `56: CURLE_RECV_ERROR` are retried for all supported cloud vendors, for all utilities which support them. (Bug #35659057)

- MySQL Shell dump and load utilities now support the Oracle Cloud Infrastructure Object Storage Dedicated Endpoints format for Pre-Authenticated Request (PAR) URLs:

  ```
  namespace.objectstorage.region.oci.customer-oci.com/.../
  ```

  For more information, see OCI Object Storage Dedicated Endpoints.

  MySQL Shell continues to support the legacy PAR URL format:

  ```
  objectstorage.region.oraclecloud.com/.../
  ```

  (Bug #35548572)

- `util.checkForServerUpgrade()` has been updated to check for columns which have foreign keys referencing columns in tables using different database storage engines, such as MyISAM. (Bug #35155064)

Utilities Bugs Fixed

- Under certain circumstances, when loading a file larger than `maxBytesPerTransaction`, (or 1.5 * `bytesPerChunk` if `maxBytesPerTransaction` was not used) a memory leak could occur. (Bug #35600174)

- As of MySQL Server 8.1.0, MySQL HeatWave Service contains a schema named `mysql_audit`. As a result, the dump and load utilities encountered a duplicate object error when copying data from one DB System to another.

As of this release, with the `ocimds` option enabled, `dumpInstance()` automatically excludes the `mysql_audit` schema. (Bug #35550282)

- If an AWS HEAD request failed with an authorization error, it was not retried. As of this release if such a request fails with a `400 HTTP` error, it is retried.

  Additionally, if the refresh process for AWS credentials has a defined expiration time, the refresh process is triggered 5 minutes before the required time. (Bug #35468541)

- `util.debug.collectDiagnostics()` failed if run on an InnoDB Cluster created with an older version of MySQL Shell. An error similar to the following was generated:

  ```
  An error occurred during data collection. Partial output deleted.
  debug.collectDiagnostics: ClassicSession.run_sql: Table
  'mysql_innodb_cluster_metadata.v2_cs_clustersets' doesn't exist (MySQL Error 1146)
  ```

  As of this release, `util.debug.collectDiagnostics()` collects diagnostics information even if tables are missing. (Bug #35468106)

- `util.debug.collectDiagnostics()` threw an exception if the server was configured to write error logs to stderr. For example, if `mysqld` was started with `--console`. (Bug #35318770)

- The MySQL Shell upgrade checker utility flagged views as corrupt in MySQL 5.7 versions up to 5.7.39. This issue occurred for views whose `from` clause contained a table schema prefix with a `group by` clause. (Bug #111813, Bug #35635009)

- The utility, `checkForServerUpgrade`, did not recognize `INTERSECT` as a reserved word.

  `INTERSECT` was reserved in MySQL Server 8.0.31. (Bug #110824, Bug #35335813)

Bugs Fixed

- The Windows installer for MySQL Shell upgrades did not recognize the existing MySQL Shell installation, overwrote that installation, and created a separate uninstall entry in Add/Remove Programs without removing the existing entry. (Bug #35869936)

- Errors generated by stored procedures were not returned over classic MySQL protocol connections. (Bug #35549008)

Changes in MySQL Shell 8.0.34 (2023-07-18, General Availability)

- AdminAPI Bugs Fixed
- Utilities Bugs Fixed
- Functionality Added or Changed
- Bugs Fixed

AdminAPI Bugs Fixed

- Rebooting a replica cluster from complete outage could result in the cluster rejoining the ClusterSet, but not the instance members of the replica cluster. This happened only if the primary cluster was under heavy load or the replica cluster was missing transactions from the ClusterSet. (Bug #35444244)

- If the X Protocol port was changed for a cluster member and that member restarted, the AdminAPI did not update the metadata with the new port number, leading to connection errors and so on.

As of this release, *`cluster`*`.status()` checks for port changes and *`cluster`*`.rescan()` updates the metadata with the new port number. (Bug #35410360)

- During a failover of a ClusterSet replication channel, the *`ClusterSet`*`.status()` value `clusterSetReplicationStatus` reported `ERROR` and `globalStatus` returned `OK_NOT_REPLICATING`. Errors and warnings relating to misconfigured or stopped channels were also returned. These statuses and errors were misleading as the channel was attempting to connect to another source or replica.

  As of this release, `clusterSetReplicationStatus` returns `CONNECTING`, and `globalStatus` returns `OK` while a channel connection attempt is ongoing. If there is an error, it is ignored until the channel state updates to either ON or OFF.

  Additionally, the `ReplicaSet.status()` field, `status`, also returns `CONNECTING`. (Bug #34614769)

Utilities Bugs Fixed

- Under certain circumstances, `util.loadDump()` could fail when retrieving a file from AWS S3, Oracle Cloud Infrastructure Object Storage, or Azure Blob Storage, even though the file was downloadable by other means.

  As of this release, if CURL errors occur, such as `52: CURLE_GOT_NOTHING`. `56 CURLE_RECV_ERROR`, or `28: CURLE_OPERATION_TIMEDOUT` `utils.loadDump()` retries the download. (Bug #35362775, Bug #35392531)

- If an exception occurred while importing a single, uncompressed file with `util.import_table()`, MySQL Shell crashed. (Bug #35313366)

- The upgrade checker utility did not check stored procedures and routines for the deprecated qualifier syntax *`.tbl_name`*. (Bug #35046623)

- If chunking was enabled for a dump of tables, but the primary key or unique index used to chunk the table contained an `ENUM` column, some of the tables rows were not exported to the dump. This occurred if the `ENUM` column's values were not ordered alphabetically.

  As of this release, primary keys or unique indexes which contain one or more `ENUM` columns, are ignored when selecting an index for chunking. (Bug #110352, Bug #35180061)

Functionality Added or Changed

- MySQL Shell now supports the `--loose` prefix.

  For more information on this prefix, see Program Option Modifiers. (Bug #110141, Bug #35112454)

Bugs Fixed

- The MySQL configuration utility `mysql_config_editor` was not bundled with MySQL Shell 8.0.33. (Bug #35459202)

  References: This issue is a regression of: Bug #34097411.

- MySQL Shell command line did not correctly handle missing optional arguments. A NULL value was used instead of a valid value, resulting in an error. (Bug #109827, Bug #35068427)

Changes in MySQL Shell 8.0.33 (2023-04-18, General Availability)

- AdminAPI Added or Changed Functionality
- AdminAPI Bugs Fixed
- Functionality Added or Changed
- Bugs Fixed

AdminAPI Added or Changed Functionality

- **Important Change:** It is now possible to configure Clusters, ClusterSets, and ReplicaSets to use SSL for:
  - Encrypting Group Replication and asynchronous replication channels.
  - Enabling replicas to verify the identity of the source.
  - Enabling replicas to use client SSL certificates for authentication.

  The following changes were made:
  - `dba.createCluster()` was extended with the following options:
    - `memberAuthType`
    - `certIssuer`
    - `certSubject`
  - `dba.createReplicaSet()` was extended with the following options:
    - `memberAuthType`
    - `certIssuer`
    - `certSubject`
    - `replicationSslMode`
  - `cluster.addInstance()` was extended with the following option:
    - `certSubject`
  - `clusterSet.createReplicaCluster()` was extended with the following option:
    - `certSubject`
  - The `cluster.createClusterSet()` option `clusterSetReplicationSslMode` was extended with the following values:
    - `VERIFY_CA`
    - `VERIFY_IDENTITY`
  - A new MySQL Shell option is added, `dba.connectivityChecks: true | false`. This option defines if connectivity checks are performed for *`cluster`*`.addInstance()`, *`clusterSet`*`.createReplicaCluster()`, and *`replicaSet`*`.addInstance()`, using the defined SSL configuration.

If an SSL error occurs, the command stops and an error is returned.

- *cluster*.options() and *replicaSet*.options() were extended to show relevant information on memberAuthType and certIssuer in the globalOptions section, and certSubject for each instance in the topology section. *cluster*.options() is also updated to list clusterSetReplicationSslMode in the globalOptions section.
- *replicaSet*.status is extended with a new field, replicationSsl, to show the SSL information for each replication channel.



Note

If the cluster's memberAuthMode is set to any value other than password, *cluster*.rescan() will fail if used with addInstances

For more information on these options, see the MySQL Shell 8.0 JavaScript API Reference or MySQL Shell 8.0 Python API Reference available from MySQL Documentation. (Bug #34256928, WL #13688)

- **Important Change:** Cluster and ReplicaSet operations setupAdminAccount() and setupRouterAccount() are extended to enable authentication with client SSL certificates. The following changes were made:
  - The following options were added to setupAdminAccount() and setupRouterAccount():
    - requireCertIssuer: Optional SSL certificate issuer for the account.
    - requireCertSubject: Optional SSL certificate subject for the account.
    - passwordExpiration: numberOfDays | Never | Default: Password expiration setting for the account.
  - setupAdminAccount() and setupRouterAccount() were also added to the ClusterSet object.
  - The following options were added to dba.configureInstance() and dba.configureReplicaSetInstance():
    - clusterAdminCertIssuer: Optional SSL certificate issuer for the account.
    - clusterAdminCertSubject: Optional SSL certificate subject for the account.
    - clusterAdminPasswordExpiration: NumberOfDays | NULL | DEFAULT | NEVER: Password expiration setting for the account.

  For more information on these options, see the MySQL Shell 8.0 JavaScript API Reference or MySQL Shell 8.0 Python API Reference available from MySQL Documentation. (WL #15438)
- It is now possible to set group_replication_paxos_single_leader using MySQL Shell.



Note

This can only be set by MySQL Shell on MySQL Server 8.0.31, or higher, because MySQL Shell requires the information provided by WRITE_CONSENSUS_SINGLE_LEADER_CAPABLE in the

`replication_group_communication_information` table, which was introduced in MySQL 8.0.31.

The following changes were made:

- `dba.createCluster()` and *`ClusterSet`*`.createReplicaCluster()` now support enabling or disabling `group_replication_paxos_single_leader`.
- `dba.rebootClusterFromCompleteOutage()` now supports changing the value of `group_replication_paxos_single_leader`.
- *`Cluster`*`.options()`, *`Cluster`*`.status({extended: 1})`, and *`ClusterSet`*`.status({extended: 2})` lists the value of `group_replication_paxos_single_leader` in use.
- `group_replication_paxos_single_leader` is generated and set automatically when adding or rejoining members to Clusters.

(Bug #33930873)

- The locking functionality introduced in MySQL Shell 8.0.20 to prevent conflicting ReplicaSet operations running concurrently has been extended to include Cluster and ClusterSet resources.

  For information on locking types and the operations which require them, see MySQL AdminAPI.

  

  Note

  The existing documentation on ReplicaSet locking has been moved to MySQL AdminAPI.

  (Bug #25803949, Bug #33250135, WL #11969)

AdminAPI Bugs Fixed

- After an upgrade, *`Cluster`*`.status` displayed the following error, under `instanceErrors`:

  `"WARNING: Incorrect recovery account (mysql_innodb_cluster_r`*`Server_ID`*`) being used. Use Cluster.rescan() to repair."`

  This warning was displayed in error, as a result of a previous bug fix which introduced a check on recovery account formats. The recovery account format was expected to be of the form `mysql_innodb_cluster_`*`server_id`* but, for an upgrade, the recovery account format is `mysql_innodb_cluster_r`*`server_id`*.

  As of this release, the recovery account format check has been updated to recognize older formats. (Bug #35046654)

  References: See also: Bug #33235502.

- Python commands, such as *`Cluster`*`.add_instance()` did not check the Cluster was connected before proceeding. This could result in MySQL Shell closing unexpectedly.

  As of this release, Python commands check the cluster is connected before proceeding. (Bug #35046432)

- *`Cluster`*`.status()` and *`Cluster`*`.rescan()` did not detect a mismatch of values between the `group_replication_view_change_uuid` stored in the metadata and the current runtime value.

This could lead to errors during operations such as *`ClusterSet`*`.setPrimaryCluster` because the reconciliation of transaction sets cannot be done if the current UUID does not match the value stored in the ClusterSet metadata.

As of this release, *`Cluster`*`.status()` raises a warning if the `group_replication_view_change_uuid` values do not match and *`Cluster`*`.rescan()` also detects the mismatch and updates the metadata accordingly. (Bug #35000998)

- `createReplicaCluster()` failed with the error, `ERROR: Error creating Replica Cluster: MySQL Error 1772 (HY000): Malformed GTID set`. This was caused by the `sql_mode` including `NO_BACKSLASH_ESCAPES`. (Bug #34837601)

- `removeCluster()` failed when attempting to remove an offline cluster from a ClusterSet, if group replication was attempting to bring the cluster back online, even if the `force` option was used. This issue could leave the Cluster metadata in a changed state.

  As of this release, `removeCluster()` stops group replication and ignores errors if the `force` option is used. The rollback process was improved and no longer depends on group replication being online. Instead, snapshots are taken of server data and are reapplied when necessary. (Bug #34693099)

- `dba.checkInstanceConfiguration()` did not properly take into account value-less variables, such as `disable_log_bin`, and listed them as requiring changes although they were correctly configured. (Bug #34569971)

- It is not possible to create sandboxes using older versions of MySQL 5.7, such as 5.7.21, due to incompatibilities between the supported versions of TLS. Newer versions of MySQL Shell do not support TLS v1.1 and older.

  As of this release, sandbox management commands which open local connections to the sandboxes, fall back to unencrypted connections if an SSL connection fails. (Bug #34548702)

- A replica's `globalStatus` value, returned by `ClusterSet.status()`, was incorrectly reported as `NOT_OK` if the ClusterSet's primary cluster was unreachable, but the replica was functioning normally.

  As of this release, the `globalStatus` value `OK_NOT_REPLICATING` is returned if the replica is functioning normally but not replicating because the primary is offline or otherwise unreachable. (Bug #34324165)

- It was not possible to upgrade the metadata schema of a cluster created by MySQL Shell 1.0.x with `adoptFromGR: true`. The following error was returned:

  ```
  "ERROR: Truncated incorrect INTEGER value: 'true'"
  ```

  (Bug #31711835)

- MySQL Shell disabled and persisted `offline_mode` when an instance was added or rejoined to a Cluster, or when rebooted. If this variable was enabled explicitly by the user, it was overwritten by MySQL Shell.

  As of this release, `offline_mode` is disabled globally, not persisted, and a new warning is added to *`Cluster`*`.status` to inform the user of the risks of enabling this variable. (Bug #108905, Bug #34778797)

- When adding a member to a cluster, the values of `auto_increment_increment` and `auto_increment_offset` were miscalculated and were off by 1. As a result, when adding or rejoining an eighth member to a cluster, the new member was configured with the correct values, while the

other seven members continued with the configuration of a seven-member cluster. (Bug #108759, Bug #34711038)

Functionality Added or Changed

- If an AWS request fails with a `HTTP 400` error and the reported error is `ExpiredToken` or `TokenRefreshRequired`, the request is silently retried if the authentication can be refreshed. (Bug #35027093)
- It is now possible to use a non-default location for the OCI CLI config file used by the `authentication_oci_client` authentication plugin, using `shell.options[oci.configFile]` to define the alternate location. (Bug #34858763)
- MySQL Shell now bundles the following client authentication plugins:
  - `authentication_fido_client`
  - `authentication_kerberos_client`
  - `authentication_ldap_sasl_client`

  These plugins are located in the `/lib/mysql/plugins` directory of your MySQL Shell installation. (Bug #34857426)
- MySQL Shell now supports AWS `credential_process` external credential provider.

  For more information, see S3-compatible Storage. (Bug #34840953)
- The Dump Loading utility (`loadDump()`) progress file now includes information on index creation status. (Bug #34840760)

Bugs Fixed

- MySQL Shell could not connect if both port and socket file were defined in the configuration file. As of this release, the last value defined takes precedence. (Bug #35023480)
- The `--user`/`-u` and `--password`/`-p` options were not recognized if they were specified before the URI in the connection string.

  They were recognized if they were specified after the URI. (Bug #35020175)
- It was not possible to run the load or dump utilities over an SSH connection. The operation failed with an error similar to the following:

```
Util.dumpSchemas: The option ssh-remote-port '33060' is
already defined.(ArgumentError)
```

  (Bug #35018617)
- The import table utility ignored the `skipRows` parameter when importing a single compressed file or multiple files.

  In addition, the documentation of `skipRows` was updated to clarify that it skips the defined number of lines from each file imported. (Bug #35018278)
- If two different plugins defined an `object.function` with the same name for both the object and function, the MySQL Shell help displayed the same help data for both, even if the functions had different signatures and description. (Bug #34979347)

- The `dumpInstance()` utility incorrectly commented out database-level `GRANT` statements referring to schema names with wildcard characters.

  As of this release, `GRANT` statement filtering has been removed for both dump and load utilities. Dump utilities print a warning if a `GRANT` on an excluded object is detected.

  The following additional, supporting changes were made:

  - Dumping with `ocimds: true`, all database level grants containing wildcards, are reported as errors. DB Systems use `partial_revokes=ON` which treats wildcard characters literally. These errors must be corrected manually.
  - A new compatibility option was added to the dump utilities, `ignore_wildcard_grants`. If enabled, ignores errors from grants on schemas with wildcards, which are interpreted differently in systems where the `partial_revokes` system variable is enabled.
  - A new option, `handleGrantErrors`, was added to `util.loadDump()`. This option defines the action taken in the event of errors related to `GRANT` or `REVOKE` errors.
    - `abort`: (default) stops the load process and displays an error.
    - `drop_account`: deletes the account and continues the load process.
    - `ignore`: ignores the error and continues the load process.

  (Bug #34952027)

- The `defaultValue` option of `shell.prompt` did not return the defined value. (Bug #34889112)
- The Upgrade Checker utility did not raise a warning for indexes which were too large. Tables with the `compact` or `redundant` row formats are not permitted to have indexes larger than 767 bytes in MySQL 8.0. As a result, such tables were inaccessible after upgrading to MySQL 8.0.

  As of this release, the Upgrade Checker checks for such index sizes on specific row formats and raises a warning if the can cause tables to be inaccessible after upgrade. (Bug #34826890)

- MySQL Shell running on Oracle Cloud Infrastructure Cloud Shell closed unexpectedly when attempting to load a dump from an Object Storage bucket while FIPS mode was enabled. The following error was displayed:

  ```
  md5_dgst.c(82): OpenSSL internal error, assertion failed:
  Digest MD5 forbidden in FIPS mode!
  ```

  (Bug #34787908)

- It was not possible to export a view with the Export Table utility (`exportTable()`. (Bug #34663934)
- If the Dump Loading utility was run with `"deferTableIndexes": "all"` and `"ignoreExistingObjects": true`, the load could fail as the utility tried to add indexes which already existed.

  As of this release, if `"ignoreExistingObjects": true`, existing tables are checked for indexes and only missing indexes are added. (Bug #34566034)

- The following issues occurred in the import table utility:
  - The reported progress exceeded 100% when importing compressed files.

  - The utility reported multiple threads in use when importing a single compressed file, although only one thread was used.
  - The utility reported an incorrect number of chunks and imported files.

  (Bug #33970577)

- The dictionary used as the Python wrapper for a map did not pass the `isinstance()` test.

  As of this release, the implementation of the map wrapper used by MySQL Shell in Python mode has been updated and now inherits from Python's `dict`.To avoid duplication of data and ensure synchronization between Python's context and internal representation of data, data is not stored in the base class, but in the map wrapper. Due to this, and the fact that data is shared between Python and JavaScript modes, it is only safe to store basic types such as numbers, strings, arrays, and dictionaries, and native Shell objects such as Cluster and Session. As the base class is not used to store the data, all methods of the list class have been reimplemented.

  Be aware of the following limitations:

  - Order of insertion is not maintained; keys are always ordered alphabetically.
  - Only string types keys are supported; non-string keys are handled as follows:
    - converted to a string using `str()` when inserting multiple values.
    - treated as missing when accessing an element.
    - treated as an error, with an exception raised, when setting a value.

  (Bug #33517575)

- The MySQL Shell source compilation documentation, `INSTALL`, did not include the `antlr4` runtime, which is a mandatory dependency.

  Thanks to Evgeniy Patlan for this contribution. (Bug #109909, Bug #35045019)

- The Dump Loading utility failed to load a dump using `"deferTableIndexes": "all"` if one of the tables being loaded contained multiple indexes and one, or more, index specified for an `AUTO_INCREMENT` column. (Bug #109313, Bug #34876423)

- The `dumpInstance()` utility incorrectly commented out `GRANT` statements referring to existing views, routines, and non-existing schemas and objects instead of only commenting out `GRANT` statements which referred to objects not included in the dump.

  As of this release, `GRANT` statement filtering has been removed for both dump and load utilities. Dump utilities now detect invalid grants and print a warning if they are detected.

  A new compatibility option is introduced, `strip_invalid_grants`, which removes invalid `GRANT` statements from the dump. (Bug #108974, Bug #34764157)

Changes in MySQL Shell 8.0.32 (2023-01-17, General Availability)



Important

This version includes a change which enables MySQL Shell to read MySQL Server option files and login paths by default. As a result, if you connect to a MySQL

> Server which uses an option file, it will be used, by default. If you do not want to use the options file, you must add `--no-defaults` to your command line.
>
> For more information, see WL#11206 in Functionality Added or Changed.

- AdminAPI Added or Changed Functionality
- AdminAPI Bugs Fixed
- Functionality Added or Changed
- Bugs Fixed

AdminAPI Added or Changed Functionality

- A new attribute named `hiddenFromRouter` is added to the output of the following commands:
  - *`Cluster`*`.status()`
  - *`Cluster`*`.describe()`
  - *`ReplicaSet`*`.status()`

  This attribute is `true` for any member hidden from MySQL Router traffic using the `_hidden` metadata tag. (Bug #34680289)

- A new option, `use_replica_primary_as_rw` is added to `<`*`ClusterSet`*`>.setRoutingOption()`.

  This option is read by MySQL Router and enables it to open or close a read-write (R/W) port on a router targeting a specific Cluster (where `target_cluster` is not set to `primary`), enabling you to use a R/W port on a ReplicaCluster. The ReplicaCluster continues to only accept R/O traffic. In the event of a switchover or failover, the R/W port remains unchanged.

  See MySQL Router Status for InnoDB ClusterSet. (Bug #34544084)

- It is now possible to set `ipAllowList` on a running cluster with *`Cluster`*`.setOption` and *`Cluster`*`.setInstanceOption`.

  

  > **Note**
  >
  > This option can only be set if the `communicationStack` is set to `XCOM`.

  See Setting Options for InnoDB Cluster.

  This also corrects an issue in *`Cluster`*`.setOption`, where instances were not checked to confirm they were online, and if the specified options were supported on each instance. (Bug #34424385)

- As of this release, ReplicaSet replication channels are created with SSL enabled by default.

  

  > **Note**
  >
  > This change does not apply to replication groups adopted using this version; their replication channels remain unencrypted.

  (Bug #30472808, Bug #30473564)

AdminAPI Bugs Fixed

- It was not possible to use `dba.dropMetadataSchema()` or *`cluster`*`.dissolve()` on a cluster which belonged to a ClusterSet. As of this release, these functions can be used on ClusterSet clusters if they are the only Cluster in the ClusterSet or they are invalidated. (Bug #34787737)
- It was not possible to create a working Cluster using MySQL Server 8.0.22 and a more recent version of MySQL Shell. (Bug #34787542)
- `rebootClusterFromCompleteOutage()` was unable to complete the reboot process if the rebooted cluster was a member of a ClusterSet and was missing purged transactions. There was no check for such purged transactions.

  As of this release, `rebootClusterFromCompleteOutage()` checks for purged transactions before attempting rejoinCluster.

  Additionally, replica clusters are now synchronized in two steps: the primary of the cluster is synchronized with the ClusterSet primary, then the rest of the cluster waits for those transactions to be applied.

  If there are both errant and unrecoverable transactions, an errant transaction error is thrown. (Bug #34692990)
- Metadata changes in `createCluster()` were prematurely committed through implicit commit statements. As of this release, these statements are executed in a separate session. (Bug #34670761)
- `super_read_only` was set on every rejoined cluster member, regardless of what mode the cluster was running, single- or multi-primary. (Bug #34647972)
- If the primary was unreachable, *`Cluster`*`.status()` failed with an error instead of automatically connecting to a secondary member. (Bug #34615265)
- If the primary was unreachable, `dba.getCluster()` failed instead of retrieving the cluster object from a secondary. (Bug #34579287)
- During reboot or failover in a ClusterSet or ReplicaSet, MySQL Shell checks if the candidate primary has the most recent `GTID_EXECUTED`. However, if load is high, it is possible that the applier is still working and `GTID_EXECUTED` still incrementing, and the check returns a premature result.

  As of this release, a `timeout` option is added to *`ClusterSet`*`.forcePrimaryCluster()` which specifies a number of seconds to wait for the applier to finish and ensure `GTID_EXECUTED` has the most up to date GTID set. The default value of `timeout` uses the value of `dba.gtidWaitTimeout`.

  The existing `timeout` option of *`ClusterSet`*`.setPrimaryCluster()` is extended to enable the applier to finish. (Bug #34465675)
- A fix in a previous version which prevented invalid endpoints being stored in the metadata did not take existing invalid values into account. As a result, attempting to add a member back to a cluster failed if the member's metadata contained an invalid endpoint. The following error was displayed:

  ```
  Debug: Cluster.addInstance: Invalid address format: ''(ArgumentError)
  ```

  As of this release, *`Cluster`*`.status()` scans for such invalid endpoints and generates a warning, and *`Cluster`*`.rescan()` corrects the invalid data. (Bug #34395705)
- Under certain circumstances, Cluster recovery accounts are leftover and not used. This can occur if a recovery process is canceled, for example.

As of this release *`cluster`*`.status()` reports the existence of such accounts in the `instanceErrors` field and *`cluster`*`.rescan()` removes any such leftover accounts. (Bug #33235502)

- An instance being provisioned for a cluster by clone was reported as `"mode": "R/W"` while it was being provisioned. As of this release, any member with `status` other than `ONLINE` will be reported as `"mode": "n/a"`. (Bug #30902267)

- The `dba.upgradeMetadata()` error message contained a typo.

  Thanks to Nico Pay for the contribution. (Bug #108861, Bug #34733164)

- Attempting to add an instance to a newly-created Replica Cluster, if the Primary Cluster was under high load, failed with several errors related to `super_read_only`.

  This issue was caused by an out-of-date topology view, leading to the newly created ReplicaCluster being considered a standalone cluster.

  As of this release, `createReplicaCluster()` synchronizes the metadata update transactions. thereby ensuring it has the correct topology view. (Bug #108426, Bug #34584939)

- The error `ReplicaSet.status: Failed to execute query on Metadata Illegal mix of collations` was returned if the server was configured with `character-set-client-handshake=OFF` or `--skip-character-set-client-handshake`, and the client attempted to use a different collation. The server collation was used for the client sessions, instead of negotiating the collation used.

  As of this release, the collation is explicitly set by AdminAPI sessions. (Bug #108209, Bug #34530914)

Functionality Added or Changed

- **Important Change:** The following options were added to the `exportTable()`, `dumpInstance()`, `dumpSchemas()`, and `dumpTables()` utilities:

  - `where`: enables you to export the data which satisfies a SQL condition.
  - `partitions`: enables you to export named partitions.

  As part of this development, the following options were added to the `dumpInstance()`, `dumpSchemas()`, and `dumpTables()` utilities:

  - `fieldsTerminatedBy`
  - `fieldsEnclosedBy`
  - `fieldsEscapedBy`
  - `fieldsOptionallyEnclosed`
  - `linesTerminatedBy`
  - `dialect` (json dialect is not supported) enabling you to dump your data as CSV or TSV file formats.

  For information on `exportTable()`, see Options

  For information on `dumpInstance()`, `dumpSchemas()`, and `dumpTables()`, see Options for Dump Output. (Bug #32669773, Bug #108504, Bug #34606598, WL #15311)

- **Important Change:** As of this release, MySQL Shell supports MySQL login paths and option files by default.

  

  Note

  If you do not want MySQL Shell to read the option file, you must add `--no-defaults` to your command line.

  The following MySQL command line options are now supported at the start of the command line:

  - `--print-defaults`
  - `--no-defaults`
  - `--defaults-file`
  - `--defaults-extra-file`
  - `--defaults-group-suffix`
  - `--login-path`

  MySQL Shell also reads a new section in the MySQL configuration file, `[mysqlsh]`, in which you can define the MySQL Shell options.

  MySQL Shell also reads the `[client]` section of the MySQL Preconfiguration file.

  For more information, see Connecting using login-path and Options Files.

  

  Note

  Some `[client]` options are not supported by MySQL Shell, such as `local-infile`, and some options have the same name in both, but take different values, such as the `[client]` option `--compress` and the `[mysqlsh]` option `compress=value`.

  MySQL Shell returns a specific error for such options, specifying the name of the option and the error.

  This is something to be aware of during an upgrade to MySQL Shell 8.0.32. Your options files may require editing, or you may need to define an alternate options file for your MySQL Shell installation.

  (WL #11206)

- As of MySQL 8.0.32, unquoted identifiers starting with a dollar sign ($) are deprecated. As of MySQL Shell 8.0.32, the Upgrade Checker utility checks for the existence of such identifiers and presents a list to the user. Stored routines are also checked. (Bug #34684193)
- The following changes were made to the `logSql` option:
  - `--log-sql=on` no longer filters `SELECT` statements by default.
  - A new filtering option, `logSql.ignorePatternUnsafe`, is added. This option defines a colon-separated list of statement patterns to filter out. Default value is `*IDENTIFIED*:*PASSWORD*`.
  - The default value of the filtering option, `logSql.ignorePattern`, is changed from `*SELECT*:*SHOW*:*IDENTIFIED*:*PASSWORD*` to `*SELECT*:*SHOW*`.

  - logSql option parameters were updated:

    - `on`: logs all SQL statements except those defined in the `logSql.ignorePatternUnsafe` and `logSql.ignorePattern` options.

    - `all`: new parameter which logs all SQL statements except those defined in the `logSql.ignorePatternUnsafe`.

  See SQL Logging Options. (Bug #34249346)

- In previous versions, chunking required a defined primary key or unique index on the table. As of this version, tables without primary key or unique index can be chunked based on the number of rows in the table, the average row length, and the `bytesPerChunk` value. (Bug #34195250)

- It is now possible to configure MySQL Shell's AWS S3 support with AWS environment variables. For more information, see S3-compatible Storage.

  The command line option `s3Region` was added to the dump and load utilities. This option enables you to define the AWS region from the command line. For more information, see MySQL Shell Utilities. (Bug #108495, Bug #34604763)

- MySQL Shell supports the Microsoft Azure Blob storage service for import, export, dump, and load operations.

  See Azure Blob Storage and MySQL Shell Utilities. (WL #15337)

Bugs Fixed

- Under certain circumstances, HTTPS read operations could fail with a libcurl-related CURLE_PARTIAL_FILE error. As of this release, the read operation is retried. (Bug #34765385)

- It was possible to install MySQL Shell (rpm) on Enterprise Linux 7 running OpenSSL 1.0.1, although OpenSSL 1.0.2 was required. (Bug #34747533)

- The following errors occurred if MySQL Shell was used to call a stored procedure that produces more than one result:

  - Using `--column-type-info` only displayed the column metadata for the first result.

  - Using JSON output, such as `--json=raw`, only displayed the first result.

  (Bug #34716739)

- Under certain circumstances, calling `util.importTable()` with a wildcard, on a bucket in Oracle Cloud Infrastructure's Object Storage, failed with an error similar to the following:

  ```
  Util.importTable: Failed to list multipart uploads: ... (404) (MYSQLSH 54404)
  ```

  (Bug #34679579)

- The `authentication_oci_client` plugin is now bundled with the MySQL Shell installation, in the `lib/mysql/plugins` directory of your MySQL Shell installation on Linux platforms, and `lib\mysql\plugins` on Windows platforms.

As of this release, the default value of `mysqlPluginDir` is set to the MySQL plugins directory of your MySQL Shell installation. For example, on Linux platforms, this is `lib/mysql/plugins`. (Bug #34676464)

- Under certain circumstances, `util.debug.collectDiagnostics` failed with the following error:

```
TypeError: object of type 'NoneType' has no len()
```

(Bug #34674675)

- When started from the command line using options with incorrect values, MySQL Shell did not return useful information on which option was in error. As of this release, the error message contains helpful information on which option contains the error. (Bug #34624922)
- Loading a dump using a Pre-Authenticated Request (PAR) failed if the filename contained URL-encoded characters. (Bug #34599319)
- The extra consistency check for the dump utilities with `consistent: true` was too sensitive and reported errors even when no DDL changes were made.

  As of this release, the consistency check analyzes statements executed during the dump and fails only if DDL-related statements are detected.

  A new option is added to the dump utilities, `skipConsistencyChecks: [true | false]`, which, if set to true, skips the additional consistency check and assumes the dump is consistent. (Bug #34556560)
- If `sql_mode` was set to `ANSI_QUOTES`, the following occurred:
  - `\status` failed with the following error:

```
Unknown column ' ' in 'field list'
```

  - `USE` schema failed with the following error if the schema's name contained consecutive quotation marks (`sample""name`, for example):

```
ERROR: Too many consecutive quote characters: 2
```

(Bug #34538796, Bug #34709673)

- Under certain circumstances, in SQL mode, splitting statements with commands such as `\q` could cause MySQL Shell to close unexpectedly. (Bug #34517691)
- When parsing SQL script, MySQL Shell did not account for the `sql_mode` current session value `NO_BACKSLASH_ESCAPES` and escaped all backslashes. (Bug #34488296)
- It was not possible to `USE` a schema with a single quote (') in its name. (Bug #34334556)

- MySQL Shell now bundles Python 3.10.8 for platforms where Python 3 is not included or is not at the minimum supported version.

  

  Note

  This is true for all builds except Oracle Linux 7, which bundles Python 3.9.15

  (Bug #109240, Bug #34844958)

- `util.loadDump` failed if a table in the dump had more than one FULLTEXT index. The following error was returned:

  ```
  InnoDB presently supports one FULLTEXT index creation at a time
  ```

  As of this release, if a table contains more than one FULLTEXT index, each additional index is created using separate `ALTER TABLE` statements. (Bug #108991, Bug #34787778)

- `loadDump` failed when loading a view which referenced another view whose DEFINER did not yet exist.

  As of this release, users are loaded after objects and view placeholders are created, but before the placeholders are replaced by the views. (Bug #108975, Bug #34768224)

- The `util` global object was not available from the command line if the persisted default mode was set to SQL. (Bug #108839, Bug #34734133)

- When using `util.exportTable()` to export to an S3 bucket, the AWS-specific information, such as the S3 bucket name, was not written to the suggested `util.importTable()` call. (Bug #108497, Bug #34657730)

- Attempting to add an instance to a newly-created Replica Cluster, when the Primary Cluster is under very high load, failed with errors related to `super_read_only`.

  (Bug #108426, Bug #34584939)

- MySQL Shell Upgrade Checker did not check for orphaned stored routines. That is, stored routines which reference non-existent schemas. As a result, the upgrade procedure could fail.

  As of this release, the Upgrade Checker utility checks for such orphaned routines and returns an error if they are found. (Bug #108005, Bug #34433132)

- Extra underscores were added to property names when querying MySQL 8.0 servers with the MySQL Shell Python API. In MySQL 8.0, the column names are uppercase, but Python naming conventions added an underscore before each uppercase letter, resulting in column names like `row.c_o_l_u_m_n_n_a_m_e`.

  As of this release, such fields are exposed as properties in both Python and JavaScript, and the column name takes precedence over the naming convention. (Bug #107853, Bug #34379393)

- A package year verification check prevented compilation of MySQL Shell, with default options, when close to the end of year. As of this release, MySQL Shell compilation defaults to the current year as package year. (Bug #101905, Bug #32246288)

Changes in MySQL Shell 8.0.31 (2022-10-11, General Availability)

- AdminAPI Added or Changed Functionality
- AdminAPI Bugs Fixed

- Functionality Added or Changed
- Bugs Fixed

AdminAPI Added or Changed Functionality

- The AdminAPI command `dba.rebootClusterFromCompleteOutage()` has been reimplemented for usability and to reduce complexity of operation and the potential for split-brain scenarios.

  Topology changes are no longer permitted. The following options were removed:

  - `removeInstances`
  - `rejoinInstances`

  The following options were added:

  - `force`: executes the operation even if some members of the Cluster cannot be reached, or the primary instance selected has a diverging or lower GTID_SET.
  - `primary`: instance definition representing the instance that must be selected as the primary.
  - `dryRun`: all validations and steps of the command are executed, but no changes are actually made.

  If enabled on the primary instance of the cluster, while in single-primary mode, `super_read_only` is automatically disabled.

  See Rebooting a Cluster from a Major Outage. (Bug #33871079, Bug #30877552, Bug #26586818, Bug #25487521, Bug #29763863, Bug #32002281, Bug #28939352, Bug #33389693, Bug #33791275, Bug #33875723, WL #11561)

- The `-- cluster` option is extended to assign a ClusterSet variable if the selected Cluster is a member of a ClusterSet. (Bug #108112, Bug #34485950)

AdminAPI Bugs Fixed

- Clusters upgraded from 8.0.26 to 8.0.27, or higher, only had access to the Group Replication member actions available in 8.0.26. The upgraded members did not have access to any action introduced after 8.0.26, such as `mysql_start_failover_channels_if_primary`, which was introduced in 8.0.27.

  As a result, attempting to create a Replica Cluster resulted in an error.

  As of this release, `createReplicaCluster()` resets the Group Replica member actions of the target instance using `group_replication_reset_member_actions()`. (Bug #34465006)

- *`clusterSet`*`.setRoutingOption(option, null)` reset all routing options to their default values instead of resetting the specified option, only. (Bug #34458017)

- It was not possible to change a cluster's options, using *`cluster`*`.setOption()`, if the cluster was a member of a ClusterSet. A runtime error occurred. (Bug #34457903)

- It was not possible to use *`cluster`*`.fenceWrites()` on Replica Clusters.

  As of this release, it is possible to use *`cluster`*`.fenceWrites()` on INVALIDATED Replica Clusters. Also, if run against a Replica Cluster with `super_read_only` disabled, it will enable it.

  If *`cluster`*`.fenceWrites()` is run on a healthy Replica Cluster, no changes are made.

It is not possible to use *cluster*.unfenceWrites() on a Replica Cluster. (Bug #34417802)

- Rejoining an instance to a Replica Cluster, using the MYSQL communication stack, resulted in an error if the primary cluster of the ClusterSet was under heavy load. The rejoin process recreates the replication account on the primary cluster of the ClusterSet and this transaction took too long to replicate to the Replica Cluster where it was required for Group Replication. (Bug #34264094)

- MySQL Router periodically updates the last_check_in of the Cluster metadata schema. Under certain circumstances, such as a ClusterSet split-brain scenario, these updates can make rejoin operations impossible due to the creation of errant transactions.

  As of this release, a new settable option is added in the Metadata schema, stats_updates_frequency. This option defines, in seconds, the interval between MySQL Router check-in updates.

  For more information on MySQL Router's behavior, see the MySQL Router 8.0.31 documentation.

  This value can be set using *Cluster*.setRoutingOptions and can be viewed using *Cluster*.routingOptions([router]).

  If set to 0 (default), no periodic updates are done. MySQL Router rounds up the value to a multiple of its TTL. For example:

  - If lower than TTL its gets rounded up to TTL. For example: if TTL=30 and stats_updates_frequency=1, the effective frequency is 30 seconds.

  - If not a multiple of TTL, it is rounded up and adjusted according to the TTL. For example, if TTL=5 and stats_updates_frequency=11, the effective frequency is 15 seconds, or if TTL=5 and stats_updates_frequency=13, the effective frequency is 15 seconds.

  If the value is null, the option value is cleared and the default value takes effect. (Bug #34190956)

- Under certain circumstances, it is possible for the transaction size on a Replica to be larger than that on the source. In ClusterSets, this can result in issues with the ClusterSet replication channel. To avoid this issue, the following changes were made:

  - group_replication_transaction_size_limit is set to the maximum value in Replica Clusters and stores the original value in the metadata schema so it can be restored in the event of a switchover or failover.

  - You can manually define group_replication_transaction_size_limit when creating a Cluster or Replica Cluster with the option transactionSizeLimit.

  - addInstance() and rejoinInstance(), automatically set the value of group_replication_transaction_size_limit to ensure all members use the same value.

  - You can change the value of group_replication_transaction_size_limit on a running Cluster with Cluster.setOption().

  - You can view the option's value with Cluster.options().

  - The original value of the option is restored if the Cluster is removed from a ClusterSet using removeCluster().

  - cluster.Status() detects inconsistencies in member values and enables you to correct them with Cluster.rescan().

(Bug #34011262)

- `dba.upgradeMetadata()` displayed messages relating to InnoDB Cluster regardless of the metadata being upgraded. (Bug #33931291)
- Commands which called external processes, such as `dba.deploySandboxInstance()`, did not include the name of the external process in the error message if the process failed. As of this release, if the external process fails, its name is included in the error message. (Bug #28543317)
- *`clusterSet`*`.status()` failed with a logic error if run against a ClusterSet where one or more of the Replica Clusters were offline and the Group Replication plugin was uninstalled from each member. (Bug #108066, Bug #34465132)
- ClusterSet commands which perform transaction set consistency checking, such as `rejoinCluster` and `setPrimaryCluster`, became unresponsive during view change log event reconciliations if the write load was high. This occurred because reconciliation included the entire transaction backlog instead of just the view change log events. (Bug #108065, Bug #34462141)
- Cluster failover could fail under high load because the cluster being promoted was compared with itself in checks to confirm the promoted cluster was the most up-to-date. This comparison failed because the applier was catching up and the GTID_EXECUTED comparison resulted in two different values.

  As of this release, the check for most up-to-date cluster does not include the promoted cluster. (Bug #108064, Bug #34465882)

Functionality Added or Changed

- The upgrade checker utility now checks for schema and table names using invalid characters. (Bug #34460142)
- A new command line option `-c`/`--pyc=`*`cmd`* is added in this release. This command enables you to execute a Python command and quit. Any options specified after the command are treated as arguments of the processed command. (Bug #34361996)
- The keyword `FULL` was added to the upgrade checker utility's keyword list. (Bug #34285219)
- Load performance is improved for parallel load of dumps with large numbers of large tables. (Bug #34173880)
- The following new diagnostics were added in this release:
  - `util.debug.collectHighLoadDiagnostics(path[, options])`: runs multiple iterations of diagnostics on the target MySQL server.
  - `util.debug.collectSlowQueryDiagnostics(path, query[, options])`: runs diagnostics for the duration of the supplied query's duration.

  For more information, see collectDiagnostics Utility.

  The options `customShell` and `customSql`, included in both new diagnostics, were also added to the existing `util.debug.collectDiagnostics`:

  - `customShell`: array of additional operating system shell commands to run.
  - `customSql`: array of additional SQL statements to run.



Note

The options `customShell` and `customSql` can be run with `before`, `during`, and `after` when run in `util.debug.collectHighLoadDiagnostics` and `util.debug.collectSlowQueryDiagnostics`. They are run before the metrics collection begins in `util.debug.collectDiagnostics` and this is not configurable.

As part of this development, the following auxiliary functions were added to the `mysql` module:

- `mysql.parseStatementAst(sql)`: returns AST encoded as a JSON structure.
- `mysql.splitScript(script)`: returns a string containing multiple statements.

(Bug #34056331, Bug #34056336, WL #14886)

- MySQL Shell now supports the parallel index creation introduced in MySQL Server 8.0.27. Previously, the dump loading utilities added indexes sequentially, one at a time. As of this release, all indexes in a table are added simultaneously.

  See Configuring Parallel Threads for Online DDL Operations for restrictions and configuration. (Bug #33976718)

- The `schema` option of `utils.loadDump()` is extended to enable creation of the schema if it does not already exist.

  This load option is supported for single schema dumps, or if the filtering options result in a single schema.

  

  Note

  If the new schema name differs from the schema name in the dump, the dump is loaded to the new schema, but no changes are made to the loaded data. That is, any reference to the old schema name remains in the data. All stored procedures, views, and so on, refer to the original schema, not the new one.

  (Bug #33799352)

- MySQL Shell now retrieves the most up-to-date view of the managed topology before performing any validation, update, or operation. `getCluster()`, `getClusterSet()`, or `getReplicaSet()` are now called first. This ensures all operations on AdminAPI objects are run against the latest version of that object.

  

  Note

  The `dba.getCluster()` option, `connectToPrimary`, is deprecated in this release. The operation now always connects to the primary member, if possible. If not, it connects to a secondary.

  This patch also resolves the problem of the excessive number of sessions and metadata refreshes happening with each command executed. Several refactors of the usage of connection pools were done. (Bug #30884646, Bug #33381060, Bug #33415656, WL #14862)

- Support was added for Python modules which invoke the Python interpreter, such as multiprocessing. (Bug #99796, Bug #31460929)

- MySQL Shell's SQL Auto Completion is now context aware. It now presents object names, keywords, and so on, which are relevant to the command being entered and the context that command is in.

  The autocompletion API is exposed to developers through the functions `shell.autoCompleteSql(statement, options)` (JavaScript) and `shell.auto_complete_sql(str statement, dict options)` (Python).

  See Autocompleting SQL. (WL #13397)

Bugs Fixed

- It was not possible to install MySQL Shell from the Compressed TAR installation package on Oracle Linux 9 due to a dependency on the `libcrypt` library, which is not delivered on that platform. (Bug #34495554)
- It was not possible to import a dump or table on Windows if the file size was larger than 4GB. The error, `stoul argument out of range` was returned. (Bug #34479702)
- Under certain circumstances, when loading a table, the index creation and analyze table tasks could occur before the table was loaded. (Bug #34465642)
- Under certain circumstances, when processing command-line arguments, such as `--cluster rescan`, MySQL Shell attempted to write to the log file before it was initialized. This resulted in a segmentation fault. As of this release, the command line parser does not attempt to write to the log. (Bug #34460776)
- Accessing a non-existing key in a Shell dictionary using the `get()` method resulted in an exception similar to the following:

  ```
  Traceback (most recent call last):
  File "<string>", line 1, in <module>
  SystemError:
  <built-in method get of shell.Dict object at 0x7fa458731c90>
  returned a result with an error set
  ```

  (Bug #34403275)
- The Python `getattr()` function threw an exception when used with MySQL Shell dictionaries. (Bug #34387005)
- It was not possible to load files with CRLF line endings in JavaScript mode. This affected the following JS loading methods:
  - `os.loadTextFile()` function
  - `\source` command
  - JavaScript plugin loading

  (Bug #34321108)
- `util.debug.collectDiagnostics()` failed if a log file included non-utf8 characters. (Bug #34208308)
- Data was not loaded for `util.loadDump` or `util.importTable`, if the server global variable `AUTOCOMMIT` was set to OFF.

As of this release, the `AUTOCOMMIT` session variable is set to ON by the load and import utilities. (Bug #34173126, Bug #33360787)

- MySQL Shell output BIT columns as base10 integers instead of HEX strings. As of this release, BIT columns are output as HEX strings. (Bug #34110337)
- It was not possible to load a dump created from a MySQL 5.7 instance if it contained a table with a BLOB, TEXT, GEOMETRY or JSON column with a default value.

  As of this release, the upgrade checker utility scans for such tables and reports an error if they are found. (Bug #34063868)
- In certain circumstances, importing dumps from MySQL-compatible databases resulted in syntax errors due to differences in `CREATE USER` syntax.

  As of this release, MySQL Shell detects the invalid syntax during the dump process, displays an error, and provides information on how to exclude such users. (Bug #34049624)
- The upgrade checker utility did not detect reserved words in stored routines. (Bug #33866437)
- MySQL Shell included `log_syslog` in the configuration files of sandbox instances. This option was removed in MySQL 8.0.13. (Bug #29882585)
- The MySQL Shell man pages were not included in the sources or installation packages. (Bug #29770787)
- Passwords were not accepted when provided by the `--passwords-from-stdin` command-line option. (Bug #108158, Bug #34501568)
- The following issues were found in the MySQL Shell source files:
  - `INSTALL` had the following issues:
    - The mandatory dependencies listed Python 3.7. Python 3.8 is the minimum required version.
    - The mandatory dependencies did not list the required libssh version, 0.9.2.
    - The mandatory dependencies listed CMake 2.8.12. CMake 3.5.1 is the minimum required version.
    - The Linux build instructions included an incorrect path change, `cd bld`, instead of `cd mysql-shell/bld`.
  - `README` contained an incorrect link for the InnoDB Cluster documentation.
  - `postrm.in` contained an inaccurate message.

  Thanks to Evgeniy Patlan for these contributions. (Bug #107412, Bug #34219276, Bug #107415, Bug #34219284, Bug #107411, Bug #34219274, Bug #107413, Bug #34219279, Bug #107416, Bug #34219285, Bug #107414, Bug #34219283)
- Session variables, such as `error_count`, which are affected by the last executed statement, changed unexpectedly if MySQL Shell custom prompts used queried variables. (Bug #104955, Bug #33362817)

Changes in MySQL Shell 8.0.30 (2022-07-26, General Availability)

- AdminAPI Bugs Fixed
- Functionality Added or Changed

- Bugs Fixed

AdminAPI Bugs Fixed

- When a replica cluster was removed from an InnoDB ClusterSet, reconciliation of view change log events was not carried out if the replica cluster had only a single member (the primary member). If that instance was then used to create a new replica cluster, it contained additional transactions that meant replication could not take place with the other clusters in the InnoDB ClusterSet. View change log events are now always reconciled when a replica cluster is dissolved, even when there is only a single member. (Bug #34069295)

- On MySQL sandbox instances created using AdminAPI, an auxiliary MySQL Shell instance is started for AdminAPI operations. Previously, the auxiliary MySQL Shell instance was placed in JSON shell mode using the `MYSQLSH_JSON_SHELL` environment variable. However, if the user's MySQL Shell instance was also in JSON shell mode, error messages returned to it by the auxiliary MySQL Shell instance could not be parsed because they were already wrapped in JSON documents, so they were not displayed to the user. Auxiliary MySQL Shell instances on sandbox instances now do not use the MYSQLSH_JSON_SHELL environment variable. (Bug #34066667)

- When the *`clusterSet`*`.setPrimaryCluster()` command or *`clusterSet`*`.rejoinCluster()` command was used, a divergent GTID set on the target cluster was not reconciled because view change events were filtered out too early in the process. If the binary logs had been purged, the situation could not be repaired and the cluster could not continue to operate in the InnoDB ClusterSet. The GTID set reconciliation process has now been fixed. (Bug #34013718)

- `ClusterSet.status()` returned `OK_NO_TOLERANCE` instead of `OK_NO_TOLERANCE_PARTIAL` for ClusterSets with one or more unreachable members and fewer than the acceptable threshold of online members. (Bug #33989031)

- It was not possible to stop an InnoDB cluster metadata upgrade if outdated router instances were detected. When prompted to proceed with the upgrade, or not, the upgrade was performed regardless of the user's choice. (Bug #33941759)

- AdminAPI's *`Cluster`*`.rescan()` command did not identify the new version of Group Replication's communication protocol that was introduced in MySQL 8.0.27. The command now offers an upgrade to the new communication protocol when it is available. (Bug #33922830)

- InnoDB Cluster commands validate that the target instance belongs to the cluster before performing the operation. When the server's address was compared against the server instances registered in the cluster's metadata, host names were compared using case sensitivity, which is contrary to the rules in RFC 1034 for domain name comparison. This could lead to a target instance not being identified as part of the cluster. Host names are now checked using a case insensitive comparison. (Bug #33893435)

- `Cluster.dissolve()` did not clean up the metadata and configuration associated with the cluster if the cluster had been removed from a ClusterSet using the `force` option.

  As of this release, `Cluster.dissolve()` ensures all recovery and ClusterSet replication accounts are removed, and the metadata schema is removed, also. (Bug #33239404)

- It was possible to add an instance to a cluster although it was already a member of a ReplicaSet. No error was generated as the instance was removed from the ReplicaSet and added to the cluster. As of this release, an error is generated and the action fails. (Bug #31964273)

- The `replicationLag` value reported by `Cluster.status()` was not calculated correctly.

  As of this release, it returns one of the following:

  - The time difference between the last transaction commit timestamp and the last transaction applied timestamp, in HH:MM:SS format.

    If multiple workers are used, the value is retrieved from the worker executing the oldest transaction.

  - `null`: The replication connection or SQL thread is not running.

  - `applier_queue_applied`: The applier queue has applied everything. That is, if the last queued transaction and the last applied transaction are the same, or the applying transaction is 0.

  For more information, see Checking a cluster's Status with *`Cluster`*`.status()`. (Bug #31693765, Bug #106826, Bug #34002328)

- The InnoDB Cluster *`Cluster`*`.status()` command did not check whether a server instance had been put into offline mode manually by setting the `offline_mode` system variable. The command now checks for this situation and reports the mode as `n/a` if the system variable has been set.

  It also reports a warning, for example:

  ```
        "instanceErrors": [
                    "WARNING: Instance has offline_mode enabled."
         ],
  ```

  (Bug #107057, Bug #34109382)

- MySQL Shell closed unexpectedly if an attempt was made to rejoin an instance to a ReplicaSet after that instance was removed from the ReplicaSet and its `server_uuid` altered.

  As of this release, the `rejoinInstance` command ensures the metadata schema is updated with the new `server_uuid`. (Bug #106847, Bug #34038210)

Functionality Added or Changed

- MySQL Server 8.0.30 introduced GIPK mode, Generated Invisible Primary Keys. When running in this mode, for any InnoDB table that is created without an explicit primary key, the MySQL server automatically adds a generated invisible primary key (GIPK) to the table. This mode is enabled by setting `sql_generate_invisible_primary_key` to ON.

  MySQL Shell's load utility option `createInvisiblePKs` uses the server's GIPK mode to generate invisible primary keys for tables which do not have primary keys.

  Under certain circumstances, if a user has insufficient privileges to use GIPK mode, MySQL Shell can fall back to the previous method of generating invisible primary keys.

  See Dump Loading Utility for more information. (Bug #34409792)

- `util.checkForServerUpgrade` is now run automatically by any of the dump utilities, if the dump option `"ocimds": true` is defined and the target instance is MySQL 5.7.

  For more information on `util.checkForServerUpgrade`, see Upgrade Checker Utility. For more information on the dump utilities, see Instance Dump Utility, Schema Dump Utility, and Table Dump Utility. (Bug #34052980)

- MySQL Shell 8.0.30 supports the Group Replication Communication Stack, introduced in MySQL 8.0.27. This allows a simpler and safer setup as it obsoletes the usage of IP allowlists, removes the explicit

need for another network address used for internal GCS communication, and introduces user-based authentication.

This patch adds support in the AdminAPI for the new communication stack by:

- Adding support to choose which GR communication stack must be used when creating a Cluster or a Replica Cluster, by exposing a new option named `communicationStack` supporting `XCOM` or `MYSQL`.
- Ensures `MYSQL` is the default communication stack in use by Clusters running MySQL 8.0.27, or higher.
- Automatically and transparently generates and sets all configurations required for both communication stacks at cluster creation and for topology changes (`addInstance`/`rejoinInstance`/`rescan`), without user intervention.
- Allows migrating an existing Cluster to a different communication stack when rebooting it from complete outage by exposing a new option named `switchCommunicationStack`.
- Allows setting a value for `group_replication_local_address` in `Cluster.rejoinInstance()` by extending the command with a new option `localAddress`.
- Adds a new check in `Cluster.rescan()` to verify whether there is any Cluster member using a recovery account that is not the one created for itself and ensure the right account is used. This ensures instances that fall in that category (added with clone recovery and waitRecovery:0) are able to auto-rejoin the Cluster.
- Extends `Cluster.rejoinInstance()` and `dba.rebootClusterFromCompleteOutage()` to include the `ipAllowList` and `localAddress` options.
- Extends `Cluster.rejoinInstance()` and `dba.rebootClusterFromCompleteOutage()` to also perform the instance checks for Cluster compliance as `Cluster.addInstance()` does.
- Extends `Cluster.status({extended:1})`, ClusterSet.status({extended:2}), and `Cluster.options()` to return the `communicationStack` values.



Note

As of this release, `group_replication_tls_source` must not be set to `mysql_admin`.

(WL #14765)

- The new option, `logSql`/`--log-sql` enables you to log all SQL statements executed by MySQL Shell commands or utilities to the MySQL Shell log file.



Note

This option deprecates `dba.logSql`.

For more information, see MySQL Shell Logging and Debug. (WL #14749)

- MySQL Shell supports the S3-compatible cloud storage services, AWS S3 and Oracle Cloud Infrastructure's S3 compatibility API.

  See MySQL Shell Utilities. (WL #14387)

Bugs Fixed

- Under certain circumstances, when dumping an instance for migration from certain compatible databases, the following error was returned during chunking:

  ```
  ERROR: [Worker004]: Error while chunking `xxxx`.`xxxx`: get_uint(8): field type is String
  ```

  (Bug #34206985)

- The number of rows loaded in each chunk was not written to the progress file. (Bug #34201239)

- Under certain circumstances, a `loadDump` operation on a dump which contained a syntax error in a `CREATE USER` statement could result in a user's password being displayed in the SQL syntax error message.

  As of this release, if a failed query contains either `IDENTIFIED` or `PASSWORD` keywords, the quoted strings are anonymized. (Bug #34141432)

- When the `dryRun` option was set for MySQL Shell's dump loading utility `util.loadDump()`, if the `updateGtidSet` option was set to `replace` or `append`, the value of the `gtid_purged` system variable on the target server instance was changed, although a dry run is not meant to affect the target server. The issue has now been fixed. (Bug #34127069)

- On Linux systems, MySQL Shell's password store used the MySQL configuration utility `mysql_config_editor` as a Secret Store Helper, provided that the MySQL client package was installed on the system, and `mysql_config_editor` was available in the system's PATH. Otherwise, the password store functionality was unavailable and MySQL Shell's credential helper functions were unavailable. MySQL Shell now bundles `mysql_config_editor` in Linux builds, so that the functionality can be used if the MySQL client package is not installed on the system. (Bug #34097411)

- `util.debug.collectDiagnostics` did not produce an error if a filename was not included in the command. A nameless zip file was created. As of this release, if a filename is not included in the command, an error is displayed and the zip file is not created. (Bug #34048754)

- When you use the `shell.create_context` function in MySQL Shell's Python mode to create a scope and context wrapper for a program, you can now use the `finalize()` function to require the release of the file handle created for the logger object in the shell context. (Bug #33988826)

- When MySQL Shell's instance dump utility `util.dumpInstance()`, schema dump utility `util.dumpSchemas()`, and table dump utility `util.dumpTables()` were used to dump tables with compound primary keys, the chunking process could give rise to a `filesort` operation to optimize the `ORDER BY` clause. In the case of a large table with multiple chunks being dumped simultaneously, the operation could cause the dump to fail for lack of disk space. This occurred when a `BETWEEN` clause used the same lower and upper bound, so the chunking process now uses an alternative condition in that situation. (Bug #33976259)

- When MySQL Shell's Python API was used to read date and time data, the conversion did not account for the fact that MySQL can accept invalid date and time values in some SQL modes, leading to Python errors. The data is now only converted into Python values when possible, and in the case of a conversion error, the string representation of the original value is returned. (Bug #33972939)

- MySQL Shell's parallel table import utility `util.importTable()` and dump loading utility `util.loadDump()` have a new option `sessionInitSQL`, which lets you specify a list of SQL statements to run at the start of each client session used for loading data into the target MySQL instance. You can use this option to change session variables. If an error occurs while running the SQL statements, the import stops and returns an error message. (Bug #33969606)

- In file systems where the `d_type` member does not appear in the `dirent` structure, GCC versions prior to 8.4.0 cannot detect the file type. MySQL Shell ignores files with unknown file types, so if it was compiled with an affected GCC version, it was unable to load a dump stored on such a file system. Now, MySQL Shell checks its own GCC version and if it is affected, uses the `stat()` function instead to determine file types. (Bug #33951851)
- Following the extensions to MySQL Shell's `shell.prompt()` function in MySQL Shell 8.0.29, the resulting JSON documents no longer included a new line at the end so that the end of the document could be identified and subsequent user input could be read. The issue has now been fixed. (Bug #33949419)
- MySQL Shell closed unexpectedly if `shell.reconnect()` was called without a global session. (Bug #33945641)
- When using the X DevAPI `CollectionModify` object, methods which accept a document path as a parameter did not properly handle arguments containing spaces. For example, the following attempted to remove an attribute named `some`:

  ```
  col.modify("_id='1'").unset("some path")
  ```

  Paths with spaces must be specified using a path expression or quoted using backtick characters. For example:

  ```
  col.modify("_id='1'").unset("$.'some path'")
  ```

  or

  ```
  col.modify("_id='1'").unset("`some path`")
  ```

  As of this release, paths containing spaces which are not properly defined with a path expression, such as `some path` or `'some path'`, are treated as errors. (Bug #33795166, Bug #33795149)
- The generic Linux packages for MySQL Shell 8.0.28 and 8.0.29 returned an error due to an indirect dependency on the `krb5` library for SSH support. The library is now bundled with MySQL Shell and the bundled version takes precedence over any system version. (Bug #33787652)
- `/n` was not parsed properly, in Python or JavaScript modes, if MySQL Shell was started with `--json`. An error was returned. (Bug #33678846)
- MySQL Shell's instance dump utility `util.dumpInstance()`, schema dump utility `util.dumpSchemas()`, and table dump utility `util.dumpTables()` upload large files to Oracle Cloud Infrastructure Object Storage buckets by chunking them and using multipart uploads. If the dump operation was canceled by the user or stopped due to an exception in a thread, multipart uploads that had already started were left in an active state in the bucket. The utilities now attempt in these situations to stop any multipart uploads pending completion. (Bug #32583524)
- MySQL Shell's upgrade checker utility `util.checkForServerUpgrade()` now checks for indexes on generated columns using `CONVERT()`,which changed its behavior in MySQL 8.0.28, and warns you to check applications that rely on the previous behavior. (Bug #106419, Bug #33840716)
- The upgrade checker utility returned an empty table name and an error for orphaned tables. As of this release, the utility parses the `innodb_sys_tables` table for the presence of orphaned tables. (Bug #106372, Bug #33822225)
- The upgrade checker utility raised errors, during a 5.7 to 8.0 upgrade check, for MySQL 5.7 schema or table names created using reserved names such as `LPT3`, `AUX`, or `PRN`. These are registered in `INFORMATION_SCHEMA.INNODB_SYS_TABLES` by appending `@@@` to their names. However, their

original names were used in `TABLES`. The upgrade checker raised schema consistency errors for such tables. (Bug #105958, Bug #33707127)

Changes in MySQL Shell 8.0.29 (2022-04-26, General Availability)

- AdminAPI Bugs Fixed
- Functionality Added or Changed
- Bugs Fixed

AdminAPI Bugs Fixed

- If MySQL Router was upgraded as part of an InnoDB Cluster upgrade process, the `dba.upgradeMetadata()` command continued to report that MySQL Router was outdated on the first try. This was because the command gave the MySQL Router account required privilege access, but MySQL Router required a metadata cache reload (TTL) to implement this and update the metadata information. Retrying the command one or more times would work when the TTL took place, which by default happens every 0.5 seconds. The command now attempts to retrieve MySQL Router's metadata information for up to two seconds before returning the results to the user, and adds a note to explain a high TTL as a possible cause of MySQL Router being reported as outdated. (Bug #33914146)

- The *`Cluster`*`.rebootClusterFromCompleteOutage()` command did not implement a changed setting for `group_replication_view_change_uuid` that was generated and set by a *`Cluster`*`.rescan()` command. The operation now checks if a new value for `group_replication_view_change_uuid` has been persisted and uses it when rebooting the cluster. (Bug #33850528)

- For MySQL Server instances at release 8.0.27 and above, for an InnoDB Cluster that is part of an InnoDB ClusterSet, the `group_replication_view_change_uuid` system variable is required and must be set to the same value on all member servers in the cluster. Previously, the *`Cluster`*`.rescan()` command generated and set a value for the system variable on the cluster's member instances if it was not found or did not match. Following this, a full reboot of the InnoDB Cluster was required to implement the change, which AdminAPI informed the administrator about but did not perform automatically. If the reboot was not performed, members with a pending `group_replication_view_change_uuid` correction were subsequently unable to rejoin the group using Group Replication's auto-rejoin function if they were restarted.

  Now, the *`Cluster`*`.rescan()` command does not set the system variable value automatically by default. The administrator may specify the new *`Cluster`*`.rescan()` option `updateViewChangeUuid` to generate and set a value for the system variable in the same way as the previous behavior. If the new option is not specified, the command makes no changes to the system variable, and just issues a warning to the administrator that the system variable value must be set and the cluster must be rebooted. (Bug #33748812)

- AdminAPI's `Cluster.setPrimaryInstance(`*`instance`*`)` command overrides the election process and forces a specific server instance in the underlying Group Replication group to become the new primary. It uses the `group_replication_set_as_primary` function. Previously, the function waited for all active transactions on the existing primary to end, including incoming transactions after the function was used, before making the current primary read only and changing to the new primary. There was no upper limit to the wait time. From MySQL Shell 8.0.29, you can use the new `runningTransactionsTimeout` option to add a timeout between 0 and 3600 seconds for transactions that are running when you use the function. When you set a timeout, incoming transactions after the command is issued are rejected. (Bug #33538559)

- If the host name and port reported by a server instance (`report_host` and `report_port`) were changed after the instance joined an InnoDB Cluster, the cluster metadata was not changed correspondingly, causing issues with operations such as setting the instance as the primary instance. The `Cluster.status()` command now reports any mismatch as an instance error, and the `Cluster.rescan()` command now detects if these items have changed and automatically updates the metadata to match the values taken from the instance. (Bug #33016337)

- When the `Cluster.addInstance` operation is used to add a server instance to an InnoDB Cluster, MySQL Shell checks for tables on the server that do not have primary keys, and fails the operation if any are present. If the user selected MySQL Clone as the provisioning method, which clears all the data on the server anyway, that check was still carried out although it was unnecessary. The check is now ignored when cloning is specified for the `Cluster.addInstance` operation. (Bug #32992693)

- The `dba.configureInstance()` function, which pre-checks instance configuration for InnoDB Cluster usage, did not handle the `log_bin` system variable correctly for MySQL 5.7 instances, where its behavior is different to MySQL 8.0 instances. The handling and output for the `log_bin` system variable have now been corrected. (Bug #31964706)

- MySQL Shell's `Cluster.setPrimaryInstance()` command uses a call to a server function to specify the instance that becomes the new primary server. However, if an error was returned from the server function, such as the specified server instance being unable to act as the primary due to its MySQL Server version being higher than others in the group, MySQL Shell did not check the error and reported the action as successful. MySQL Shell now checks for errors returned by server functions and displays them to the user. (Bug #31775698)

- When the `adoptFromGR` option of the `dba.createCluster` command was set to `false`, the behavior was the same as when the option was omitted, with a confirmation prompt provided and the operation proceeding if the user confirmed it. Specifying `adoptFromGR: false` now prevents cluster creation if the instance belongs to a Group Replication group, without presenting a prompt to the user. (Bug #30548447)

- AdminAPI's validation of the expel timeout value for group members (`group_replication_member_expel_timeout`), which can be set using the `expelTimeout` option when creating a cluster, has been corrected to take into account changes to the default and maximum values since the option was introduced in MySQL 8.0.13 and to make the check release specific. (Bug #29337169)

- If an instance was added to an InnoDB Cluster using MySQL Clone, and **Ctrl+C** or the `waitRecovery: 0` option was used to let recovery take place in the background, the recovery account from the donor instance was used instead of the recovery account created for the new instance. If the new instance was later removed with a `Cluster.removeInstance` command, the recovery account created for it was not dropped, since AdminAPI was only checking for the recovery account that was actually in use on the instance (the donor's account). The account removal logic now considers the account created for the instance and cleans it up even if it was not used. (Bug #105776, Bug #33630808)

- If MySQL Router was bootstrapped to use Unix sockets or to disable TCP ports, AdminAPI did not handle the resulting endpoint values correctly, resulting in errors during AdminAPI operations relating to MySQL Router management such as `Cluster.listRouters()`. (Bug #105649, Bug #33602309)

Functionality Added or Changed

- MySQL Shell's new diagnostics utility, `util.debug.collectDiagnostics`, enables you to gather diagnostic information from the connected MySQL server, generate reports in TSV and YAML formats, and present them in a zip archive in the location of your choice.

This utility enables you to retrieve diagnostic information from standalone servers, members of replication topologies, MySQL Database Service DB Systems, and so on.

Information retrieved includes the contents of the `performance_schema.error_log` and `processlist`, lists of global and persisted variables, replication group information, and a list of tables without primary keys.

See collectDiagnostics Utility. (WL #14863)

- MySQL Shell now supports the `--fido-register-factor` connection option to register a FIDO device for authentication to the server. You use the option with `mysqlsh` in the same way as with the `mysql` client program, and MySQL Shell passes on any errors raised by the MySQL client library or the client side authentication plugin. (WL #14769)

- In the event of a failure, MySQL Shell's instance dump utility `util.dumpInstance()`, schema dump utility `util.dumpSchemas()`, table dump utility `util.dumpTables()`, and dump loading utility `util.loadDump()` now report an error code and user-friendly message for recognizable errors. The messages provide context about the stage in the process where the error occurred, and the names of the objects involved. Error codes for the dump operations are in the range 52000-52999, and errors for the load operation are in the range 53000-53999. Error codes for connection and network errors that can be experienced by any of the utilities are in the range 54000-54999, and the error code matches the HTTP error involved. For example, error 54404 occurs when the target of a URL is not found. (WL #14841)

- MySQL Shell's `shell.prompt()` function, part of the MySQL Shell API, enables developers to create scripts for external applications to interact with a user through MySQL Shell to gather data. The function has new options to support additional prompt types besides a plain string value and a password prompt, comprising: a question with predefined answers, selecting from a list of options, a directory path, a file path for a new file to be saved, and a file path for an existing file to be opened. You can also supply a title, which MySQL Shell does not show to the user but applications can use to identify the prompt, and a description for the prompt, which MySQL Shell shows to the user as separate paragraphs before the prompt message. You can specify a default value for any of the prompt types to be returned to the external application if the user provides no data. (WL #14872)

Bugs Fixed

- When uninstalling MySQL Shell from the Debian or Ubuntu platforms, a number of files and directories were left behind. The uninstaller now clears these up. (Bug #33934625)

- MySQL Shell has a new API function `Column.getFlags()` that makes the list of enabled column flags available to the scripting interface. Reporting of column flags was standardized between X Protocol and classic MySQL protocol. (Bug #33916064)

- Some issues with MySQL Shell's handling of binary data have been corrected. BINARY, VARBINARY, and BLOB data types are now correctly reported as bytes, and TEXT, CHAR, and VARCHAR data types are reported as strings. For printed results, binary columns are encoded using Base64 for JSON output format, and as hex strings for other output formats. In the scripting layer, for Python, binary data is represented as binary strings, and for JavaScript, it uses ArrayBuffers; binary data can be inserted into the database using either of those formats. (Bug #33891697, Bug #32789317, Bug #32108582)

- When MySQL Shell's instance dump utility `util.dumpInstance()`, schema dump utility `util.dumpSchemas()`, and table dump utility `util.dumpTables()` add an invisible primary key column to a table with the `compatibility` option's `create_invisible_pks` modification, it is now added as the first column rather than the last column. (Bug #33880935)

- Throughput for MySQL Shell's JSON import utility `util.importJSON()` slowed greatly when the option `convertBsonTypes: true` or `convertBsonOid: true` was used. The cause of the slowdown (a JSON document parser method) has now been fixed. (Bug #33799202)

- To tolerate recent and future changes to the list of reserved keywords, MySQL Shell's instance dump utility `util.dumpInstance()`, schema dump utility `util.dumpSchemas()`, table dump utility `util.dumpTables()`, and dump loading utility `util.loadDump()` now quote all identifiers. (Bug #33744583)

- MySQL Shell's instance dump utility `util.dumpInstance()`, schema dump utility `util.dumpSchemas()`, and table dump utility `util.dumpTables()`, and dump loading utility `util.loadDump()` could not be used by a user account that contained an @ (at sign) character in the username. The issue could also occur with AdminAPI commands such as `dba.checkInstanceConfiguration` which retrieved information about the currently active user account. MySQL Shell now permits the @ (at sign) character in a user name when the account details have been returned by MySQL's CURRENT_USER() function. (Bug #33743612)

- The `BACKUP_ADMIN` privilege is no longer required to perform a consistent dump of a MySQL Server instance (with the `consistent` option). MySQL Shell's instance dump utility `util.dumpInstance()`, schema dump utility `util.dumpSchemas()`, and table dump utility `util.dumpTables()` now carry out a consistency check. If the user account does not have the `BACKUP_ADMIN` privilege and `LOCK INSTANCE FOR BACKUP` cannot be executed, the utilities make an extra consistency check during the dump. If this check fails, an instance dump is stopped, but a schema dump or a table dump continues and returns an error message to alert the user that the consistency check failed. (Bug #33699844, Bug #33697289)

- When setting up an SSH tunnel, MySQL Shell now selects from a list of approved SSH algorithms and parameters which excludes algorithms and parameters that do not meet acceptable security standards. (Bug #33670923)

- MySQL Shell's dump loading utility `util.loadDump()` could stop unexpectedly if two or more threads loading data from an OCI Object Storage bucket (using the `osBucketName` option and the local OCI profile) performed an operation that triggered the headers cache cleaning operation. The cache cleaning class is no longer shared between threads. (Bug #33654278)

- From MySQL Shell 8.0.27, the instance dump utility `util.dumpInstance()`, schema dump utility `util.dumpSchemas()`, and table dump utility `util.dumpTables()` returned an error when attempting to dump an NDB table. This was because the utility's partition awareness feature used an operation that is disabled for a storage engine that supports automatic partitioning. Partition awareness is now disabled for tables that use the NDB storage engine. (Bug #33643395)

- MySQL Shell's plugin manager now validates the MySQL Shell version and returns an error if a plugin cannot be installed on the current version. The URL of the official MySQL Shell plugin repository (https://cdn.mysql.com/mysql_shell/plugins_manifest.zip) has also been updated. (Bug #33642210)

- MySQL Shell's dump loading utility `util.loadDump()` could fail with the error "Invalid character in identifier" when loading a routine that was created when the ANSI_QUOTES SQL mode was in effect. This was caused by incorrect handling of identifiers quoted with double quotes. The issue has now been fixed, along with an issue in the dump utilities where identifiers in triggers were not quoted when required. (Bug #33640887)

- MySQL Shell's dump loading utility `util.loadDump()` now ensures that when the `deferTableIndexes` option is used to defer the creation of secondary indexes until after the table load, a full-text index is added first. Adding the first full-text index rebuilds the table, so doing this first ensures the other indexes do not need to be re-created. (Bug #33624751)

- Following a regression in MySQL Shell 8.0.28, zero dates returned by MySQL resulted in an exception in Python mode. The issue was fixed along with a number of other date and time value handling issues. (Bug #33621406)
- The online help for `autorejointries` listed the default value as 0. This was correct up to MySQL Shell 8.0.20. The default value since MySQL Shell 8.0.21 is 3. (Bug #31356257)

Changes in MySQL Shell 8.0.28 (2022-01-18, General Availability)

- Deprecation and Removal Notes
- AdminAPI Added or Changed Functionality
- AdminAPI Bugs Fixed
- Functionality Added or Changed
- Bugs Fixed

Deprecation and Removal Notes

- Support for the TLSv1 and TLSv1.1 connection protocols is removed from MySQL Server as of release 8.0.28. The protocols were deprecated from MySQL 8.0.26. For background, refer to the IETF memo Deprecating TLSv1.0 and TLSv1.1. Make connections between MySQL Shell and MySQL Server using the more-secure TLSv1.2 and TLSv1.3 protocols. TLSv1.3 requires that both the MySQL server and the client application be compiled with OpenSSL 1.1.1 or higher.

  If you attempt to make a connection using TLS/SSL from any version of MySQL Shell to a MySQL Server instance at 8.0.28 or above, and you specify the TLSv1 or TLSv1.1 protocol using the `--tls-version` option, you will see the following results:

  - For TCP connections, the connection fails, and an error is returned to MySQL Shell.
  - For socket connections, if `--ssl-mode` is set to `REQUIRED`, the connection fails. If `--ssl-mode` is not set to `REQUIRED`, the connection is made but with TLS/SSL disabled.

  (WL #14771)

AdminAPI Added or Changed Functionality

- When you create an InnoDB Cluster, InnoDB ClusterSet, or InnoDB ReplicaSet using MySQL Shell 8.0.28 and later, if you have security requirements that all accounts created automatically by AdminAPI have strict authentication requirements, you can set a value for the `replicationAllowedHost` configuration option. The `replicationAllowedHost` option means that all accounts created automatically can only connect from allowed hosts, using strict subnet-based filtering.

  You can use `setOption` option to set the `replicationAllowedHost` configuration on an existing InnoDB Cluster, InnoDB ClusterSet, or InnoDB ReplicaSet. (WL #14751)

- From MySQL Shell 8.0.28, three fencing operations are available:
  - *`Cluster`*`.fenceWrites()`: Stops write traffic to a primary cluster of a ClusterSet. Replica clusters do not accept writes, so this operation has no effect on them.
  - *`Cluster`*`.unfenceWrites()`: Resumes write traffic. This operation can be run on a cluster that was previously fenced from write traffic using the `cluster.fenceWrites` operation.
  - *`Cluster`*`.fenceAllTraffic()`: Fences a cluster from all traffic. If you have fenced a cluster from all traffic using *`Cluster`*`.fenceAllTraffic()`, you have to reboot the cluster using the `dba.rebootClusterFromCompleteOutage()` MySQL Shell command.

Even though you primarily use fencing on clusters belonging to a clusterset, it is also possible to fence standalone clusters using *`Cluster`*`.fenceAllTraffic()`. (WL #14537)

AdminAPI Bugs Fixed

- Following a regression in MySQL Shell 8.0.22, if an instance was part of an InnoDB Cluster and was removed from it, then added to a different InnoDB Cluster, the existing metadata schema for the first InnoDB Cluster was not cleared in the process. MySQL Shell now drops and re-creates the cluster metadata schema on the instance to ensure that it corresponds exactly to that for the new cluster. (Bug #33574005)

- Due to an error in a metadata query, MySQL Shell 8.0.27 cannot be used to create or manage an InnoDB Cluster running MySQL Server 8.0.25. The query was verifying a setting that was introduced in the following release. To work around this issue, upgrade MySQL Server to the 8.0.26 or 8.0.27 release on the InnoDB Cluster member instances before using MySQL Shell 8.0.27 with the cluster. The issue is fixed in MySQL Shell 8.0.28. (Bug #33532056, Bug #33580945)

- When the recovery method used to add an instance to an InnoDB Cluster was incremental state transfer from an existing member server's binary log (incremental recovery), AdminAPI uninstalled the clone plugin from the cluster members, which could cause subsequent auto-rejoin processes to fail. The uninstall no longer takes place. (Bug #33492812)

- AdminAPI's `cluster.addInstance()`, `cluster.rejoinInstance()`, and `dba.rebootClusterFromCompleteOutage()` commands did not inspect the `offline_mode` system variable, so if the variable was enabled, connection and replication errors occurred. These operations now inspect the value of the system variable and disable it if necessary. (Bug #33396423)

- The `groupSeeds` option is now deprecated for InnoDB Cluster creation commands and unsupported for InnoDB ClusterSet creation commands, and it has been undocumented. InnoDB Cluster's handling of the list of Group Replication group seed members (the `group_replication_group_seeds` system variable) has been revised to update the list when an instance is added, removed, or rejoined, and to check the list in the cluster metadata rather than on individual member server instances. (Bug #33389693, Bug #33573834)

- With an InnoDB ClusterSet deployment operating under heavy load, a replica cluster's status could be intermittently reported as `OK_NOT_CONSISTENT`, meaning that the set of transactions on the replica cluster (the GTID set) had more transactions than that on the primary cluster. This was happening because transactions were sent to replica clusters before they were added to the primary cluster's `gtid_executed` GTID set. Transactions that have been received but not yet applied are now subtracted from a replica cluster's `gtid_executed` GTID set before it is compared to that on the primary cluster. The set of retrieved transactions is shown in the extended output of the *`clusterSet`*`.status()` command. If the ClusterSet replication channel is reset or has its source changed, the set of received transactions is reset, so the inconsistency may be reported until the primary has externalized all the transactions that took place before the reset. (Bug #33330871)

- If an InnoDB Cluster was rebooted after an outage using the `dba.rebootClusterFromCompleteOutage()` command, and then rejoined to an InnoDB ClusterSet using the `clusterSet.rejoinCluster()` command, the rejoin failed if the `rejoinInstances` or `removeInstances` option had been used on the `dba.rebootClusterFromCompleteOutage()` command. These options now cannot be used if the InnoDB Cluster is part of an InnoDB ClusterSet and has been invalidated. (Bug #33301229)

- The *`clusterSet`*`.removeCluster()` command for InnoDB ClusterSet now implicitly dissolves an InnoDB Cluster when it is removed from the ClusterSet, so that all the members become standalone

instances. This enables the instances to be reused in a new cluster in the same or another ClusterSet without any conflicting metadata. (Bug #33267834)

Functionality Added or Changed

- MySQL 8.0.28 and higher includes Connection timeout MySQL Shell options, to allow users to set timeouts for sessions using both AdminAPI and non-AdminAPI connections.

  Certain operations that open many connections to servers can take a long time to execute when one or more servers are indeed unreachable, for example, the `cluster.status()` command. The connection timeout may not provide enough time for a response.

  From MySQL Shell 8.0.28, you can use the MySQL Shell configuration option `connectTimeout` to set the connection timeout in seconds for any session not using AdminAPI.

  From MySQL Shell 8.0.28, you can use the MySQL Shell configuration option `dba.connectTimeout` to set the connection timeout in seconds for any session using AdminAPI. (WL #14698)

- MySQL Shell now supports SSH tunneling for connections to MySQL server instances. Once established, an SSH tunnel can be shared between connections to the same host from the same user connecting from the same remote server instance. MySQL Shell's Secret Store can store passwords and passphrases for connection to an SSH server and for the identity file, to be automatically retrieved for future connections. A new `shell.listSshConnections()` function returns a list of the active SSH tunnels. You can configure the buffer size for data transfer using the MySQL Shell configuration option `ssh.bufferSize`.

  You can specify the SSH connection details in the standard SSH configuration file (`~/.ssh/config`), or a custom configuration file named by the MySQL Shell configuration option `ssh.configFile`. You can override the set defaults using the connection option `ssh-config-file`. The identity file can be the standard private key file in the SSH configuration folder (`~/.ssh/id_rsa`), or a key file named by the connection option `ssh-identity-file`. You can use the `--ssh` option on the MySQL Shell `\connect` command or the command line to provide the URI for connection to the SSH server, or you can use the `shell.connect()` method and specify the connection data as key-value pairs.

  The use of AdminAPI commands is not supported over connections made using SSH tunneling, with the exception of the commands to deploy, start, stop, kill, and delete sandbox instances. (WL #14246)

- MySQL 8.0.27 and higher includes support for multifactor authentication. This capability includes forms of MFA that require up to three authentication values (2FA and 3FA). The `authentication_policy` system variable defines how many authentication factors accounts may have (or are required to have) and the authentication methods that can be used for each factor. To enable authentication to the MySQL server using accounts that require multiple passwords, MySQL Shell now supports the `--password1`, `--password2`, and `--password3` options for command-line connections, permitting up to three passwords to be specified. The existing `--password` option and the `--password1` option are treated as equivalent. You can specify a password value following the option on the command line (which is insecure), or if the options are given without a password value, MySQL Shell prompts the user for each password in turn. These options are only supported for classic MySQL protocol connections made using command-line arguments. (WL #14649)

- MySQL Shell's instance dump utility `util.dumpInstance()`, schema dump utility `util.dumpSchemas()`, table dump utility `util.dumpTables()`, and dump loading utility `util.loadDump()` have new filtering options to cover all objects that can be dumped and loaded. The options give you fine control over the content of the dump files and over what objects are loaded onto the destination server, and enable you to skip any objects that are causing issues. At an appropriate level for the utility, the options allow you to include or exclude a specified list of schemas, tables, routines, events, or triggers. (WL #14244)

Bugs Fixed

- MySQL Shell 8.0.28 does not support the following Linux distribution:

  - SUSE Linux Enterprise Server 12 (SLES12)
  - Enterprise Linux 6 (EL6)

  (Bug #33787657)

- MySQL Shell's instance dump utility `util.dumpInstance()`, schema dump utility `util.dumpSchemas()`, and table dump utility `util.dumpTables()` could not be used with a MySQL server instance started with the `--skip-grant-tables` option, meaning that the utilities could not be used to extract data in this emergency situation. The utilities now detect the special account used for all connections in this situation and assume that account has all available privileges. The `users` option is also automatically set to `false`, because some grant information cannot be collected in this situation. (Bug #33592520)

- When loading a dump from a manifest file, if the PARs in the file had expired, MySQL Shell's dump loading utility `util.loadDump()` would fail with an unclear message. The utility now compares the current time and the expiry time of the PARs, and if they have expired, returns an appropriate message and stops the load process. (Bug #33508311)

- A hardcoded domain name was used in MySQL Shell code relating to PARs (pre-authenticated requests), resulting in an error if a domain name other than oraclecloud.com was used in a PAR. The issue has now been corrected and any domain name is permitted. (Bug #33506737)

- MySQL Shell's instance dump utility `util.dumpInstance()` stopped the dump with an error if the instance had no schemas or filtering resulted in an empty set of schemas, so it was not possible to just dump the users from an instance. The operation now succeeds in that situation if the `users` option is set to `true` (the default), and continues to stop with an error if the `users` option is set to `false`. (Bug #33502098)

- On Windows, when MySQL Shell was started by another application, a prompt expecting a yes or no answer (such as whether to store a supplied password in the credential manager) was repeated indefinitely even if a valid answer was received. This was due to the carriage return character (`\r`) not being trimmed in addition to the newline character (`\n`). The issue has now been fixed. (Bug #33499602)

- When the `--json=raw` option was used to generate raw JSON from MySQL Shell's output, newlines were not included, resulting in unformatted text. (Bug #33486098)

- On Windows, a regression meant that MySQL Shell could not connect to a server using a URI-like connection string that specified a default schema but did not specify a scheme element (such as `mysqlx://`). (Bug #33485693)

- For an extension object registered with MySQL Shell, the help lookup function now searches for an exact match to the search string in the event that multiple topics are found. Plugin help, which was previously loaded only in local context, is now loaded in global or local context depending on where the plugin is loaded. (Bug #33468265)

- For an extension object registered with MySQL Shell as a member of a higher-level extension object, Python decorators did not take naming conventions into account when looking for the object hierarchy, but the registration function did, which could cause incorrect results. The registration function now converts the object name using MySQL Shell’s algorithm so that the object is properly found. (Bug #33462107)

- MySQL Shell could not be built from source using Python 3.10 due to API changes. The issue has now been corrected. (Bug #33454039)
- For an extension object registered with MySQL Shell, the help lookup did not distinguish between objects with the same name where only one was available as a global object, and the other was a subsidiary object. The help registration and help lookup functions have now been updated to match the fully qualified ID of an object, and also prioritize any help item whose fully qualified ID exactly matches the search string. (Bug #33451028)
- MySQL Shell's dump loading utility `util.loadDump()` can defer the creation of secondary indexes until after the table data is loaded, with the `deferTableIndexes` option. Previously, the deferred indexes could not be created on tables with a secondary engine defined, because DDL statements cannot be executed on these tables. The utility now removes the SECONDARY_ENGINE clause from the table if indexes are being deferred, and adds it back in after the indexes have been created. (Bug #33414321)
- MySQL Shell's instance dump utility `util.dumpInstance()`, schema dump utility `util.dumpSchemas()`, and table dump utility `util.dumpTables()` now chunk tables with an index column of data type DECIMAL using arithmetic operations, which improves the execution speed. (Bug #33400387)
- MySQL Shell's instance dump utility `util.dumpInstance()`, schema dump utility `util.dumpSchemas()`, and table dump utility `util.dumpTables()` now print a summary of the number of dumped schemas, tables, views, events, routines, triggers, and users, and the number of these objects that are available in the database. (Bug #33396153)
- MySQL Shell stopped if it was not able to write to the directory specified by the `$HOME` environment variable. An error is now returned when MySQL Shell starts up if this is the case. (Bug #33390034)
- When MySQL Shell's `\source` command was used to execute a script file, if the script did not contain a complete statement, MySQL Shell entered multiline mode instead of executing the script and returning an appropriate error. MySQL Shell's stream processing now disables multiline handling to force the cached code to be executed even if it is incomplete, so that an error is returned. (Bug #33337598)
- When MySQL Shell's dump loading utility `util.loadDump()` was used to load a dump that was currently in progress, the utility's progress reporting was inaccurate. (Bug #33332497)
- When MySQL Shell's dump loading utility `util.loadDump()` was used to load a dump that was currently in progress using a prefix PAR, an error was reported for the dump's index file. (Bug #33332080)
- MySQL Shell masks passwords that are supplied after the user name in a URI-like string specified by the `--uri` command-line option. If the user name in the string was percent encoded and a password was supplied, the comparison to a plain user name was not made correctly, and the connection failed. MySQL Shell now removes the password from a URI-like string leaving only the user name before the comparison is carried out. (Bug #33273652, Bug #104714)
- MySQL Shell's built-in global objects can be accessed using Python's subscript operator, for example, `dba["help"]()`. However, an unspecified error was returned if an invalid type was used as a subscript. `TypeError` is now returned in this situation. (Bug #32315014)

Changes in MySQL Shell 8.0.27 (2021-10-19, General Availability)

- Deprecation and Removal Notes
- AdminAPI Bugs Fixed
- Functionality Added or Changed

- Bugs Fixed

Deprecation and Removal Notes

- The options for AdminAPI's `dba.rebootClusterFromCompleteOutage()` operation to specify a user and password to run the required tasks are now deprecated and should not be used. The user and password are taken from the active MySQL Shell session. (Bug #26586818)

AdminAPI Bugs Fixed

- MySQL Shell's progress reporting for InnoDB Cluster operations could result in non-JSON output being produced when MySQL Shell was set to return JSON output. The issue has now been corrected. (Bug #33237648)
- The recovery user did not support SSL. As of this release, if the server is configured to use SSL, the recovery user is also configured to use it. (Bug #31227280)
- InnoDB Cluster error messages for an issue with a specific target instance now include the host name and port for the affected instance, so that it can be identified if the operation involved multiple instances. (Bug #28365584)
- AdminAPI's `dba.stopSandboxInstance()` operation attempted to verify the existence of the PID file for logging purposes, but the operation stopped if the file was not found. The operation now proceeds in the absence of the file, and uses the sandbox port to provide the logging information. (Bug #25096293)

Functionality Added or Changed

- MySQL InnoDB ClusterSet is a new solution that provides disaster tolerance for MySQL InnoDB Cluster deployments by linking a primary InnoDB Cluster with one or more replicas of itself in alternate locations, such as different datacenters. InnoDB ClusterSet automatically manages replication from the primary cluster to the replica clusters using a dedicated ClusterSet replication channel. If the primary cluster becomes unavailable due to the loss of the data center or the loss of network connectivity to it, you can make a replica cluster active instead to restore the availability of the service.

  Emergency failover between the primary InnoDB Cluster and a replica cluster in an InnoDB ClusterSet deployment can be triggered by an administrator through MySQL Shell, using AdminAPI, which is included with MySQL Shell. You can also carry out an active controlled switchover while the primary cluster is still available, for example if a configuration change or maintenance is required on the primary cluster. MySQL Router automatically routes client applications to the right clusters in an InnoDB ClusterSet deployment. (WL #12804, WL #12805, WL #13815, WL #11894, WL #14538)

- MySQL Shell's instance dump utility `util.dumpInstance()`, schema dump utility `util.dumpSchemas()`, and table dump utility `util.dumpTables()` now automatically detect partitioned tables in a dump, and parallelize chunking and dumping of each partition as an independent table. The data files in the dump are named in the format `schema@table@partition`. MySQL Shell's dump loading utility `util.loadDump()` then schedules loading for the chunks of each partition as if they belonged to different tables, improving parallelization and performance.

  Dumps created using the utilities from MySQL Shell 8.0.27 onwards cannot be loaded by versions of MySQL Shell before 8.0.27, due to incompatible changes in the metadata manage future additions of features that are not supported by older versions of the utilities, a capability list is also introduced in this release, which records the features that are actually used when creating a dump. From MySQL Shell 8.0.27 onwards, the dump loading utility checks the list and reports an error if it finds a feature there that it does not support. In future releases, the capability list will allow you to attempt a dump load with older versions of MySQL Shell and succeed wherever possible. (WL #14632)

- Oracle Cloud Infrastructure Object Storage now supports using a single pre-authenticated request (PAR) for accessing all objects in a bucket or objects in a bucket with a specific prefix. MySQL Shell's dump loading utility `util.loadDump()` has added support for this type of PAR. The previous method of using a PAR created for a MySQL Shell dump manifest file requires generating a PAR for each item in the dump, which is time consuming when exporting a large dataset. This method also requires recreating dumps when PARs expire. When using bucket and prefix PARs, only the PAR needs to be recreated, not the entire dump.

  With the introduction of support for bucket and prefix PARs, the MySQL Shell dump utility's `ociParManifest` option, which was previously set to `true` when the `ocimds` option was enabled and the `osBucketName` was specified (in order to generate the MySQL Shell dump manifest file with a PAR for each file in the dump), is now set to false by default and is only enabled if set to `true` explicitly. (WL #14645)

- The new `maxBytesPerTransaction` option, introduced for use with MySQL Shell dump loading utility `util.loadDump()`, defines the maximum number of bytes that can be loaded from a data file in a single `LOAD DATA` statement. If a data file exceeds the `maxBytesPerTransaction` size, multiple `LOAD DATA` statements load the data from the file in chunks less than or equal to the `maxBytesPerTransaction` value. An intended use for this option is to load data in smaller chunks when a data file is too large for the target server's limits, such as the limits defined by the server's `group_replication_transaction_size_limit` or `max_binlog_cache_size` settings. For more information, see Dump Loading Utility. (WL #14577)

- MySQL Shell now has full support for LDAP authentication methods and Kerberos authentication for classic MySQL protocol connections. Previously, some client-side plugins needed for these authentication methods were not available when using MySQL Shell, which used the MySQL client library for client-side authentication for classic MySQL protocol connections. Also, for Kerberos authentication, MySQL Shell could not take advantage of a cached Ticket Granting Ticket (TGT) that eliminates the need for a user name and password in the authentication process. This is because MySQL Shell automatically provides a system user name, prompts for a password, or supplies a password using the Secret Store Helper, if the user name or password are not present when accessing a system.

  The new persistent `shell.options.mysqlPluginDir` setting and non-persistent command-line option `--mysql-plugin-dir` can be used to specify paths to client-side authentication plugins that are not built into the `libmysqlclient` client library, including `authentication_ldap_sasl_client` and `authentication_kerberos_client`. These plugins are required to support SASL-based LDAP authentication and Kerberos authentication.

  When the `--auth-method` command option is used to specify the `mysql_clear_password` plugin, which is required for simple LDAP authentication, MySQL Shell now enables and uses the plugin. This client-side plugin is built in to the MySQL client library, but for security it is not enabled by default. This authentication type is only suitable for a secure connection that uses SSL or sockets.

  Cached TGTs for Kerberos authentication are supported when the `--auth-method` option is used to specify the `authentication_ldap_sasl_client` or `authentication_kerberos_client` plugin. When one of these options is set for a connection, MySQL Shell does not supply the system user name if the user name is missing, does not prompt for a password, and does not attempt to use the Secret Store helper to retrieve or store credentials. (WL #14553)

Bugs Fixed

- **Incompatible Change:** To manage additions of features that are not supported by older versions of the utilities, MySQL Shell's instance dump utility `util.dumpInstance()`, schema dump utility `util.dumpSchemas()`, table dump utility `util.dumpTables()`, and dump loading utility

`util.loadDump()` now write a list of capabilities to the metadata file for a dump. If a feature is used while creating the dump, it is written to the capability list, otherwise it is omitted. The dump loading utility checks the capability list, and reports an error if it finds a feature. Partitioning awareness, introduced in this release, is the first feature compatible with this feature management capability.

Due to this change, older releases of MySQL Shell cannot read the metadata produced by the dump utilities from MySQL Shell 8.0.27 and later, which means they cannot import dumps made with the later versions. (Bug #33063035)

- To improve MySQL Shell's integration capabilities when embedded in other applications, a new JSON shell mode has been added that can be activated by defining the `MYSQLSH_JSON_SHELL` environment variable. In this mode, MySQL Shell accepts the following commands formatted as JSON documents:

  - `{"execute":`*`json-string`*`}`

    Executes the given code in the active MySQL Shell mode (JavaScript, Python or SQL). The code is executed as a complete unit, and an error is returned if it is incomplete.

  - `{"command":`*`json-string`*`}`

    Executes the given MySQL Shell command.

  - `{"complete":{"data":`*`json-string`*`[, "offset": `*`uint`*`}}`

    Determines the options for auto-completion based on the given data and the current MySQL Shell context.

  Also, an issue was fixed where quotes were incorrectly added to string values when they were output in MySQL Shell's JSON mode. (Bug #33310170)

- When MySQL Shell's instance dump utility `util.dumpInstance()`, schema dump utility `util.dumpSchemas()`, table dump utility `util.dumpTables()`, and dump loading utility `util.loadDump()` were interacting with an Oracle Cloud Infrastructure Object Storage bucket, minor clock variations could lead to an HTTP 401 error relating to the date of the Authorization header. This was caused by the utilities reusing headers when they retried a request. The utilities now refresh the headers for a request periodically, and also resend the request automatically in case of this specific error. (Bug #33271037)

- When MySQL Shell's command-line option `--file` is used to specify a file to process, the arguments after the file name are now passed to the script in interactive mode as well as in batch mode. (Bug #33235685)

- MySQL Shell's instance dump utility `util.dumpInstance()`, schema dump utility `util.dumpSchemas()`, and table dump utility `util.dumpTables()` returned an error when attempting to chunk a table where a column name contained a comma character. The issue has now been fixed. (Bug #33232480)

- A progress checker for MySQL Shell's instance dump utility `util.dumpInstance()`, schema dump utility `util.dumpSchemas()`, table dump utility `util.dumpTables()`, and dump loading utility `util.loadDump()` did not check what output format was set for MySQL Shell. As a result, some non-JSON output was produced when JSON mode output was selected and the option `showProgress: true` was set for the utility. (Bug #33223635)

- The value for the `waitDumpTimeout` option for MySQL Shell's dump loading utility `util.loadDump()` was not validated correctly and negative values could be used. (Bug #33212873)

- When the `ociParManifest` option is used, MySQL Shell's instance dump utility `util.dumpInstance()`, schema dump utility `util.dumpSchemas()`, and table dump utility `util.dumpTables()` now present progress information or a start and end message for the process of creating PARs and writing the manifest file. (Bug #33181308)
- When MySQL Shell was built with Python 3.8 or later, which set the locale to the user's preference, Unicode characters in the MySQL Shell prompt were not displayed correctly on Windows. (Bug #33174999)
- Due to varying privilege requirements for the `FLUSH TABLES WITH READ LOCK` statement, MySQL Shell's instance dump utility `util.dumpInstance()`, schema dump utility `util.dumpSchemas()`, and table dump utility `util.dumpTables()` did not always correctly identify when the statement would succeed. If the statement failed, the dump was canceled rather than falling back to the use of a `LOCK TABLES` statement. Now, if the `FLUSH TABLES WITH READ LOCK` statement fails because access was denied, a `LOCK TABLES` statement is used instead. The `FLUSH TABLES WITH READ LOCK` statement is attempted first to keep the dry run consistent with the actual run, but it is set to a small timeout and releases the lock immediately if acquired. (Bug #33173739)
- When the `MYSQLSH_USER_CONFIG_HOME` environment variable was used to set a custom MySQL Shell configuration path, the path to the `prompt.json` file was built incorrectly if a path separator was not appended. The path build has been corrected to append the separator if needed. (Bug #33164726)
- When the `ocimds` option is enabled for MySQL Shell's instance dump utility `util.dumpInstance()`, schema dump utility `util.dumpSchemas()`, and table dump utility `util.dumpTables()`, the utilities now flag explicit grants on objects in the `mysql` schema, which cannot be imported into a MySQL Database Service instance. The `compatibility` option's `strip_restricted_grants` modification now removes such grants. (Bug #33162928)
- When the `ocimds` option is used with MySQL Shell's instance dump utility `util.dumpInstance()`, schema dump utility `util.dumpSchemas()`, or table dump utility `util.dumpTables()` to enable checks and modifications for compatibility with MySQL Database Service, the utilities now make an additional check for the number of columns in each table. If a table exceeds the `InnoDB` limit of 1017 columns, this is reported as an issue requiring a manual fix. (Bug #33159903)
- When MySQL Shell's dump loading utility `util.loadDump()` was used to load a dump from a pre-authenticated request (PAR) URL, if the URL was invalid, the utility did not recognize it as an attempted PAR, and the resulting error message was not informative. The utility now recognizes an invalid PAR and returns an error message including the expected URL format. The PAR URL itself is replaced with the word `secret` in error messages and the log. (Bug #33155373)
- MySQL Shell's dump loading utility `util.loadDump()` used a small input buffer when downloading data files written using zstd compression. This slowed performance for nonlocal files as each request was made separately. The buffer size has now been increased. (Bug #33150332)
- When a MySQL user account name included a netmask as part of the host value, MySQL Shell's dump loading utility `util.loadDump()` did not handle this correctly when initializing the load operation. As a result, user accounts with a netmask could not be used to run the utility. The issue has now been fixed. (Bug #33144419, Bug #104375)
- MySQL Shell's dump loading utility `util.loadDump()` now sets default roles for users after loading `GRANT` statements, to ensure that all the roles have been created and the appropriate grants have been granted. (Bug #33128803, Bug #104339)
- Some queries used by MySQL Shell's instance dump utility `util.dumpInstance()`, schema dump utility `util.dumpSchemas()`, and table dump utility `util.dumpTables()` had a dependency on the server version, including the use of `EXPLAIN SELECT` for chunking and identifying support for roles. The utilities now make the relevant checks without referencing the server version. (Bug #33103480)

- On Unix systems, the SIGINT signal did not interrupt a prompt or password prompt when sent from outside MySQL Shell. MySQL Shell now handles the signal consistently. (Bug #33096667)
- A data type mapping issue meant that Python plugins for MySQL Shell could not retrieve the column type for multiple columns using `column.get_type()`. (Bug #33084551)
- When the `strip_definers` compatibility requirement is applied to tables dumped by MySQL Shell's instance dump utility `util.dumpInstance()`, schema dump utility `util.dumpSchemas()`, and table dump utility `util.dumpTables()`, the `SQL SECURITY` clause for views and routines is changed to specify `INVOKER` instead of `DEFINER`. If a stored routine contained only a simple statement, the clause was inserted at the wrong location, resulting in a dump that could not be loaded. The clause was also incorrectly omitted in some circumstances. The issues have now been corrected, and in addition the lack of the `SQL SECURITY` clause for a view or routine is now reported separately as a compatibility issue. (Bug #33079172, Bug #33087120)
- MySQL Shell's help search function produced duplicate topic results if the pattern matching used a wildcard prefix. (Bug #33060756)
- When a user-defined MySQL Shell Python plugin used the `@plugin_function()` decorator, the required attribute for a dictionary entry was overwritten to `false`. The issue has now been fixed. (Bug #33026024)
- Python 3.9.5 was bundled with MySQL Shell 8.0.26 was missing libraries required for Paramiko and for the Oracle Cloud Infrastructure SDK for Python. (Bug #32985104)
- The information gathering phase for MySQL Shell's instance dump utility `util.dumpInstance()` has been improved to reduce the time required to identify and fetch metadata from the source server. Filtering queries to retrieve metadata are now limited to the filtering specifically requested by the user, to improve the response time. Some unnecessary calls and requests have been removed, and threading and connection keep-alive functions have been added. As a result of the changes, the `EXECUTE` privilege is no longer required when a view in the dump calls a function to get its data. (Bug #32969290)
- Progress reporting and the final summary report for MySQL Shell's instance dump utility `util.dumpInstance()`, schema dump utility `util.dumpSchemas()`, and table dump utility `util.dumpTables()` have been improved. The final summary report now shows the duration of a dump with and without the preparation step, which was previously excluded from the time. The progress reporting includes additional steps, and status messages are now logged to the MySQL Shell application log file as informational messages, rather than being printed to the console. An error in calculating the throughput has been fixed. (Bug #32969287)
- MySQL Shell's instance dump utility `util.dumpInstance()`, schema dump utility `util.dumpSchemas()`, table dump utility `util.dumpTables()`, and dump loading utility `util.loadDump()` now log any SQL statements that result in an error being returned, as well as logging the error. (Bug #32966510)
- An incorrect assumption in the chunking algorithm for MySQL Shell's instance dump utility `util.dumpInstance()`, schema dump utility `util.dumpSchemas()`, and table dump utility `util.dumpTables()` meant that if an individual table row exceeded the size specified by the `bytesPerChunk` option, and should therefore have been in its own chunk file, other rows might be written to the file as well. The algorithm has now been corrected. (Bug #32955616)
- When the consistent option is set to false for MySQL Shell's instance dump utility `util.dumpInstance()`, schema dump utility `util.dumpSchemas()`, and table dump utility `util.dumpTables()`, the instance is not locked for backup while the dump is taken. The utilities now check the `gtid_executed` GTID set at the start and end of the dump process, and issue a warning if they are different. (Bug #32954757)

- MySQL Shell's dump loading utility `util.loadDump()` could stop with a deadlock error on a slow server when executing DDL for a table that is referenced by an existing view. Previously, the operation was retried up to 20 times with a 200ms delay between. Now, the delay is doubled after each attempt, and the utility retries for up to five minutes. (Bug #32928528)

- The help and documentation for the `excludeUsers` and `includeUsers` options of MySQL Shell's instance dump utility `util.dumpInstance()`, schema dump utility `util.dumpSchemas()`, table dump utility `util.dumpTables()`, and dump loading utility `util.loadDump()` incorrectly stated that a user name specified without a host was equivalent to `'user_name'@'%'`. Actually all user accounts with that user name are excluded. MySQL Shell also now excludes certain accounts that should always be excluded, even if they are not specified using the `excludeUsers` option. (Bug #32799076)

- MySQL Shell's parallel table import utility `util.importTable()` was unable to import a compressed input file if the content exceeded the `max_binlog_cache_size` setting for the server. If you receive the error `"MySQL Error 1197 (HY000): Multi-statement transaction required more than 'max_binlog_cache_size' bytes of storage"` on attempting to import a file, set the utility's new option `maxBytesPerTransaction` to a value less than or equal to the server instance's `max_binlog_cache_size` setting. The minimum value that the utility will use is 4096 bytes. When the option is set, the utility uses multiple `LOAD DATA` statements to request larger input files in chunks that fit into the specified cache size. When the option is not set, compressed input files are loaded in a single block. (Bug #32575351)

- MySQL Shell's dump loading utility `util.loadDump()` now includes the UUID of the target server instance in the progress file. If the load operation is stopped and resumed, the utility checks that the saved UUID matches the UUID of the current target server, and returns an error and halts the operation if it does not. This check ensures that a meaningful error is returned in case of accidental use of the wrong progress file due to copying the command to run the utility. (Bug #32561035)

- MySQL Shell's plugin handling assumed that packages used the Egg packaging format, leading to errors in locating and loading modules. The Wheel packaging format is now taken into account when handling module paths. (Bug #32451772)

- MySQL Shell's instance dump utility `util.dumpInstance()`, schema dump utility `util.dumpSchemas()`, and table dump utility `util.dumpTables()` failed while the user was connected to the server using a socket connection, unless the `--host` option was also specified. The utilities include a description of the host in a comment in the dump DDL files, and incorrectly assumed this would be present in the connection options data. The host is now set to `localhost` for the comment if the `--host` option is not specified. (Bug #32109967, Bug #101439)

- MySQL Shell's dump loading utility `util.loadDump()` did not provide progress updates while the metadata files were being scanned for a dump loaded from an Oracle Cloud Infrastructure Object Storage bucket. For a large dump, it could appear that progress had stalled. The utility now reports progress during this stage, and stores the timestamp of each operation in the progress log. A number of performance improvements have also been made, including multithreaded scanning and the use of a pool of background threads. The size of the thread pool can be set using the new `backgroundThreads` option, and defaults to the value of the `threads` option for a dump loaded from the local server, or four times the value of the `threads` option for a dump loaded from a non-local server. (Bug #31645798)

- A new command-line option `--log-file` has been added that lets you change the location of MySQL Shell's application log file `mysqlsh.log` from the default. The default location is the user configuration path, `%APPDATA%\MySQL\mysqlsh\` on Windows or `~/.mysqlsh/` on Unix. It was previously possible to override the user configuration path by defining the environment variable `MYSQLSH_USER_CONFIG_HOME`, but now the `--log-file` option lets you do this from the command line when you start an individual MySQL Shell instance, so that different instances can write to different locations. (Bug #27253110)

- Comments entered interactively in MySQL Shell's Python mode were not included in the MySQL Shell history, although JavaScript comments were. Both are now included. (Bug #26484975)
- When MySQL Shell's command-line option `--dba=enableXProtocol` was used to install and enable X Plugin on connection with a MySQL 5.7 server, the process verified that the install command did not return any errors, but did not verify that the plugin had installed correctly and accepted the appropriate connection type. MySQL Shell now verifies the plugin state before returning a response to the user so they can see whether the plugin is now usable. (Bug #24554967, Bug #82759)
- Compiling without Python support resulted in a fatal error. This was caused by Python test files included in the compilation process. (Bug #101923, Bug #32256576)

Changes in MySQL Shell 8.0.26 (2021-07-20, General Availability)

- Deprecation and Removal Notes
- AdminAPI Bugs Fixed
- Bugs Fixed

Deprecation and Removal Notes

- The TLSv1 and TLSv1.1 connection protocols are now deprecated, and support for them is subject to removal in a future MySQL Server version. (For background, refer to the IETF memo Deprecating TLSv1.0 and TLSv1.1.) It is recommended that connections be made using the more-secure TLSv1.2 and TLSv1.3 protocols. TLSv1.3 requires that both the MySQL server and the client application be compiled with OpenSSL 1.1.1 or higher.

  MySQL Shell's `--tls-version` command option, which is one of the options available to configure an encrypted connection at startup of MySQL Shell, now returns a deprecation warning if you specify TLSv1 or TLSv1.1 explicitly as part of the value for the option. (WL #14560)

AdminAPI Bugs Fixed

- The last instance that remains in `ONLINE` status in an InnoDB Cluster cannot be removed using the *`cluster`*`.removeInstance()` operation. Previously, when this was attempted, the operation failed with an unrelated error, but it is now handled correctly. (Bug #32955287)
- AdminAPI operations no longer change the Group Replication communication protocol version automatically in situations where it is possible to increase the version due to a change in group membership. The automated operations caused delays on changes to the cluster. Instead, where relevant, the operations return a message to inform the user that a version upgrade should be carried out. A new option `upgradeCommProtocol` has been added to the `rescan()` operation so that the administrator can upgrade the communication protocol version when they choose to do so. (Bug #32769580)

Bugs Fixed

- When loading a dump that was already in progress, MySQL Shell's dump loading utility `util.loadDump()` flagged a table as ready to process before the metadata was read, which could potentially result in an issue in the event that the metadata was not completely downloaded and parsed before processing began on the table. (Bug #32934878)
- When MySQL Shell's dump loading utility `util.loadDump()` was used to load a dump that contained a zero value in an AUTO_INCREMENT column, the utility did not set the NO_AUTO_VALUE_ON_ZERO SQL mode, meaning that the next sequence number was generated and used instead. The behavior has now been corrected. (Bug #32926856)

- MySQL Shell's instance dump utility `util.dumpInstance()`, schema dump utility `util.dumpSchemas()`, and table dump utility `util.dumpTables()` now report an error for a table that uses `ROW_FORMAT=FIXED` when the `ocimds` option is used, and remove that clause from `CREATE TABLE` statements when the `force_innodb` compatibility modification is specified. (Bug #32925914)
- MySQL Shell now bundles Python 3.9.5 for platforms where Python 3 is not included or is not at the minimum supported version. The previous bundled version was Python 3.7.7. (Bug #32924896)
- From MySQL Shell 8.0.26, you can use MySQL Shell's instance dump utility `util.dumpInstance()` to dump user accounts from a MySQL 5.6 instance. The SUPER privilege is required when user accounts are included in the dump. (Bug #32883314)
- The error messages returned by MySQL Shell's instance dump utility `util.dumpInstance()`, schema dump utility `util.dumpSchemas()`, and table dump utility `util.dumpTables()` have been enhanced to provide more context for the error, such as the schema or table involved and the step that was being executed. The utilities also now cache frequently used values such as quoted table names. Other enhancements have been made to optimize the chunking procedure. (Bug #32850177)
- MySQL AdminAPI now uses the terms “source”, “replica”, and “mta” for target MySQL instances where these terms are used in system variables and other areas, and user messages have been updated to use the new terms. (Bug #32774826)
- If the primary key of a table was a signed integer, an overflow in the data range during chunking by MySQL Shell's instance dump utility `util.dumpInstance()`, schema dump utility `util.dumpSchemas()`, and table dump utility `util.dumpTables()` could cause MySQL Shell to consume excessive memory and disk space. (Bug #32773468)
- Previously, when MySQL Shell's instance dump utility `util.dumpInstance()`, schema dump utility `util.dumpSchemas()`, and table dump utility `util.dumpTables()` were run with the `dryRun` option set to `true`, so that the data was not actually dumped, the utilities attempted to acquire the needed table locks. To avoid impacting on the database, the utilities now only check whether the user ID has the appropriate privilege to acquire a lock, and behave according to the result. Any missing privileges are reported. The utilities’ method of fetching user privileges has also been refactored to use the `SHOW GRANTS` statement, and to support partial revokes. (Bug #32695301)
- X DevAPI’s ClassicSession and Session objects have a new method `getSocketFd` to fetch the session’s socket descriptor. The Session object has new methods `enableNotices` and `fetchNotice` to fetch asynchronous notifications published by X Protocol for Group Replication events such as view changes. (Bug #32665405)
- On UNIX systems, the list of plugins could not be retrieved from MySQL Shell’s plugins repository because the manifest path was specified in Windows format. The path has now been changed to POSIX mode. (Bug #32663889)
- MySQL Shell’s command-line integration did not support deprecated options and treated them as invalid. (Bug #32651899)
- MySQL Shell's instance dump utility `util.dumpInstance()`, schema dump utility `util.dumpSchemas()`, and table dump utility `util.dumpTables()` use an algorithm to divide table data into chunks. If the algorithm carried out chunking using an integer column, large gaps in the sequence of integers caused by deleted rows could result in the ranges and chunks becoming increasingly larger until the benefit of chunking was lost. The algorithm has now been improved to use an increased number of iterations and make additional checks if the results are not as expected. (Bug #32602325)
- MySQL Shell’s log now records more detailed information when there is an issue with an Oracle Cloud Infrastructure operation, including the target URL, the request and response headers, and the response

data. If the target URL is a pre-authenticated request (PAR), the sensitive components are masked in the log file.

An issue with a change to the logging level of messages not activating message logging at the right level has also been fixed. (Bug #32593125)

- MySQL Shell retrieved SQL identifiers that used quoting as a series of separate tokens. (Bug #32532347)
- MySQL Shell's instance dump utility `util.dumpInstance()`, schema dump utility `util.dumpSchemas()`, and table dump utility `util.dumpTables()` use multiple threads to dump data from the MySQL instance, each using its own session. If one of these threads was slow to start and an exception was thrown in the main dumper thread in the meantime, a segmentation fault occurred when the thread tried to access the main dumper thread's session. The utilities now keep the main dumper thread's session available even after an exception is thrown, and only delete it when the utility ends. (Bug #32528110)
- In MySQL Shell's JavaScript mode, if a variable was defined using the `let` keyword but was not properly initialized, it was left in a state where it could neither be accessed nor initialized. MySQL Shell now permits variables defined using the `let` keyword to be redefined with a different value, which can be used to correct the issue. Variables defined using the `const` keyword cannot be redefined in this way. (Bug #32470621)
- When an integer value received by MySQL Shell was converted for representation in JavaScript mode, rounding errors occurred for values greater than 2 ^ 31 – 1. The data type used for the conversion has been changed so that values up to the limit accepted by the JavaScript language can be converted correctly. (Bug #32423886)
- MySQL Shell's Python mode implements its own array wrapper for lists. Previously, the `isinstance` function did not identify these as the `list` object type. The implementation has been changed to inherit from Python's `list` so that the `isinstance` function operates correctly. MySQL Shell stores data in the array wrapper rather than the base class. Due to this fact, and the fact that data is shared between MySQL Shell's Python and JavaScript modes, it is only safe to use lists to store basic data types and MySQL Shell global objects. (Bug #32220514)
- MySQL Shell's Python mode now provides full support for programs that use multithreading. Using the new function `shell.createContext`, output, logging, and user input for a thread can be isolated, and a separate log file can be specified for each thread. (Bug #32101215)
- It was not possible to install MySQL Shell on Mac OS X because the packages were not signed. (Bug #103855, Bug #32943928)

Changes in MySQL Shell 8.0.25 (2021-05-11, General Availability)

Bugs Fixed

- When the consistent option is set to true for MySQL Shell's instance dump utility `util.dumpInstance()`, schema dump utility `util.dumpSchemas()`, and table dump utility `util.dumpTables()`, the utility locks the tables for backup during the dump. If the user ID used to run the utility has the `RELOAD` privilege, the utility uses a `FLUSH TABLES WITH READ LOCK` statement to set a global read lock. If the user ID does not have that privilege but does have the `LOCK TABLES` privilege, the utility issues a series of `LOCK TABLES` statements. Previously, the utility would issue these statements in the same session, causing previously locked tables to be unlocked. The utility now locks the system tables in the main thread's session, and creates additional sessions to lock the other tables that are being dumped. (Bug #32788788)

- MySQL Database Service validates that all user accounts have passwords, but MySQL Shell's instance dump utility `util.dumpInstance` did not previously check this when the `ocimds` option was specified, resulting in errors when loading the dump files. The utility now includes this in the compatibility checks and returns an error if a user account does not have a password set, except where the user account is identified as a role. The `skip_invalid_accounts` modification, which can be applied using the `compatibility` option, now also removes user accounts that do not have passwords set. In the case of a user account that is identified as a role, the utility dumps the account using the `CREATE ROLE` statement. (Bug #32741098)
- In some situations, MySQL Shell's dump loading utility `util.loadDump()` did not delete the progress state file when the `resetProgress` option was specified. (Bug #32734880)
- When MySQL Shell's dump loading utility `util.loadDump()` was used to apply a dump that was still in the process of being created (with the `waitDumpTimeout` option), and pre-authenticated requests were used for the dump, an authorization error could be returned when fetching the `.idx` file for a table. The utility's behavior when handling the manifest has now been refactored to create the correct handle to the `.idx` files and use the correct pre-authenticated request URL. (Bug #32734817)

Changes in MySQL Shell 8.0.24 (2021-04-20, General Availability)

- AdminAPI Added or Changed Functionality
- AdminAPI Bugs Fixed
- Functionality Added or Changed
- Bugs Fixed

AdminAPI Added or Changed Functionality

- The API command-line integration, used for scripting MySQL Shell, has been improved to use the function and option metadata to properly parse arguments and associate them to the corresponding parameter in the API call. Previously, the command-line integration was processing all the arguments as strings, causing errors using operations such as:

```
> mysqlsh user@hostname:3306 -- cluster setOption "autoRejoinTries"  5
```

  The command-line integration now interprets the data being passed in based on what the API functions expect to receive. This enhancement also introduces support for lists in command-line integration calls.

  Additionally, you can now access the MySQL Shell online help from the command-line integration. For example, to get help on the `shell.options` functions, issue:

```
$ mysqlsh -- shell options --help
```

  For more information, see API Command Line Integration.

  In addition, when registering a new MySQL Shell extension function, the new boolean `cli` option is supported by the `shell.addExtensionObjectMember()` operation. When an operation is registered with the `cli` option set to `true`, the object and the functions are made available for the command-line integration. This enables you to extend the scripting possibilities of MySQL Shell. (Bug #31186637, WL #14297)

AdminAPI Bugs Fixed

- Most of the AdminAPI operations contain metadata preconditions to determine if it is valid or not to execute them. When this check was being done, it was adding two entries to the log, one to indicate the

check was about to be done, and another to indicate the metadata state. This meant that operations such as monitoring a cluster, which implies executing regular status requests, resulted in a large number of log entries. Now, these two log messages have been merged into a single entry that gets logged at the info level when the metadata state is not correct, and as debug info when the state is correct. In other words, if the metadata state is correct, the message is only logged when `log_level=debug`. (Bug #32582745)

- The `memberRole` is now included in the default output of *`Cluster`*`.status()`. Previously, this information was only included when the `extended` option had a value of 1 or higher. This makes it easier to know what an instance's role is in the cluster, regardless of whether its operating mode is R/W or R/O. (Bug #32381513)

- `dba.checkInstanceConfiguration()` was performing an incorrect validation regarding the required privileges. This resulted in an endless loop where the operation detected missing privileges, you would give the grants as specified by the interactive help, but the operation would fail, indicating that the grants were missing. Now, the verification checks the correct list. Additionally, the internal list of required grants included `SUPER`, which is deprecated in 8.0. The fix replaces the `SUPER` grant with the fine-grained grants. (Bug #32287986)

- The `memberSslMode` option did not support the `VERIFY_CA` and `VERIFY_IDENTITY` modes for the following operations:

  - `dba.createCluster()`
  - *`Cluster`*`.addInstance()`
  - *`Cluster`*`.rejoinInstance()`

  Now, the `memberSslMode` option supports these modes, and when they are used there is a validation to ensure that the CA certificates are supplied. If you choose to use the `VERIFY_CA` or `VERIFY_IDENTITY` mode, on each cluster instance you must manually supply the CA certificates using the `ssl_ca` and/or `ssl_capath` option. For more information, see Securing InnoDB Cluster.

  Thanks to Daniël van Eeden for the contribution. (Bug #32247631, Bug #32241000, Bug #31227139)

- In version 8.0.23, a check was added to verify if the `server_id` of all cluster instances is registered in the metadata as an instance attribute, and if not then the metadata is updated accordingly. This check is executed on add, rejoin and rescan operations. However, when upgrading a cluster from a version earlier than 8.0.23 to version 8.0.23 and higher, the `server_id` was not registered in the metadata unless you performed a manual rejoin. This was being silently ignored because it was not included in any diagnostic messages. Now, a new verification checks if the `server_id` of cluster instances is missing from the metadata and includes a note message in the `instanceErrors` attribute in the output of *`Cluster`*`.status()` indicating to use *`Cluster`*`.rescan()` to fix it. (Bug #32226871)

- From MySQL Shell version 8.0.17, AdminAPI stores the replication or recovery accounts used for each added instance in the metadata schema, in the instances table. However, the specific transaction could fail or that entry might have been manually removed from the metadata schema, resulting in failures when trying to add other instances to the cluster, and there was no way to resolve this using AdminAPI. In such a situation, if you tried to add an instance the operation failed with an error. Now, AdminAPI tries to detect problems in the metadata related to these accounts for an InnoDB Cluster or InnoDB ReplicaSet. The `status()` operation prints a message if the required account is missing in the metadata schema, and also if it is not the one actually used. In such situations, the MySQL Shell help instructs you to either re-add the instance or run `rescan()` based on the detected problem. The `addInstance()` operation also prints a hint to call `rescan()` if any missing recovery users are found in the metadata. (Bug #32157182)

- *Cluster*.addInstance() was permitting usage of the expelTimeout and consistency options when it should not. These options are cluster level settings that can only be set using dba.createCluster() and *Cluster*.setOption(). (Bug #29779995)

- The dba.checkInstanceConfiguration() operation detects if the instance has any tables that do not have a primary key. Group Replication requires every table that is to be replicated by the group to have a defined primary key. However, this does not mean that having a table without a primary key causes Group Replication to block or fail. Rather, the outcome is that changes to that table are not replicated but the group continues operating. Previously, if the dba.checkInstanceConfiguration() operation detected a table without a primary key, the operation returned with a status of ok and only mentioned tables with a missing primary key and unsupported engines. Now, if the operation detects such a table, it returns with a status of error. As part of this work, the dba.createCluster() operation has been changed to fail if it finds such tables. (Bug #29771457)

- As part of the fix for Bug#28701263, AdminAPI started setting and persisting a default value of READ_ONLY for the group_replication_exit_state_action system variable. The default used by Group Replication was ABORT_SERVER. However, in MySQL Server 8.0.16 the default value of group_replication_exit_state_action became READ_ONLY so AdminAPI should not change and persist it. Now, on instances running 8.0.16 and later, the value of group_replication_exit_state_action is not modified. (Bug #29037274)

- The exception information listed in the online help had become outdated and unwieldy for the interactive MySQL Shell, so it has been removed. (Bug #28542904)

  References: See also: Bug #29853828, Bug #32426083, Bug #32157120, Bug #28825389.

- Using the allowRootFrom option with the dba.deploySandboxInstance() operation was creating a different remote root account depending on whether MySQL Shell was running in interactive mode or not. Now, the default value of allowRootFrom is consistent between both modes, and the account is created as root@% in both interactive and non-interactive mode. (Bug #27369121)

- When you issue dba.createCluster() and dba.createReplica(), tables are created to store the metadata. If the default storage engine was not InnoDB, these operations could fail. Now, metadata creation operations always use the InnoDB storage engine. (Bug #101446, Bug #32110085)

Functionality Added or Changed

- From MySQL 8.0.24, SQL statements that you issue in MySQL Shell's SQL mode can be sent to the operating system's system logging facility (syslog on Unix, or the Windows Event Log). You can select this option by specifying the --syslog command-line option when starting MySQL Shell, or by setting the history.sql.syslog MySQL Shell configuration option. SQL statements that would be excluded from the MySQL Shell code history are also excluded from the system logging facility. (Bug #31995742, Bug #31514599, WL #14358)

- MySQL Shell's instance dump utility util.dumpInstance(), schema dump utility util.dumpSchemas(), and table dump utility util.dumpTables() can now check for tables that do not contain primary keys. The check is carried out when the ocimds option is enabled for checking compatibility with MySQL Database Service, and an error is reported for every table included in the dump that does not have a primary key. The compatibility option, which implements appropriate measures for compatibility, has two new modification choices to notify MySQL Shell's dump loading utility to create primary keys in invisible columns for tables that do not have them, or to ignore the missing primary keys. Primary keys are required for MySQL Database Service High Availability, which uses Group Replication. (WL #14506)

Bugs Fixed

- Previously, MySQL Shell retried requests to Oracle Cloud Infrastructure Object Storage a maximum of 5 times, with a 30 second wait in between retries, and a maximum overall wait of 5 minutes. The retry strategy has now been changed to increase the wait window and reduce the possibility of a dump or load operation failing. MySQL Shell now retries a maximum of 10 times, with a 1 minute wait in between retries, and a maximum overall wait of 10 minutes. (Bug #32592962)
- MySQL Shell's instance dump utility `util.dumpInstance()` stopped with an error if the last schema to be dumped was a schema that contained no tables. The issue has now been fixed. (Bug #32540460)
- MySQL Shell's instance dump utility `util.dumpInstance()` has been optimized so that it can still be used successfully if there are limitations on the server's resources such as disk space or the thread stack. To handle such situations, the queries from the utility can be repeated to retrieve smaller chunks of data if required, and file sorting is avoided. (Bug #32528186)
- MySQL Shell's instance dump utility `util.dumpInstance()` incorrectly removed grants of all privileges to users. The utility now expands `GRANT ALL` statements in the dump to list all privileges granted on all schemas and tables (`*.*`), and to list allowed privileges for system schemas. The dump loading utility `util.loadDump()` now extracts the lists of allowed and revoked global privileges during loading, and strips these from `GRANT` statements relating to system schemas and to all schemas and tables. (Bug #32526567)
- MySQL Shell's dump loading utility `util.loadDump()` now grants privileges after all the data is loaded. Previously, an error could occur if the utility tried to grant a privilege on a routine that did not yet exist. (Bug #32526496)
- MySQL Shell's instance dump utility `util.dumpInstance()`, schema dump utility `util.dumpSchemas()`, and table dump utility `util.dumpTables()` could not complete the dump if the `gtid_executed` system variable or the Information Schema's `COLUMN_STATISTICS` table was unavailable. The utilities now display a warning message and log a detailed error message in this situation. These items are not required for a successful dump. (Bug #32515696)
- MySQL Shell's handling and formatting has been improved for the help text that you provide for dictionary parameters and their options when you register a Python plugin. (Bug #32509309)
- MySQL Shell's instance dump utility `util.dumpInstance()`, schema dump utility `util.dumpSchemas()`, and table dump utility `util.dumpTables()` no longer write a `FLUSH TABLES` statement to the binary log, as this can interfere with replication. (Bug #32490714)
- From MySQL 8.0.23, MySQL Server supports replication from a source server that does not have GTIDs enabled and does not use GTID-based replication, to a replica that has GTIDs enabled, using the `ASSIGN_GTIDS_TO_ANONYMOUS_TRANSACTIONS` option of the `CHANGE REPLICATION SOURCE TO` statement. MySQL Shell's instance dump utility `util.dumpInstance()`, schema dump utility `util.dumpSchemas()`, and table dump utility `util.dumpTables()` now support this functionality by storing the binary log file name and position in the dump metadata, in addition to the `gtid_executed` GTID set. The additional privilege `REPLICATION CLIENT` is required in order for the utilities to be able to collect this information, although if the user ID does not have that privilege, the dump continues but without the binary log information.

  The binary log information can be used after loading the dumped data into the replica server to set up replication with a non-GTID source server. MySQL Shell's dump loading utility `util.loadDump()` prints the binary log and GTID set information from the dump metadata (in YAML format) when you specify the new option `showMetadata: true`. (Bug #32430402)
- MySQL Shell did not correctly handle an empty array that was added to a collection. The result set normally returned from the server is now skipped in this situation. (Bug #32377134)

- MySQL Shell's instance dump utility `util.dumpInstance()`, schema dump utility `util.dumpSchemas()`, and table dump utility `util.dumpTables()` were unable to chunk table data from a MySQL 5.6 server instance, due to differences in the output of the `EXPLAIN SELECT` statement for that version. The utilities now account for the difference, and also cache the server version information for faster access. (Bug #32376447)
- MySQL Shell's parallel table import utility `util.importTable()` set a zero exit code if a non-critical error occurred that did not interrupt the import, such as a directory or file not being found. The utility now sets a non-zero error code instead when the first non-critical error is observed. (Bug #32286186)
- MySQL Shell's upgrade checker utility `util.checkForServerUpgrade()` now checks for spatial data columns that were originally created in MySQL 5.6. The underlying data type for such columns in MySQL 5.6 does not match their underlying data type in MySQL 8.0, so upgrade of the table is prohibited, and it must be recreated. (Bug #32257211, Bug #101944)
- When MySQL Shell casts a string to a boolean value, the operation is now case insensitive. Previously, the results could differ between platforms. (Bug #32217910)
- When MySQL Shell's `\warnings` command was used to show warnings after each statement, warnings were not displayed for a classic MySQL protocol connection. (Bug #32151137)
- MySQL Shell's parallel table import utility `util.importTable()` now checks whether an uploaded object is a directory, and excludes these from wildcard matching that was specified for files. (Bug #31991122)
- MySQL Shell's dump loading utility `util.loadDump()` can split oversized chunks of data into smaller chunks for upload. Previously, if loading was stopped then resumed partway through this stage, the rows in the smaller chunks that were already loaded were not taken into account and skipped, which could lead to deadlocks. The utility's progress file now records the smaller chunks individually so that they can be skipped if the load is stopped and resumed. (Bug #31961688)
- An event that contained a sequence of two semi-colons caused MySQL Shell's instance dump utility `util.dumpInstance()`, schema dump utility `util.dumpSchemas()`, and table dump utility `util.dumpTables()` to enter an infinite loop looking for delimiters. (Bug #31820571)
- The `decodeColumns` option for MySQL Shell's parallel table import utility `util.importTable()` could be specified without an accompanying `columns` option, resulting in the import stopping with an error. (Bug #31407058)
- If a script that was run interactively in MySQL Shell's Python mode did not have a newline character at the end, and the script ended with a multiline command, MySQL Shell waited for input instead of processing the command. The user had to press Enter to finish running the script, and the last line of the script was incorrectly saved in MySQL Shell's code history. MySQL Shell now adds an empty line after processing a script input stream, to ensure that this situation does not occur. (Bug #30765725)
- MySQL Shell used a different character set for collations depending on whether X Protocol or classic MySQL protocol was used to connect to the MySQL server instance, leading to inconsistency and in some situations, errors. For MySQL 5.7 instances, MySQL Shell now uses a `SET NAMES` statement to set all the relevant session system variables to the `utf8mb4` character set. For MySQL 8.0 instances, MySQL Shell now sets the `collation_connection` system variable to the `utf8mb4_0900_ai_ci` character set. (Bug #30516645)

Changes in MySQL Shell 8.0.23 (2021-01-18, General Availability)

- AdminAPI Added or Changed Functionality
- AdminAPI Bugs Fixed

- Functionality Added or Changed
- Bugs Fixed

AdminAPI Added or Changed Functionality

- The output of the `status()` operation has been extended to provide more information relevant to diagnosing errors. The following information is available for InnoDB Clusters and InnoDB ReplicaSets:

  - the `memberState` field shows the actual status of the instance as queried locally, which can be one of `offline`, `error`, `recovering`, or `online`.
  - a `recovery.recoveryChannel` field shows instances performing incremental recovery or in which the recovery channel status is not off
  - a new `instanceErrors` field exists for each instance, displaying any diagnostic information that can be detected for it
  - when the `extended` option is set to greater than 0, the output includes an `applierChannel` field, with replication information if the instance is either online and the applier channel status is not on, or the status is not recovering or online and the applier channel status is not off

  For more information, see Checking a cluster's Status with *`Cluster`*`.status()`. (WL #11560)

- InnoDB Cluster and InnoDB ReplicaSet now support and enable parallel replication appliers, sometimes referred to as a multi-threaded replica. With the advances in MySQL such as binary log transaction dependency tracking and XXHASH64 based GTID set extraction, using multiple replica applier threads improves the throughput of both the replication applier and incremental recovery.

  This has resulted in the following changes:

  - the requirements for instances running 8.0.23 and later now also include:

    - `binlog_transaction_dependency_tracking=WRITESET`
    - `slave_preserve_commit_order=ON`
    - `slave_parallel_type=LOGICAL_CLOCK`
    - `transaction_write_set_extraction=XXHASH64`

    this means that new instances running 8.0.23 have these options configured by `dba.configureInstance()` and `dba.configureReplicaSetInstance()`. Attempting to add an instance running version 8.0.23 or later which does not have these variables configured results in an error. When you upgrade a cluster or replica set that has been running a version of MySQL server and MySQL Shell earlier than 8.0.23, the parallel replication applier is not enabled on the instances. This means you are not taking advantage of this feature, and you should reconfigure your instances to use the parallel replication applier. For more information, see Configuring the Parallel Replication Applier.

  - `dba.checkInstanceConfiguration()` validates if parallel replication appliers are enabled or not.
  - the new `applierWorkerThreads` option configures the number of replication applier threads the instance uses for replication, and defaults to 4 threads. Use this option with the `dba.configureInstance()` and `dba.configureReplicaSetInstance()`. You can change this option while the instance is online, but the change is only made after the instance is restarted.

- the output of the `.status(extended=1)` and `options()` operations now includes information about the configuration of parallel appliers.

(WL #13837)

AdminAPI Bugs Fixed

- The fix for Bug#29305551 extended the `dba.checkInstanceConfiguration()` operation to include a check to verify if asynchronous replication is configured and running on the target instance, and print a warning when that is the case. This check is also used by the *`Cluster`*`.addInstance()` and *`Cluster`*`.rejoinInstance()` operations to terminate them with an error when such a scenario is detected, and is also used by the `dba.rebootClusterFromCompleteOutage()` operation whenever there are instances to be rejoined to the cluster. However, the `dba.createCluster()` operation was erroneously skipping the check, and the `dba.rebootClusterFromCompleteOutage()` operation was skipping the check on the instance being used to bootstrap the cluster. The fix ensures that the check is also performed whenever creating or rebooting a cluster from complete outage. Additionally, it adds support to override the check for the `dba.createCluster()` operation by making use of the `force` option, and it improves the error messages. (Bug #32197222)

- The fix for Bug#29305551 extended `dba.checkInstanceConfiguration()` to verify if asynchronous replication is configured and running on the target instance and print a warning if that was the case. However, the check missed verifying if the replication channel was configured but not running. This fix ensures the verification also considers replication channels which are configured but are not actively running. Additionally, an erroneous message which suggested the possibility of using `STOP REPLICA` to override this check has been removed and replaced with an informative message which explains that unmanaged replication channels are not supported and the possible dangers of their usage. (Bug #32197197)

- Based on the terminology changes in WL#14189, AdminAPI has been aligned with the new terms. Error and log messages now use the terms source (previously master) and replica (previously slave). (Bug #32152133)

- During a *`Cluster`*`.rebootClusterFromCompleteOutage()` operation, the GTID superset is used to detect which instance should be used to reboot the cluster. If an instance had a diverging GTID set and you wanted to explicitly remove it from the cluster, the operation blocked because it could not determine which instance had the GTID superset. Previously, in such a situation there was no way to exclude the instance from the instances used to detect the GTID superset. Now, if you answer no during the interactive wizard, or configure the `removeInstances` option, the instance is not checked as part of finding the GTID superset. (Bug #32112864)

- When an instance had left a ReplicaSet, and then its configuration was changed in a way that made it invalid for InnoDB ReplicaSet usage, the *`ReplicaSet`*`.rejoinInstance()` operation did not detect that the configuration was invalid. Now, instances are checked to ensure they are valid before rejoining them to a ReplicaSet. (Bug #31975416)

- When upgrading the metadata using `dba.upgradeMetadata()`, if there are MySQL Router instances that need to be upgraded, the operation waits until all instances are upgraded before continuing. The operation offers you an option to re-check for outdated MySQL Router instances and continue with the metadata upgrade. A MySQL Router upgrade is only complete after a restart of the application, however the message printed did not mention that. This message now includes the information that MySQL Router instances must be restarted after the binaries are upgraded. (Bug #31882876)

- When you were connected to a secondary instance, attempting to issue operations such as *`Cluster`*`.rejoinInstance()`, *`Cluster`*`.addInstance()`, *`Cluster`*`.dissolve()` and so on would fail. Now, AdminAPI always connects to the current primary.

As part of this work the following changes were made:

- Now, in the event that `dba.createCluster()` or *`Cluster`*`.addInstance()` fail with a Group Replication error, AdminAPI returns the `performance_schema.error_log` entries.
- The *`Cluster`*`.rejoinInstance()` operation has been changed to succeed if the instance is already in the cluster, instead of throwing an exception.
- The `dba.rebootCluster()` operation has been changed to not clear `super_read_only` on the instance.

(Bug #31757737)

- As part of the default settings for InnoDB Cluster, to ensure that instances automatically rejoin the cluster, the `group_replication_start_on_boot` option is automatically set to true. However, this meant that in environments with an external tool managing the cluster life cycle, for example an orchestrator such as Kubernetes, the automatic enabling of rejoin could cause conflicts with the tool. In addition, if the automatic rejoining of an instance was enabled at an unsuitable time (for example when rebooting, or while repairing a split-brain, and so on), a deadlock or long freezes could occur until a timeout happened. In some situations, instances could even potentially join the wrong cluster during a reconfiguration.

  To avoid such situations, the `manualStartOnBoot` boolean option has been added, which defaults to false. To disable the automatic rejoining of an instance, for example while repairing a split-brain, set the `manualStartOnBoot` option to true. This prevents the instance rejoining the cluster automatically while you make changes. You then need to rejoin the instance to the cluster manually, before setting the `manualStartOnBoot` option back to false to ensure instance it rejoins the cluster automatically again. Similarly, if you are using an external orchestrator to manage the life cycle of instances, set the `manualStartOnBoot` option to true across the whole cluster, to disable the automatic rejoining of instances to the cluster. Your orchestrator should then be configured to rejoin the instances manually. (Bug #31643595)

- Calling `dba.checkInstanceConfiguration()` with `verifyMyCnf` set to a file which did not exist, the operation completed successfully saying the configuration file had been checked. The fix checks if the file specified by `verifyMyCnf` exists, prints an error if not, and ensures the console does not show unnecessary error messages. (Bug #31468546)

- On an instance with the `sql_mode` variable set to `ANSI_QUOTES`, attempting to upgrade the metadata schema with `dba.upgradeMetadata()` failed with the error: `Unknown column 'MySQL Router' in 'field list'`. This was related to a query which uses single quotes to quote strings. As part of this fix, the upgrade metadata operation now prepares the session to be used by AdminAPI, and amongst other sanity checks it ensures that the `sql_mode` for that session uses the default value to avoid incompatible user configured settings. Additionally, the same was done for the `dba.getCluster()` and `dba.dropMetadataSchema()` operations. (Bug #31428813)

- If the MySQL Shell global session was connected to a sandbox instance, and that instance was stopped, MySQL Shell tried to incorrectly reconnect to the instance. Now, if the active session is connected to a sandbox instance which is being stopped, MySQL Shell closes the session. (Bug #31113914)

- The output of *`Cluster`*`.status()` now includes additional information about instances that are registered in the metadata but not currently online. MySQL Shell now connects to offline instances found in the metadata and attempts to diagnose them, providing additional information such as their connectivity and status. (Bug #30501615)

- Instances that are part of the underlying group but are not identified in the metadata, for example because they were configured manually and bypassing MySQL Shell, or because they were previously

removed from the InnoDB Cluster but were not properly decommissioned, are now shown in the output of `Cluster.status()`, along with diagnostic warnings about the metadata discrepancy. This ensures you can detect situations where an instance is participating in the group but is not being managed by MySQL Shell. (Bug #27882663)

- An instance that belongs to an InnoDB Cluster is identified by its server UUID. If the UUID changed after the instance had left the cluster, for example because you used MySQL Enterprise Backup to restore from a backup, then the instance could not be rejoined to the cluster. Now, if the cluster encounters this situation, it checks the metadata to see if the instance can be identified using its host and port. If found, the metadata is updated based on the options used for the rejoin operation. This check is executed during the `Cluster.rejoinInstance()` and `Cluster.rescan()` operations.

  Additionally, a check is executed to verify the `serverId` of all the instances is registered in the metadata as an instance attribute. If it is not, the metadata is updated accordingly. This check is executed on add, rejoin and rescan operations. (Bug #26649039)

Functionality Added or Changed

- MySQL Shell's parallel table import utility can now import a specified list of input data files, and it supports wildcard pattern matching to include all relevant files from a location. Multiple files uploaded by a single run of the utility are placed into a single relational table, so for example, data that has been exported from multiple hosts and stored in multiple files could be merged into a single table to be used for analytics. The files can be compressed in the gzip or zstd format, and in that case the utility reads them from storage in the compressed format, saving bandwidth for that part of the transfer. The utility then uses its parallel connections to decompress and upload several files simultaneously to the target server. (WL #13362)

Bugs Fixed

- When MySQL Shell's instance dump utility `util.dumpInstance()` was run with the `ocimds` option set to `true` to check compatibility with MySQL Database Service, and the `users` option set to `true` to include users and their roles and grants in the dump, the utility reported some compatibility errors for privileges that actually were permitted. MySQL Shell's allowed list of privileges for MySQL Database Service has now been updated. (Bug #32213605)

- The behavior of MySQL Shell's table dump utility `util.dumpTables()` and dump loading utility `util.loadDump()` regarding the schemas for single table dumps and loads has been changed. Previously, the dump files produced for a single table did not contain the SQL statements to recreate the schema, so the schema had to exist in the target MySQL instance before the dump loading utility could load the table. Now, the dumps produced by the table dump utility contain the schema statements, and when they are loaded with the dump loading utility, by default, the schema is created in the target MySQL instance if it does not already exist. The `schema` option can be used to load the table dump into another schema that exists in the target MySQL instance. Table dumps created using the earlier version of the utility still require the `schema` option and an existing schema. (Bug #32165101)

- MySQL Shell's table dump utility `util.dumpTables()` now supports the `ocimds`, `compatibility`, `ociParManifest`, and `ociParExpireTime` options, so you can check compatibility with MySQL Database Service, and generate pre-authenticated request URLs for the dump files. Also, the `ignoreVersion` option has been extended to allow the import of a dump that was created without the `ocimds` option into a MySQL DB System. (Bug #32140970)

- If a dump included users that were created with external authentication plugins, MySQL Shell's dump loading utility `util.loadDump()` was unable to load the dump if those plugins were not available on the target server instance. The `ocimds` option for MySQL Shell's instance dump utility `util.dumpInstance()` and schema dump utility `util.dumpSchemas` which checks compatibility with MySQL Database Service, now checks for accounts using authentication plugins that are not

supported in MySQL Database Service. The compatibility option has an additional modification option `skip_invalid_accounts`, which removes such user accounts. (Bug #32115948)

- Previously, MySQL Shell's dump loading utility `util.loadDump()` stopped with an error if the `loadUsers` option was set to true but the supplied dump files did not contain user accounts. The utility now displays a warning and continues in this situation. (Bug #32115861)
- MySQL Shell's instance dump utility `util.dumpInstance()`, schema dump utility `util.dumpSchemas()`, and table dump utility `util.dumpTables()` falls back to using the `LOCK TABLES` privilege to lock dumped tables if the consistent option is set to true, which is the default, and the `RELOAD` privilege is not available. However, the locking operation could cause an implicit commit on active transactions, meaning that the data was not dumped consistently. The locking has now been corrected to ensure consistency in this situation. (Bug #32107327, Bug #101410)
- When MySQL Shell's dump loading utility `util.loadDump()` used indexes to identify row boundaries, an error occurred if an index pointed beyond the data in the read buffer. The utility now checks for this situation and ignores the index if so. (Bug #32072961)
- When MySQL Shell was attempting to reconnect to a server, **Ctrl + C** did not interrupt the operation. The interrupt now functions and sets the retry attempts counter to zero so that the sequence exits correctly. (Bug #32041342)
- MySQL Shell can now be built using Python 3.9. (Bug #32020230)
- The `updateGtidSet` option for MySQL Shell's dump loading utility `util.loadDump()` could not be used with MySQL DB System due to a permissions restriction. The utility now uses a stored procedure that is permitted, so the option can be used. (Bug #32009225)
- When MySQL Shell's instance dump utility `util.dumpInstance()`, schema dump utility `util.dumpSchemas()`, or table dump utility `util.dumpTables()` was exporting to an Oracle Cloud Infrastructure Object Storage bucket, if there was a loss of connectivity or routing to the Object Storage server, MySQL Shell stopped unexpectedly. The error is now handled correctly. (Bug #32005418)
- MySQL Shell's dump loading utility `util.loadDump()` returned an exception if a header value in a response was empty. (Bug #31979374)
- MySQL Shell did not initialize Python 3.8's new `cf_feature_version` compiler flag field, which could cause an exception when format strings were used. (Bug #31926697)
- Where MySQL Shell is using a system installation of Python rather than the bundled version, the minimum version that MySQL Shell supports is now Python 3.6. Python 3.4.3 was the previous minimum for a system installation. The bundled version is Python 3.7.7. (Bug #31900744)
- MySQL Shell's instance dump utility `util.dumpInstance()`, schema dump utility `util.dumpSchemas()`, and table dump utility `util.dumpTables()` use table statistics to identify a suitable default row size. If the statistics for a table are outdated or not present, this can cause issues for the chunking process. In this situation, MySQL Shell now issues a message to suggest using an `ANALYZE TABLE` statement to produce up to date statistics. (Bug #31766490)
- The `skipBinlog` option for MySQL Shell's dump loading utility `util.loadDump()` skips binary logging on the target MySQL instance for the import. The option is not suitable for MySQL DB System as the binary logging status cannot be changed, and the import now fails with an error message if the option is used in that situation. For other MySQL instances, the utility now checks whether the user has the required privileges to set the `sql_log_bin` system variable, and fails with an error message if they do not. (Bug #31748786)
- MySQL Shell's instance dump utility `util.dumpInstance()`, schema dump utility `util.dumpSchemas()`, and table dump utility `util.dumpTables()` ordered the data fetched for

export using the first column of a unique index for the table. The same method was used to query data for chunking purposes. The utilities now use all columns of the unique index for ordering. In addition, performance is improved by the addition of a cache to store frequently-used instance metadata. The cache is populated for all the schema objects at once, rather than by individual queries as needed. (Bug #31706755)

- MySQL Shell's disconnect function was added to the `shell` global object. (Bug #31704380)

Changes in MySQL Shell 8.0.22 (2020-10-19, General Availability)

- AdminAPI Bugs Fixed
- Functionality Added or Changed
- Bugs Fixed

AdminAPI Bugs Fixed

- Due to the changes introduced in bug#31467823, AdminAPI user messages have been updated to use the same terminology. (Bug #31798724)

  References: See also: Bug #31462783, Bug #31467823.

- In 8.0.22, Group Replication deprecates the `group_replication_ip_whitelist` system variable in favor of `group_replication_ip_allowlist`. Therefore AdminAPI now includes a new option named `ipAllowlist` and deprecates the `ipWhitelist` option used with `dba.createCluster()`, *`Cluster`*`.addInstance()` and *`Cluster`*`.rejoinInstance()`.

  Regardless of the target instance version, if `ipWhitelist` is used MySQL Shell prints a deprecation warning. When the target instance is 8.0.22 or later Group Replication reports a deprecation warning, but AdminAPI avoids this by ensuring the appropriate variable is set according to the version of MySQL running on the instance. If the target instance is 8.0.22 or later and the old `ipWhitelist` option is used, AdminAPI updates the new `group_replication_ip_allowlist` system variable. On instances running MySQL 8.0.21 or earlier, AdminAPI updates the older `group_replication_ip_whitelist` system variable.

  This work also fixes a bug in *`Cluster`*`.options()`, which shows the list of all Group Replication options that are configurable by MySQL Shell. When an option is not set or does not exist (for example in a certain version) it is displayed as having a null value. However, when you passed in the `all` option to *`Cluster`*`.options()`, such variables were being excluded from the list. (Bug #31798495)

- The `dba.configureLocalInstance()` and `dba.configureInstance()` operations could not be used against instances that were part of unmanaged replication groups. This made it impossible to create a cluster administrator account, which is required when adopting a Group Replication group into an InnoDB Cluster. Creating cluster administrator accounts is very important for the remote management of InnoDB Clusters, and to avoid having to manually create the user and required privileges on each instance. The fix ensures that `dba.configureLocalInstance()` and `dba.configureInstance()` can be executed against instances belonging to unmanaged replication groups. Because such instances are ready for InnoDB Cluster usage, a message confirms this. (Bug #31691232)

- The `dba.rebootClusterFromCompleteOutage()` operation was only checking `GTID_EXECUTED` when validating for the instance that has the most transactions. Transactions that were received (and certified) but not executed yet were not included in that check. Also, transactions received through replication channels other than the Group Replication applier were also not being considered in that check, because the instance was probably the primary while the cluster was running. Ignoring these

transactions led to data loss. Now, any known and managed replication channels are considered as part of the check. (Bug #31673163)

- The `dba.removeInstance()` operation failed if the instance being removed was not only unreachable but also unresolvable, which could be the case when the instance was running in a container that takes down its own DNS record when removed.

  Instances could not be removed even if force was enabled, because of a validation that ensured that the given instance is a valid address by checking if it is resolvable. If that validation failed, nothing else was attempted. The fix completely removes address resolution from the whole AdminAPI. That check was redundant, because invalid addresses would eventually lead to an error anyway when a connection was opened. IPv6 address syntax validations were left to remind you that `::1` should be specified as `[::1]`. (Bug #31632606)

- Calling *`Cluster`*`.rejoinInstance("`*`host`*`:`*`port`*`")` with no user name specified caused AdminAPI to try and connect using the operating system user name instead of the credentials used to connect the cluster object. Now, if credentials are not provided, they are taken from the target server's connection options. (Bug #31632554)

- When adding an instance to a cluster using MySQL Clone and monitoring the transfer of data, MySQL Shell could stop unexpectedly. This was due to the assumption that the Performance Schema would provide information on all four stages of cloning, which might not exist if the data set was very small. The fix ensures that updates are performed according to the information which is available at each time. (Bug #31545728)

- When an unreachable primary instance was forcibly removed from a cluster, the `group_replication_group_seeds` system variable was not updated because the group status was queried from the primary, which was missing. The cluster was left in an inconsistent state, and if any of the instances were restarted, they would not be able to automatically rejoin, because `group_replication_group_seeds` contained an invalid address, which caused Group Replication to abort trying to join the group. (Bug #31531704)

- If the `validate_password` plugin was enabled, the `setupAdminAccount()` and `setupRouterAccount()` operations would fail with an error indicating the password did not meet the policy requirements. This happened regardless of whether the password met the policy requirements or not. (Bug #31491092)

- In the event of `dba.createCluster()` failing, the metadata record was left behind and Group Replication was also started. The cluster was left in an inconsistent state that could not be recovered from when calling `dba.createCluster()` again. Now, metadata changes during a `dba.createCluster()` operation are enclosed in a transaction, so that both the cluster and instance records are only committed if the operation succeeds. Group Replication is also stopped if the create operation does not complete successfully.

  Similarly, *`Cluster`*`.addInstance()` has been changed to ensure that its metadata record is only inserted when everything else has completed successfully. On retry, it skips any steps that would cause conflicts because Group Replication is already running. This introduces a behavior change, where adding an instance that is part of the group but not in the metadata just adds it to the metadata, instead of aborting and requiring you to execute *`Cluster`*`.rescan()`. (Bug #31455419)

- If a connection timed out during a *`Cluster`*`.status()` operation, MySQL Shell could appear to hang for a long time. To improve the responsiveness, the default timeout of AdminAPI operations has been reduced from 10 seconds to 2 seconds. This ensures operations like *`Cluster`*`.status()` do not appear to freeze for a long time when there are unreachable instances. (Bug #30884174)

- Instances operating in an InnoDB Cluster or InnoDB ReplicaSet are all required to have the same password for the administrative account. In a situation where the password on an instance joining a

InnoDB Cluster or InnoDB ReplicaSet did not match the other instances, the error message did not explain this and the instance failed to join. Now, in such a situation the error is detected and the resulting message mentions that the password is the cause of the failure. It is recommended that you set up administrator accounts using the `setupAdminAccount()` operation, see Creating User Accounts for AdminAPI. (Bug #30728744)

Functionality Added or Changed

- Two new utilities are available in MySQL Shell to export single tables from a MySQL server instance.
  - MySQL Shell's new table dump utility `util.dumpTables()` supports the export of a selection of tables or views from a schema, from an on-premise MySQL instance into an Oracle Cloud Infrastructure Object Storage bucket or a set of local files. It works in the same way as the instance dump utility `util.dumpInstance()` and schema dump utility `util.dumpSchemas()` introduced in 8.0.21, but with a different selection of suitable options. The exported items can then be imported into a MySQL Database Service DB System (a MySQL DB System, for short) or a MySQL Server instance using MySQL Shell's dump loading utility `util.loadDump()`.
  - MySQL Shell's new table export utility `util.exportTable()` exports a MySQL relational table into a data file in a variety of formats, either on the local server or in an Oracle Cloud Infrastructure Object Storage bucket. The data can then be uploaded into a table on a target MySQL server using MySQL Shell's parallel table import utility `util.importTable()`, which uses parallel connections to provide rapid data import for large data files. The data file can also be used to import data to a different application, or as a lightweight logical backup for a single data table.

  (WL #13804)
- From MySQL Shell 8.0.22, when you export schemas to an Oracle Cloud Infrastructure Object Storage bucket using MySQL Shell's instance dump utility and schema dump utility, `util.dumpInstance()` and `util.dumpSchemas()`, during the dump you can generate a pre-authenticated request URL for every item. The utilities do this by default when the `ocimds` option is set to true, and you can control the feature using the `ociParManifest` and `ociParExpireTime` options. The user account that runs MySQL Shell's dump loading utility `util.loadDump()` then uses the pre-authenticated request URLs to load the dump files without additional access permissions. (WL #14154)

Bugs Fixed

- MySQL Shell's dump loading utility `util.loadDump()` would stop with an error if a data file's size was larger than an applicable server limit relating to the maximum transaction size, such as the `max_binlog_cache_size` limit. Now, the utility stops the data load in mid-file if the number of bytes uploaded is about to exceed 1.5 times the `bytesPerChunk` setting of the utility that created the data files, which is stored in the dump metadata. The data load is then restarted to upload the remainder of the file. Due to this new safeguard, the default `bytesPerChunk` setting of MySQL Shell's instance dump utility `util.dumpInstance()`, schema dump utility `util.dumpSchemas()`, and table dump utility `util.dumpTables()` has been increased from 32 MB to 64 MB. (Bug #31945539)
- MySQL Shell's instance dump utility `util.dumpInstance()` and schema dump utility `util.dumpSchemas()` did not take the row size into account when deciding whether to chunk table data, only the number of rows. A table with a smaller number of rows containing large amounts of data might therefore bypass chunking even if the row size exceeded the specified chunk size for the dump. The utilities now carry out chunking regardless of the number of rows, unless the data is estimated to fit in a single chunk of the specified size. (Bug #31938831)
- MySQL Shell's instance dump utility `util.dumpInstance()`, schema dump utility `util.dumpSchemas()`, and table dump utility `util.dumpTables()` use a chunking algorithm to split the data for a table into multiple files. It was possible for the algorithm to create a large number of very

small files, causing issues with the dump. A minimum valid value is now specified, and the algorithm has been enhanced to ensure that the range being checked is always large enough to include sufficient rows. (Bug #31909408)

- When MySQL Shell's instance dump utility `util.dumpInstance()` or schema dump utility `util.dumpSchemas()` was splitting a table into chunks, an integer overflow, loop, and consequent out of memory error could occur if the maximum value in the index column was close to the maximum value of an integer. Extra checks have now been added to avoid this situation. (Bug #31896448)
- Corrupted SQL files could occur in MySQL Shell's dumps if the dumped string was exactly 2048 bytes, due to an internal module error which is now accounted for. (Bug #31843832)
- The administrative user account on an Oracle Cloud Infrastructure Compute instance, which was used when importing data with MySQL Shell's dump loading utility `util.loadDump()`, was unable to revoke privileges on MySQL system schemas (`mysql` and `sys`) that it did not have itself. Affected `REVOKE` statements are now stripped from an instance or schema dump created by MySQL Shell's instance dump utility `util.dumpInstance()` and schema dump utility `util.dumpSchemas()` when the `strip_restricted_grants` compatibility option is used. (Bug #31842532)
- MySQL Shell's instance dump utility and schema dump utility, dumpInstance() and dumpSchemas(), previously fetched the value of the `gtid_executed` system variable before the read lock was established, which could potentially lead to an inconsistency with the dumped data. The GTID set is now retrieved while the read lock is active. (Bug #31706940)
- From MySQL Shell 8.0.22, you can use the `–pym` command-line option to execute a specified Python module as a script in MySQL Shell's Python mode. `--pym` works in the same way as Python's `-m` command line option. (Bug #31694202)
- In MySQL Shell in Python mode, functions registered in JavaScript could not be called from global extension objects if they had optional arguments that were not provided. (Bug #31693096)
- MySQL Shell treated the character sequence */ as the end of a comment even when it was part of a quoted string. (Bug #31689135)
- MySQL Shell's instance dump utility and schema dump utility, `dumpInstance()` and `dumpSchemas()`, automatically select an index column to order and chunk the data. When an index with a functional key part was present for a table, the query for index columns returned `NULL` for the column name, causing an exception in the utility. These column names are now filtered out in the query. (Bug #31687059)
- MySQL Shell's parallel table import utility `util.importTable()` has a new option `decodeColumns`, and an enhancement to the columns option, to enable you to capture columns from the import file for input preprocessing (or to discard them) in the same way as with a `LOAD DATA` statement. The `decodeColumns` option specifies preprocessing transformations for the captured data in the same way as the `SET` clause of a `LOAD DATA` statement, and assigns them to columns in the target table. (Bug #31683641)
- MySQL Shell's dump loading utility, `loadDump()`, now verifies that it can open and write to the progress state file before it starts to retrieve the dump files, so that the utility does not spend time fetching the dump files if the import is subsequently going to fail for that reason. (Bug #31667539)
- Before MySQL 8.0, user accounts that have all privileges used the statement `GRANT ALL PRIVILEGES`, rather than the full list of privileges. When MySQL Shell's instance dump utility dumpInstance() was used to dump an instance, `ALL PRIVILEGES` was stripped from the statement, leaving the accounts with no privileges. The utility now replaces this grant with the administrator role. (Bug #31661180)
- MySQL Shell's instance dump utility and schema dump utility, `dumpInstance()` and `dumpSchemas()`, and dump loading utility, `loadDump()`, previously timed out after the main thread had been idle for 8

hours. The timeout has now been extended indefinitely (to 1 year) so that long data load times do not cause the dump or import to fail. (Bug #31652265)

- MySQL Shell's dump loading utility, `loadDump()`, now includes a comment in its `LOAD DATA` statements to identify the data chunk that is currently being loaded. If you need to cancel the import, you can use this information to decide whether to let the import of this chunk complete or stop it immediately. (Bug #31646650)
- Previously, MySQL Shell's dump loading utility `loadDump()` closed DDL files after all tables from the schema had finished loading. For a dump with a very large number of DDL files, this could lead to unexplained runtime errors being returned. The utility now closes DDL files immediately after reading them. (Bug #31645896)
- MySQL Shell's dump loading utility, `loadDump()`, previously used only its main thread to execute the DDL scripts for tables. For a dump containing a large number of tables, fetching the DDL scripts could have a significant impact on the time taken. The utility now fetches and executes DDL scripts for tables using all its threads, with the exception of DDL scripts for views, which are fetched in parallel but executed only in the main thread to avoid race conditions. (Bug #31645806)
- MySQL Shell's dump loading utility, `loadDump()`, now loads views in sequence and only after all tables and placeholders from all schemas have been loaded, ensuring that views do not reference items that do not yet exist. (Bug #31645792)
- MySQL Shell now ignores any other available Python installations when a bundled compatible version of the Python interpreter has been installed. (Bug #31642521)
- When MySQL Shell's dump loading utility, `loadDump()`, is importing users and their roles and grants, an error is now returned if the user already exists in the target instance, and the user's grants from the dump files are not applied. Previously, the grants were applied to the existing user. (Bug #31627432)
- MySQL Shell's dump loading utility, `loadDump()`, now has an option `updateGtidSet` to apply the `gtid_executed` GTID set from the source MySQL instance to the `gtid_purged` GTID set on the target MySQL instance. You can append or replace the GTID set depending on the release of the target MySQL instance. (Bug #31627419)
- MySQL Shell's instance dump utility, `dumpInstance()`, and dump loading utility, `loadDump()`, now have options to include (`includeUsers`) or exclude (`excludeUsers`) named user accounts from the dump files or from the import. You can use these options to exclude user accounts that are not accepted for import to a MySQL DB System, or that already exist or are not wanted on the target MySQL instance. (Bug #31627292)
- With the `deferTableIndexes` option set to `all`, MySQL Shell's dump loading utility `loadDump()`, defers creation of all secondary indexes until after the table is loaded. Previously, a table with a unique key column containing an auto-increment value failed to load in this situation. The utility now also creates indexes defined on columns with auto-increment values when the `deferTableIndexes` option is set to `all`. (Bug #31602690)
- The upload method used by MySQL Shell's instance dump utility `util.dumpInstance()` and schema dump utility `util.dumpSchemas()` to transfer files to an Oracle Cloud Infrastructure Object Storage bucket has a file size limit of 1.2 TiB. In MySQL Shell 8.0.21, the multipart size setting means that the numeric limit on multiple file parts applies first, creating a limit of approximately 640 GB. From MySQL Shell 8.0.22, the multipart size setting has been changed to allow the full file size limit. (Bug #31589858)
- MySQL Shell's upgrade checker utility checkForServerUpgrade() now checks for the obsolete `NO_AUTO_CREATE_USER` SQL mode. (Bug #31501981, Bug #99903)
- The parallelization of table loading by MySQL Shell's dump loading utility, `loadDump()`, has been improved. (Bug #31441903)

- If the settings for the server's global character set variables differed from the settings for the current session, MySQL Shell's parallel table import utility, `importTable()`, could not import data into a table created in the session whose name contained non-ASCII characters. Now, when the utility's characterSet option is specified, MySQL Shell executes a `SET NAMES` statement with the given value. (Bug #31412330)
- MySQL Shell's parallel table import utility, `importTable()`, could not import data if the global SQL mode `NO_BACKSLASH_ESCAPES` was set. The utility now clears the global SQL mode in sessions created to run the import. (Bug #31407133)
- When a function that was defined as a member of a MySQL Shell extension object had a number of optional parameters but no required parameters, in some situations calling the function with zero parameters or one parameter returned an error. (Bug #30744994)
- MySQL Shell's `db` global object did not return the current schema when its properties were queried. (Bug #30296825, Bug #96839)
- MySQL Shell has a new command `\disconnect`. The command disconnects MySQL Shell's global session (the session represented by the `session` global object) from the currently connected MySQL server instance, so that you can close the connection but still continue to use MySQL Shell. (Bug #28240416)

Changes in MySQL Shell 8.0.21 (2020-07-13, General Availability)

- AdminAPI Added or Changed Functionality
- AdminAPI Bugs Fixed
- Functionality Added or Changed
- Bugs Fixed

AdminAPI Added or Changed Functionality

- A new user configurable tag framework has been added to the metadata, to allow specific instances of a cluster or ReplicaSet to be marked with additional information. Tags can be any ASCII character and provide a namespace. You set tags for an instance using the `setInstanceOption()` operation. In addition, AdminAPI and MySQL Router 8.0.21 support specific tags, which enable you to mark instances as hidden and remove them from routing. MySQL Router then excludes such tagged instances from the routing destination candidates list. This enables you to safely take a server instance offline, so that applications and MySQL Router ignore it, for example while you perform maintenance tasks, such as server upgrade or configuration changes. To bring the instance back online, use the `setInstanceOption()` operation to remove the tags and MySQL Router adds the instance back to the routing destination candidates list, and it becomes online for applications. For more information, see Tagging Metadata. (WL #13788)

AdminAPI Bugs Fixed

- **Important Change:** Previously, Group Replication did not support binary log checksums, and therefore one of the requirements for instances in InnoDB Cluster was that binary log checksums were disabled by having the `binlog_checksum` system variable set to `NONE`. AdminAPI verified the value of `binlog_checksum` during the `dba.checkInstanceConfiguration()` operation and disallowed creating a cluster or adding an instance to a cluster that did not have binary log checksums disabled. In version 8.0.21, Group Replication has lifted this restriction, therefore InnoDB Cluster now permits instances to use binary log checksums, with `binlog_checksum` set to `CRC32`. The setting for `binlog_checksum` does not have to be the same for all instances. In addition, sandboxes deployed

with version 8.0.21 and later do not set the `binlog_checksum` variable, which defaults to `CRC32`. (Bug #31329024)

- Adopting a Group Replication setup as a cluster can be performed when connected to any member of the group, regardless of whether it is a primary or a secondary. However, when a secondary member was used, `super_read_only` was being incorrectly disabled on that instance. Now, all operations performed during an adoption are done using the primary member of the group. This ensures that no GTID inconsistencies occur and that `super_read_only` is not incorrectly disabled on secondary members. (Bug #31238233)

- Using the `clusterAdmin` option to create a user which had a netmask as part of the host resulted in an error when this user was passed to the `dba.createCluster()` operation. Now, accounts that specify a netmask are treated as accounts with wildcards, meaning that further checks to verify if the account accepts remote connections from all instances are skipped. (Bug #31018091)

- The check for instance read-only compatibility was using a wrong MySQL version as the base version. The cross-version policies were added to Group Replication in version 8.0.17, but the check was considering instances running 8.0.16. This resulted in a misleading warning message indicating that the added instance was read-only compatible with the cluster, when this was not true (only for instances 8.0.16). The fix ensures that the check to verify if an instance is read-compatible or not with a cluster is only performed if the target instance is running version 8.0.17 or later. (Bug #30896344)

- The maximum number of instances in an InnoDB Cluster is 9, but AdminAPI was not preventing you from trying to add more instances to a cluster and the resulting error message was not clear. Now, if a cluster has 9 instances, *`Cluster`*`.addInstance` prevents you adding more instances. (Bug #30885157)

- Adding an instance with a compatible GTID set to a InnoDB Cluster or InnoDB ReplicaSet on which provisioning is required should not require any interaction, because this is considered a safe operation. Previously, in such a scenario, when MySQL Clone was supported MySQL Shell still prompted to choose between cloning or aborting the operation. Now, the operation proceeds with cloning, because this is the only way to provision the instance.

  

  Note

  instances with an empty GTID set are not considered to have a compatible GTID set when compared with the InnoDB Cluster or InnoDB ReplicaSet. Such scenarios are considered to be unknown, therefore MySQL Shell prompts to confirm which action should be taken.

  (Bug #30884590)

- The Group Replication system variables (prefixed with `group_replication`) do not exist if the plugin has not been loaded. Even if the system variables are persisted to the instance's option file, they are not loaded unless the Group Replication plugin is also loaded when the server starts. If the Group Replication plugin is installed after the server starts, the option file is not reloaded, so all system variables have default values. Instances running MySQL 8.0 do not have a problem because `SET PERSIST` is used. However, on instances running version MySQL 5.7, the `dba.rebootCluster()` operation could not restore some system variables if the Group Replication plugin was uninstalled. Now, the `dba.configureInstance()` operation persists the Group Replication system variables to configuration files with the `loose_` prefix. As a result, once the Group Replication plugin is installed, on instances running 5.7 the persisted values are used instead of the default values. (Bug #30768504)

- The `updateTopologyMode` option has been deprecated and the behavior of *`Cluster`*`.rescan()` has been changed to always update the topology mode in the Metadata when a change is detected. MySQL Shell now displays a message whenever such a change is detected. (Bug #29330769)

- The `cluster.addInstance()` and `cluster.rejoinInstance()` operations were not checking for the full range of settings which are required for an instance to be valid for adding to the cluster. This resulted in attempts to use instances which run on different operating systems to fail. For example, a cluster running on two instances that were hosted on a Linux based operating system would block the addition of an instance running Microsoft Windows. Now, `cluster.addInstance()` and `cluster.rejoinInstance()` operations validate the instance and prevent adding or rejoining an instance to the cluster if the value of the `lower_case_table_names`, `group_replication_gtid_assignment_block_size` or `default_table_encryption` of the instance are different from the ones on the cluster. (Bug #29255212)

Functionality Added or Changed

- MySQL Shell now has an instance dump utility, dumpInstance(), and schema dump utility, dumpSchemas(). The new utilities support the export of all schemas or a selected schema from an on-premise MySQL server instance into an Oracle Cloud Infrastructure Object Storage bucket or a set of local files. The schemas can then be imported into a MySQL Database Service DB System using MySQL Shell's new dump loading utility. The new utilities provide Oracle Cloud Infrastructure Object Storage streaming, MySQL Database Service compatibility checks and modifications, parallel dumping with multiple threads, and file compression. See Instance Dump Utility, Schema Dump Utility, and Table Dump Utility. (WL #13807)

- MySQL Shell's new dump loading utility, loadDump(), supports the import of schemas dumped using MySQL Shell's new instance dump utility and schema dump utility into a MySQL Database Service DB System. The dump loading utility provides data streaming from remote storage, parallel loading of tables or table chunks, progress state tracking, resume and reset capability, and the option of concurrent loading while the dump is taking place. See Dump Loading Utility. (WL #13808)

- The X DevAPI implementation now supports JSON schema validation, which enables you to ensure that your documents have a certain structure before they can be inserted or updated in a collection. To enable or modify JSON schema validation you pass in a JSON object like:

```
{
    validation: {
      level: "off|strict",
      schema: "json-schema"
    }
}
```

  Here, `validation` is JSON object which contains the keys you can use to configure JSON schema validation. The first key is `level`, which can take the value `strict` or `off`. The second key, `schema`, is a JSON schema, as defined at http://json-schema.org. If the `level` key is set to `strict`, documents are validated against the *`json-schema`* when they are added to the collection, or when an operation updates the document. If the document does not validate, the server generates an error and the operation fails. If the `level` key is set to `off`, documents are not validated against the *`json-schema`*.

  You can pass a `validation` JSON object to the `schema.createCollection()` operation, to enable JSON schema validation, and `schema.modifyCollection()` operation, to change the current JSON schema validation, for example to disable validation. For more information, see JSON Schema Validation. (WL #13019)

Bugs Fixed

- MySQL Shell plugins now support the use of the **kwargs syntax in functions defined in Python that are made available by the plugin. Using **kwargs in a function definition lets you call the function using a variable-length list of keyword arguments with arbitrary names. If the function is called from MySQL Shell's JavaScript mode, MySQL Shell passes the named arguments and their values into a dictionary

object for the Python function. MySQL Shell first tries to associate a keyword argument passed to a function with any corresponding keyword parameter that the function defines, and if there is none, the keyword argument is automatically included in the **kwargs list. As a side effect of this support, any API function called from Python in MySQL Shell that has a dictionary of options as the last parameter supports defining these options using named arguments. (Bug #31495448)

- When switching to SQL mode, MySQL Shell queries the SQL mode of the connected server to establish whether the `ANSI_QUOTES` mode is enabled. Previously, MySQL Shell could not proceed if it did not receive a result set in response to the query. The absence of a result is now handled appropriately. (Bug #31418783, Bug #99728)
- In SQL mode, when the results of a query are to be printed in table format, MySQL Shell buffers the result set before printing, in order to identify the correct column widths for the table. With very large result sets, it was possible for this practice to cause an out of memory error. MySQL Shell now buffers a maximum of 1000 rows for a result set before proceeding to format and print the table. Note that if a field in a row after the first 1000 rows contains a longer value than previously seen in that column in the result set, the table formatting will be misaligned for that row. (Bug #31304711)
- Context switching in MySQL Shell's SQL mode has been refactored and simplified to remove SQL syntax errors that could be returned when running script files using the source command. (Bug #31175790, Bug #31197312, Bug #99303)
- The user account that is used to run MySQL Shell's upgrade checker utility checkForServerUpgrade() previously required `ALL` privileges. The user account now requires only the `RELOAD`, `PROCESS`, and `SELECT` privileges. (Bug #31085098)
- In Python mode, MySQL Shell did not handle invalid UTF-8 sequences in strings returned by queries. (Bug #31083617)
- MySQL Shell's parallel table import utility importTable() has a new option `characterSet`, which specifies a character set encoding with which the input data file is interpreted during the import. Setting the option to `binary` means that no conversion is done during the import. When you omit this option, the import uses the character set specified by the `character_set_database` system variable to interpret the input data file. (Bug #31057707)
- On Windows, if the MySQL Shell package was extracted to and used from a directory whose name contained multi-byte characters, MySQL Shell was unable to start. MySQL Shell now handles directory names with multi-byte characters correctly, including when setting up Python, loading prompt themes, and accessing credential helpers. (Bug #31056783)
- MySQL Shell's JSON import utility importJSON() now handles UTF-8 encoded files that include a BOM (byte mark order) at the start, which is the sequence `0xEF 0xBB 0xBF`. As a workaround in earlier releases, remove this byte sequence, which is not needed. (Bug #30993547, Bug #98836)
- When the output format was set to JSON, MySQL Shell's upgrade checker utility checkForServerUpgrade() included a description and documentation link for a check even if no issues were found. These are now omitted from the output, as they are with the text output format. (Bug #30950035)

Changes in MySQL Shell 8.0.20 (2020-04-20, General Availability)

- AdminAPI Added or Changed Functionality
- AdminAPI Bugs Fixed
- Functionality Added or Changed

- Bugs Fixed

AdminAPI Added or Changed Functionality

- MySQL Shell now enables you to create and configure the MySQL user accounts required by InnoDB Cluster, InnoDB ReplicaSet, and MySQL Router using AdminAPI operations. Previously, accounts required by InnoDB Cluster and InnoDB ReplicaSet had to be configured using the `clusterAdmin` option, and accounts required by MySQL Router had to be configured manually using SQL. The following AdminAPI operations are now available:

  - use *`Cluster`*`.setupAdminAccount(user, [options])` and *`Replicaset`*`.setupAdminAccount(user, [options])` to configure a MySQL user account with the necessary privileges to administer an InnoDB Cluster or InnoDB ReplicaSet.

  - use *`Cluster`*`.setupRouterAccount(user, [options])` and *`Replicaset`*`.setupRouterAccount(user, [options])` to create a MySQL user account or upgrade an existing account so that it that can be used by MySQL Router to operate on an InnoDB Cluster or InnoDB ReplicaSet. This is now the recommended method of adding MySQL Router accounts to use with InnoDB Cluster and InnoDB ReplicaSet.

  (WL #13536)

- AdminAPI now uses a locking mechanism to avoid different operations from performing changes on an InnoDB ReplicaSet simultaneously. Previously, different instances of MySQL Shell could connect to an InnoDB ReplicaSet at the same time and execute AdminAPI operations simultaneously. This could lead to inconsistent instance states and errors, for example if *`ReplicaSet`*`.addInstance()` and *`ReplicaSet`*`.setPrimary()` were executed in parallel.

  Now, the InnoDB ReplicaSet operations have the following locking:

  - `dba.upgradeMetadata()` and `dba.createReplicaSet()` are globally exclusive operations. This means that if MySQL Shell executes these operations on an InnoDB ReplicaSet, no other operations can be executed against the InnoDB ReplicaSet or any of its instances.

  - *`ReplicaSet`*`.forcePrimaryInstance()` and *`ReplicaSet`*`.setPrimaryInstance()` are operations that change the primary. This means that if MySQL Shell executes these operations against an InnoDB ReplicaSet, no other operations which change the primary, or instance change operations can be executed until the first operation completes.

  - *`ReplicaSet`*`.addInstance()`, *`ReplicaSet`*`.rejoinInstance()`, and *`ReplicaSet`*`.removeInstance()` are operations that change an instance. This means that if MySQL Shell executes these operations on an instance, the instance is locked for any further instance change operations. However, this lock is only at the instance level and multiple instances in an InnoDB ReplicaSet can each execute one of this type of operation simultaneously. In other words, at most one instance change operation can be executed at a time, per instance in the InnoDB ReplicaSet.

  - `dba.getReplicaSet()` and *`ReplicaSet`*`.status()` are InnoDB ReplicaSet read operations and do not require any locking.

  (WL #13540)

  References: See also: Bug #30349849.

- Use the `--replicaset` option to configure MySQL Shell to work with an InnoDB ReplicaSet at start up. You must specify a connection to a replica set instance for this option to work correctly. If a replica set is found, this option populates the `rs` global object, which can then be used to work with the InnoDB

ReplicaSet. As part of this addition, the `--redirect-primary` and `--redirect-secondary` options have been updated to also work with InnoDB ReplicaSet.

When running MySQL Shell, use the `shell.connectToPrimary([connectionData, password])` to check whether the target instance belongs to an InnoDB Cluster or InnoDB ReplicaSet. If so, MySQL Shell opens a new session to the primary, sets the global session to the established session and returns it. If no *`connectionData`* is provided, the current global session is used. (WL #13236)

AdminAPI Bugs Fixed

- During distributed recovery which is using MySQL Clone, the instance restarts after the data files are cloned, but if the instance has to apply a large back log of transactions to finish the recovery process, then the restart process could take longer than the default 1 minute timeout. Now, when there is a large back log use the `dba.restartWaitTimeout` option to configure a longer timeout to ensure the apply process has time to process the transactions. (Bug #30866632)

- The `dba.deleteSandboxInstance()` operation did not provide an error if you attempted to delete a sandbox which did not exist. Now, in such a situation the `dba.deleteSandboxInstance()` operation throws a `runtimeError`. (Bug #30863587)

- The *`Cluster`*`.forceQuorumUsingPartitionOf()` operation was not stopping Group Replication on any reachable instances that were not part of the visible membership of the target instance, which could lead to undefined behavior if any of those instances were automatically rejoining the cluster. The fix stops Group Replication on any reachable instances that are not included in the new forced quorum membership. (Bug #30739252)

- It was possible for AdminAPI to select an invalidated instance as the latest primary, despite it having a lower `view_id`. This was because the process of getting the primary of the InnoDB ReplicaSet was incorrectly reconnecting to an invalidated member if it was the last instance in the InnoDB ReplicaSet. (Bug #30735124)

- When a cluster that was created with a MySQL Shell version lower than 8.0.19 was offline (for example after a server upgrade of the instances), if you then used MySQL Shell 8.0.19 to connect to the cluster, `dba.upgradeMetadata()` and `dba.rebootClusterFromCompleteOutage()` blocked each other. You could not run `dba.upgradeMetadata()` because it requires `dba.rebootClusterFromCompleteOutage()` to be run first to bring the cluster back online. And you could not run `dba.rebootClusterFromCompleteOutage()` because `dba.upgradeMetadata()` had not been run. To avoid this problem please upgrade to MySQL Shell 8.0.20, where the preconditions for `dba.rebootClusterFromCompleteOutage()` and `dba.forceQuorumUsingPartitionOf()` have been updated to ensure they are compatible with clusters created using earlier versions. In other words, they are available even if the metadata was created using an older MySQL Shell version. (Bug #30661129)

- Using MySQL Clone as the distributed recovery method to add an instance to an InnoDB ReplicaSet resulted in a segmentation fault if the target instance did not support `RESTART`. Now, the *`ReplicaSet`*`.addInstance()` operation aborts in such a situation and reverts changes after the connection timeout limit is reached. This is because the add operation needs to connect to the target instance to finish the operation. In such a situation, if it is not possible to upgrade the instance to a version of MySQL which supports `RESTART`, you have to restart the server manually, and then issue *`ReplicaSet`*`.addInstance()` again to retry. The retry can then use incremental recovery, which does not trigger the clone and the subsequent restart. (Bug #30657911)

- *`Cluster`*`.addInstance()` normally fails if there are errant GTIDs in the added instance, but if MySQL Clone is being used for distributed recovery, that check is bypassed because the cloning process fixes

the problem. However, if all members are using IPv6, MySQL Clone cannot be used, so incremental recovery is used instead. In such a situation, the instance was being added to the cluster without errors, but the errant transaction persisted. Now, if all of the cluster's online instances are using IPv6 addresses and the operation tries to use MySQL Clone for distributed recovery, an error is thrown. (Bug #30645697)

- When an InnoDB Cluster or InnoDB ReplicaSet is using the MySQL Clone plugin, AdminAPI ensures the `performance_schema.clone_status` table is cleared out when the clone process starts. However, in some rare and very specific scenarios a race condition could happen and the clone operation was considered to be running before actually clearing out the table. In this situation, the MySQL Shell clone monitoring could result in an unexpected halt.

  As part of this fix, a potential infinite loop in the clone monitoring phase that could happen very rarely when the cloning process was extremely fast has also been fixed. (Bug #30645665)

- Group Replication system variable queries were being executed early, without considering whether the Group Replication plugin was installed yet. Now, the reboot operation has been fixed so that if system variable queries fail with `ER_UNKNOWN_SYSTEM_VARIABLE` then the Group Replication plugin is installed automatically. (Bug #30531848)

- The *`Cluster`*`.removeInstance(`*`instance`*`)` operation was not correctly handling the following cases:

  - if the instance had `report_host` set to a different value from the `instance_name` in the metadata, specifying the instance using its IP failed.

  - if the instance was unreachable, it was not possible to remove it if the given address did not match the address in the metadata.

  - when an instance was `OFFLINE` but reachable (for example because Group Replication stopped but the server was still running), *`Cluster`*`.removeInstance(`*`instance`*`)` failed. Now, in such a situation, if you are sure it is safe to remove the instance, use the `force=true` option, which means that synchronization is no longer attempted as part of the remove operation.

  - if the instance was `OFFLINE` but reachable, removing the instance through an address that did not match what was in the metadata would make the operation appear to succeed but the instance was not actually removed from the metadata.

  (Bug #30501628, Bug #30625424)

- After operations such as removing an instance or dissolving a cluster, the `group_replication_recovery` and `group_replication_applier` replication channels were not being removed. (Bug #29922719, Bug #30878446)

- The default location of the MySQL option file, for example `/etc/my.cnf`, stopped being detected by the `dba.configureInstance()` operation on some platforms (Debian and so on). This was a regression. The fix ensures that the predefined paths to option files matches the defaults, such as `/etc/my.cnf` and `/etc/mysql/my.cnf`. (Bug #96490, Bug #30171324)

Functionality Added or Changed

- A new method `shell.openSession` is provided in the `shell` global object to let you create and return a session object, rather than set it as the global session for MySQL Shell. (WL #13328)

- You can now request compression for MySQL Shell connections that use X Protocol, as well as those that use classic MySQL protocol. For X Protocol connections, the default is that compression is

requested, and uncompressed connections are allowed if the negotiations for a compressed connection do not succeed. For classic MySQL protocol connections, the default is that compression is disabled. After the connection has been made, the MySQL Shell `\status` command shows whether or not compression is in use for a session.

New compression controls in MySQL Shell let you specify in the connection parameters whether compression is required, preferred, or disabled, select compression algorithms for the connection, and specify a numeric compression level for the algorithms. (WL #13328)

Bugs Fixed

- When you create an extension object for MySQL Shell, the `options` key is no longer required when you specify a parameter of the data type "dictionary". If you do define options for a dictionary, MySQL Shell validates the options specified by the end user and raises an error if an option is passed to the function that is not in this list. If you create a dictionary with no list of options, any options that the end user specifies for the dictionary are passed directly through to the function by MySQL Shell with no validation. (Bug #30986260)

- A bug in MySQL Shell 8.0.19, affecting classic MySQL protocol connections only, meant that access was denied if a user had stored the connection's password with MySQL Shell and then changed it. The password store now removes invalid passwords and presents the user with a password prompt as expected. (Bug #30912984, Bug #98503)

- When MySQL Shell's `\source` command was used in interactive mode to execute code from a script file, multi-line SQL statements in the script file could cause MySQL Shell to enter a loop of repeatedly executing the script. The issue has now been fixed. (Bug #30906751, Bug #98625)

- If a stored procedure was called in MySQL Shell but its result was not used, any subsequent SQL statement returned a result set error, and exiting MySQL Shell at that point resulted in an incorrect shutdown. MySQL Shell cleared the first result set retrieved by a stored procedure in order to run a subsequent SQL statement, but did not check for any additional result sets that had been retrieved, which were left behind and caused the error. This check is now carried out and the additional result sets are discarded before another statement is executed. (Bug #30825330)

- Due to a regression in MySQL Shell 8.0.19, the upgrade checker utility `checkForServerUpgrade()` did not accept any runtime options if connection data was not provided as the first argument. The issue has been fixed and the utility's argument checking has been enhanced. (Bug #30689606)

- MySQL Shell, which now bundles Python 3.7.4, could not be built from source with Python 3.8. The incompatibilities have now been corrected so Python 3.8 may be used. (Bug #30640012)

- MySQL Shell's upgrade checker utility `checkForServerUpgrade()` did not flag removed system variables that were specified using hyphens rather than underscores. The utility also now continues with its sequence of checks if a permissions check cannot be performed at the required time. (Bug #30615030, Bug #97855)

- MySQL Shell's `\status` command showed that a connection was compressed if the connection had been created while starting MySQL Shell, but not if it was created after starting MySQL Shell. Compression is now shown in both cases. (Bug #29006903)

Changes in MySQL Shell 8.0.19 (2020-01-13, General Availability)

- AdminAPI Added or Changed Functionality

- AdminAPI Bugs Fixed

- Functionality Added or Changed
- Bugs Fixed

AdminAPI Added or Changed Functionality

- **Incompatible Change:** The AdminAPI now includes InnoDB ReplicaSets, that enables you to administer asynchronous replication set ups in a similar way to InnoDB Cluster. The addition of InnoDB ReplicaSet means that the InnoDB Cluster metadata schema has been upgraded to version 2.0. Regardless of whether you plan to use InnoDB ReplicaSet or not, to use MySQL Shell 8.0.19 and AdminAPI you must upgrade the metadata of your clusters. Connect MySQL Shell's global session to your cluster and use the new `dba.upgradeMetadata()` operation to upgrade the cluster's metadata to use the new metadata.

  

  Warning

  Without upgrading the metadata you cannot use MySQL Shell 8.0.19 to change the configuration of a cluster created with earlier versions. You can only read the configuration of the cluster, for example using *`Cluster`*`.status()`.

  The `dba.upgradeMetadata()` operation upgrades any automatically created MySQL Router users to have the correct privileges. Manually created MySQL Router users with a name not starting with `mysql_router_` are not automatically upgraded. This is an important step in upgrading your cluster, only then can the MySQL Router application be upgraded. See Upgrade InnoDB Cluster.

  

  Warning

  A cluster which is using the new metadata cannot be administered by earlier MySQL Shell versions, for example once you upgrade to version 8.0.19 you can no longer use version 8.0.18 or earlier to administer the cluster.

  To get information on which of the MySQL Router instances registered with a cluster require the metadata upgrade, issue `cluster.listRouters({'onlyUpgradeRequired':'true'})`. (WL #11033, WL #13386)

- AdminAPI now supports socket connections to be used for cluster and replica set operations. (Bug #26265826)

- You can now get information about the MySQL Router instances registered with an InnoDB Cluster, and unregister a Router from a cluster, for example when you stop using it. Use the *`Cluster`*`.listRouters()` operation to show a list of all Routers registered with the cluster. The results provides information such as the hostname, ports, and so on.

  To filter the list to only show Router instances that do not support the latest metadata version use the `onlyUpgradeRequired` option, for example by issuing *`Cluster`*`.listRouters({'onlyUpgradeRequired':'true'})`. The returned information shows whether the Router instance is compatible or not with the Metadata version supported by the version of MySQL Shell you are using, which you can use when upgrading a cluster.

  To remove a registered Router from a cluster, use the *`Cluster`*`.removeRouterMetadata(`*`router`*`)` operation. (WL #13466, WL #13462)

- The AdminAPI includes support for InnoDB ReplicaSet, that enables you to administer asynchronous replication sets in a similar way to InnoDB Cluster. InnoDB ReplicaSet enables you to deploy an asynchronous replication set consisting of a single primary and multiple secondaries (traditionally referred to as the MySQL replication master and slaves). You administer a ReplicaSet using AdminAPI

operations, for example to check the status of the InnoDB ReplicaSet, and manually failover to a new primary in the event of a failure. Similar to InnoDB Cluster, MySQL Router supports bootstrapping against InnoDB ReplicaSet, which means you can automatically configure MySQL Router to use your InnoDB ReplicaSet without having to manually configure files. This makes InnoDB ReplicaSet a quick and easy way to get MySQL replication and MySQL Router up and running, making it well suited to scaling out reads, development environments, and applications that do not require the high availability offered by InnoDB Cluster. See Upgrade InnoDB Cluster. (WL #13235)

AdminAPI Bugs Fixed

- The `dba.configureLocalInstance()` operation could fail with a `key not found (LogicError)` error when executed on a non-sandbox instance where it did not have access to the `my.cnf` option file and the operation requested an output configuration file to be specified. (Bug #30657204)
- The extended status information now displays the version of metadata found on an InnoDB Cluster or InnoDB ReplicaSet. For example issue:

  ```
  mysql-js> Cluster.status({extended=1})
  mysql-js> ReplicaSet.status({extended=1})
  ```

  (Bug #30624615)
- If a cluster had been deployed with MySQL Shell version 8.0.14 or earlier, the metadata contained an invalid port number for X Protocol connections. The metadata upgrade catches such ports and removes the invalid number. To avoid problems with routing due to this incorrect port, upgrade your cluster's metadata. See Upgrade InnoDB Cluster. (Bug #30618264)
- When a replication replica was configured to read from an InnoDB Cluster primary, even with the appropriate replication filtering to ignore the metadata replication was failing when an instance was added to the cluster using MySQL Clone as the recovery method. This was because the recovery process was granting a privilege on an account, which failed and broke replication. (Bug #30609075)
- The *`Cluster`*`.removeInstance()` operation was issuing a misleading error message when the instance was unreachable, indicating that it did not belong to the cluster when an alternative valid host or IP was used. Now, the error indicates that the instance is unreachable. (Bug #30580393)
- Although MySQL Shell version 8.0.18 added support for IPv6 in WL#12758, using an InnoDB Cluster which consisted of MySQL Shell version 8.0.18 and MySQL Router 8.0.18 with IPv6 addresses was not possible. With the release of version 8.0.19 of both MySQL Shell and MySQL Router, be aware that:

  - combining MySQL Shell version 8.0.18 with MySQL Router 8.0.18 causes Router to fail, due to no IPv6 support. (Bug#30354273)
  - combining MySQL Shell version 8.0.18 with Router 8.0.19 in a cluster which uses X Protocol connections, results in AdminAPI storing `mysqlX` IPv6 values incorrectly in the metadata, causing Router to fail. (Bug#30548843) However, combining MySQL Shell version 8.0.18 with Router 8.0.19 in a cluster which uses MySQL classic protocol connections, is possible.

  Therefore, to use InnoDB Cluster with IPv6 addresses, regardless of the protocol used, the recommended deployment is MySQL Shell 8.0.19 and MySQL Router 8.0.19. (Bug #30548843)

  References: See also: Bug #30354273.
- When using automatic rejoin, if a target instance was rejoining the cluster, operations such as `dba.rebootClusterFromCompleteOutage()`, *`Cluster`*`.status()`, and so on were failing. Now, clusters consider automatic rejoin as an instance state instead of a check that always aborts the operation. This ensures that *`Cluster`*`.status()` is reported even for instances which are rejoining

the cluster, and that `dba.rebootClusterFromCompleteOutage()` can detect instances which are rejoining the cluster and override the rejoin operation so that the cluster can be properly rebooted. (Bug #30501590)

- SSL client certificate options for the `clusterAdmin` user were not being copied when setting up connection options, which made them fail when connecting. (Bug #30494198)
- When the automatically calculated `localAddress` is not valid, for example when it exceeds the valid range, the error message has now been improved. See Configuring InnoDB Cluster Ports. (Bug #30405569)
- The AdminAPI ensures that all members of a cluster have the same consistency level as configured at cluster creation time. However, when high and non-default consistency levels were chosen for the cluster, adding instances to it resulted in an error 3796 which indicates that `group_replication_consistency` cannot be used on the target instance. This happened because the consistency values of `BEFORE`, `AFTER` and `BEFORE_AND_AFTER` cannot be used on instances that are `RECOVERING` and several transactions happen while the instance is in the `RECOVERING` phase. Other AdminAPI commands result in the same error for the same scenario (high global consistency levels) whenever at least one member of the cluster is `RECOVERING`. For example, `dba.getCluster()`. The fix ensures that all sessions used by the AdminAPI use the consistency level of `EVENTUAL` when the cluster's consistency level is `BEFORE`, `AFTER` or `BEFORE_AND_AFTER`. (Bug #30394258, Bug #30401048)
- Some privileges required for persisting configuration changes on MySQL 8.0 servers were missing for the `clusterAdmin` users created by AdminAPI. In particular, an `Access Denied` error was being issued indicating the `SYSTEM_VARIABLES_ADMIN` and `PERSIST_RO_VARIABLES_ADMIN` privileges were required. Now these privileges are added for the `clusterAdmin` user on MySQL 8.0 servers. (Bug #30339460)
- When using MySQL Clone as the recovery method, trying to add an instance that did not support `RESTART` to a cluster caused MySQL Shell to stop unexpectedly. Now, in such a situation a message explains that *`Cluster`*`.rescan()` must be used to ensure the instance is added to the metadata. (Bug #30281908)
- The `autocommit` and `sql_mode` system variables are session settings, but they can be set globally to different values. AdminAPI was failing if these variables had non-default values in several different ways, for example DML was failing, system variables could not be set and so on. (Bug #30202883, Bug #30324461)
- Attempting to bootstrap MySQL Router against an InnoDB Cluster which had the cluster administration user modified or removed was failing. This was caused by the privileges granted on the InnoDB Cluster metadata table. The recommended solution is to upgrade to metadata 2.0, which changes the privileges on the metadata to ensure this issue does not occur. See Upgrade InnoDB Cluster. (Bug #29868432)
- When you created a multi-primary cluster, the `group_replication_enforce_update_everywhere_checks` system variable was not being set automatically. However, switching to multi-primary mode automatically enables `group_replication_enforce_update_everywhere_checks` and switching to single-primary disables it. Now, the `dba.createCluster()` operation sets the `group_replication_enforce_update_everywhere_checks` variable as appropriate for single-primary or multi-primary clusters. (Bug #29794779)
- In version 8.0.16, the `autoRejoinTries` option was added to define the number of times an instance attempts to rejoin the cluster after being expelled. The option is a valid cluster setting, configurable through the AdminAPI like many other options. However, the `autoRejoinTries` option was not being listed by *`Cluster`*`.options()`. (Bug #29654346)

- The InnoDB Cluster metadata now supports host names up to 265 characters long, where 255 characters can be the host part and the remaining characters can be the port number. (Bug #29507913)

- `dba.createCluster()` could fail if the instance had been started with `innodb_default_row_format=COMPACT` or `innodb_default_row_format=REDUNDANT`. This was because no `ROW_FORMAT` was specified on the InnoDB Cluster metadata tables, which caused them to use the one defined in `innodb_default_row_format`. The metadata schema has been updated to use `ROW_FORMAT = DYNAMIC`. (Bug #28531271)

- When an instance restarted, for example after a complete outage, it could have `super_read_only` disabled. This meant that instances which were not the primary could be written to, resulting in the instances no longer being in synchrony. This could result in `dba.rebootClusterFromCompleteOutage()` failing with a `Conflicting transaction sets` error. The fix ensures that all instances have `super_read_only=1` persisted while they belong to the cluster, either through `SET PERSIST_ONLY`, or through `dba.configureLocalInstance()` for instances which do not support persisting. (Bug #97279, Bug #30545872)

- The *`Cluster`*`.status()` operation could report an error `get_uint(24): field value out of the allowed range` because it was always expecting a positive value for some fields that could in fact have negative values. For example, this could happen when the clocks of different instances were offset. (Bug #95191, Bug #29705983)

- If you changed the name of the `clusterAdmin` user once a cluster had been created, you could encounter an error such as `The user specified as a definer does not exist`. This was because the `clusterAdmin` user was used as the `DEFINER` of the views required by InnoDB Cluster, and if this user is renamed then the definer is in effect missing. In version 8.0.19 the InnoDB Cluster metadata has been changed to avoid this problem, use `dba.upgradeMetadata()` to upgrade the cluster. See Upgrade InnoDB Cluster. Clusters deployed with 8.0.19 and later do not suffer from this issue. (Bug #92128, Bug #28541069)

- It was not possible to create a multi-primary cluster due to cascading constraints on the InnoDB Cluster metadata tables. This has been fixed in version 8.0.19 and so to solve this issue upgrade your cluster using `dba.upgradeMetadata()`. See Upgrade InnoDB Cluster. (Bug #91972, Bug #29137199)

Functionality Added or Changed

- The JavaScript function `require()` has been improved in MySQL Shell to support loading of local modules, in addition to built-in modules and modules that are on module search paths already known to MySQL Shell. If you specify the module name or path prefixed with `./` or `../`, MySQL Shell now searches for the specified module in the folder that contains the JavaScript file or module currently being executed, or in interactive mode, searches in the current working directory. If the module is not found in that folder, MySQL Shell proceeds to check the well-known module search paths specified by the sys.path variable. (WL #13119)

- MySQL Shell's upgrade checker utility (the `util.checkForServerUpgrade()` operation) includes the following new and extended checks:

  - The utility now flags all `date`, `datetime`, and `timestamp` columns that have a default value of zero, and states if the SQL mode (either global or for the current session) allows the insertion of zero values for these column types. By default, these are no longer permitted in MySQL, and it is strongly advised to replace zero values with valid ones, as they might not work correctly in the future.

  - The check for usage of removed functions now includes the `PASSWORD()` function.

  - The utility now checks for any orphaned tables which are recognized by `InnoDB`, but the SQL layer thinks they are handled by a different storage engine. This situation can happen if manual updates are made in the data directory. Orphaned tables can stall the upgrade process if they are present.

  (WL #13526)

- In MySQL Shell's interactive mode, for JavaScript, Python, or SQL, the `\source` command or its alias `\.` can be used to execute code from a script file at a given path. For compatibility with the `mysql` client, in SQL mode only, you can now execute code from a script file using the `source` command with no backslash and an optional SQL delimiter. `source` can be used both in MySQL Shell's interactive mode for SQL, to execute a script directly, and in a file of SQL code processed in batch mode, to execute a further script from within the file. In SQL mode only, you can also now use the `\source` command's alias `\.` (which does not use a SQL delimiter) in a file of SQL code processed in batch mode. So with MySQL Shell in SQL mode, you could now execute the script in the `/tmp/mydata.sql` file from either interactive mode or batch mode using any of these three commands:

  ```
  source /tmp/mydata.sql;
  source /tmp/mydata.sql
  \. /tmp/mydata.sql
  ```

  The command `\source /tmp/mydata.sql` is also valid, but in interactive mode only. (WL #13320)

Bugs Fixed

- When searching for startup scripts in the platform's standard global configuration path (in the folder `%PROGRAMDATA%\MySQL\mysqlsh` on Windows, or `/etc/mysql/mysqlsh/` on Unix), MySQL Shell checked for the incorrect script name `shellrc`, rather than the correct name `mysqlshrc`. (Bug #30656548)

- On Windows, MySQL Shell passed UTF-8 encoded strings to the NTFS file system, which stores file names in UTF-16 encoding. The mismatch caused files and folders to be incorrectly named or located when non-ASCII characters were used. MySQL Shell now converts all strings used in operations on NTFS from UTF-8 to UTF-16, and converts back to UTF-8 all strings received from Unicode function versions of Windows API calls. (Bug #30538516)

- Some host names were not parsed correctly in the connection data provided when running MySQL Shell's upgrade checker utility `checkForServerUpgrade()`, including where the user account's host name had been set up by specifying an IP address and subnet mask. (Bug #30536355, Bug #30696901, Bug #98056)

- If the MySQL Server environment variable `MYSQL_UNIX_PORT` (which specifies the default Unix socket file) was updated by the same process that was then used to create a MySQL Shell connection to a MySQL server using a socket file, MySQL Shell cached and connected using the socket file path that had previously been set, but reported that a connection had been made using the updated socket file path. The correct socket file used for the connection is now displayed. (Bug #30533318)

- If a prompt theme file used to customize the MySQL Shell prompt contained an ill-formed UTF-8 sequence, on startup an error message was displayed in place of the prompt text. MySQL Shell now validates the prompt theme file before loading it, and if there is a problem, uses a default prompt instead and issues an error message. (Bug #30406283)

- If MySQL Shell was installed on Microsoft Windows at a non-default location, and subsequently uninstalled, files created after installation by the Python library used by MySQL Shell were not removed. These files are now removed when MySQL Shell is uninstalled from any location. (Bug #30333801)

- Previously, most MySQL Shell options that expected an integer value could be set with an empty value, in which case the value 1 was applied. The exception was the `logLevel` option, which required a value. The behavior has now been standardized so all MySQL Shell options that expect a non-string value must be specified with a value, with the exception of options set on the command line. The affected options are `dba.gtidWaitTimeout`, `dba.logSql`, `history.maxSize`, and `verbose`. (Bug #30320839)
- When using MySQL Shell in interactive mode, using a template literal in a multiple-line JavaScript statement resulted in an error. The issue has now been fixed. (Bug #30248651)
- In Python mode, when multiple statements were input to MySQL Shell at the same time for execution in interactive mode, only the first statement was executed correctly. (Bug #30029568)
- The Debian control file for MySQL Shell has been corrected to remove packaging errors that occurred when certain variables were not defined. Thanks to Evgeniy Patlan for the fix. (Bug #29802600, Bug #95158)
- In MySQL Shell's parser for URI-like connection strings, handling of path separators was previously platform dependent. Unified parsing has now been introduced so that Windows named pipes can be parsed correctly on Unix platforms, and Unix socket files can be parsed correctly on Windows platforms. (Bug #29456981)
- MySQL Shell now looks up host names by obtaining the fully qualified domain name of the provided address and using the absolute form of this name (with a trailing dot). This method avoids potential issues caused by some network configurations that resolve host names as loopback addresses when they are actually addressable externally. (Bug #27704559)

Changes in MySQL Shell 8.0.18 (2019-10-14, General Availability)

- AdminAPI Added or Changed Functionality
- AdminAPI Bugs Fixed
- Functionality Added or Changed
- Bugs Fixed

AdminAPI Added or Changed Functionality

- MySQL Shell can now optionally log SQL statements that are executed by AdminAPI operations, and output them to the console if the `--verbose` option is set. The `dba.logSql` MySQL Shell configuration option or `--dba-log-sql` command line option activates logging for these statements. Statements executed by sandbox operations are excluded. Viewing the statements lets you observe the progress of the AdminAPI operations in terms of SQL execution, which can help with problem diagnosis for any errors. (WL #13294)
- AdminAPI now supports IPv6 addresses if the target MySQL Server version is higher than 8.0.13. When using MySQL Shell 8.0.18 or higher, if all cluster instances are running 8.0.14 or higher then you can use an IPv6 or hostname that resolves to an IPv6 address for instance connection strings and with options such as `localAddress`, `groupSeeds` and `ipWhitelist`. For more information on using IPv6 see Support For IPv6 And For Mixed IPv6 And IPv4 Groups. (WL #12758)

  References: See also: Bug #29557250, Bug #30111022, Bug #28982989.
- You can now reset the passwords for the internal recovery accounts created by InnoDB Cluster, for example to follow a custom password lifetime policy. Use the

Cluster.resetRecoveryAccountsPassword() operation to reset the passwords for all internal recovery accounts used by the cluster. The operation sets a new random password for the internal recovery account on each instance which is ONLINE. If an instance cannot be reached, the operation fails. You can use the force option to ignore such instances, but this is not recommended, and it is safer to bring the instance back online before using this operation. This operation only applies to the passwords created by InnoDB cluster and cannot be used to update manually created passwords.



Note

The user which executes this operation must have all the required clusterAdmin privileges, in particular CREATE USER, in order to ensure that the password of recovery accounts can be changed regardless of the password verification-required policy. In other words, independent of whether the password_require_current system variable is enabled or not.

(WL #12776)

- MySQL Shell now supports specifying TLS version 1.3 and TLS cipher suites for classic MySQL protocol connections. You can use:
  - the --tls-version command option to specify TLS version 1.3.
  - the --tls-ciphersuites command option to specify cipher suites.
  - the tls-versions and tls-ciphersuites connection parameters as part of a URI-type connection string.



Note

tls-versions (plural) does not have a key-value equivalent, it is only supported in URI-type connection strings. Use tls-version to specify TLSv1.3 in a key-value connection string.

To use TLS version 1.3, both MySQL Shell and MySQL server must have been compiled with OpenSSL 1.1.1 or higher. For more information see Using Encrypted Connections. (WL #12768)

AdminAPI Bugs Fixed

- The *Cluster*.rejoinInstance() operation was not setting the auto increment values defined for InnoDB Cluster, leading to the use of the default Group Replication behavior if the instance configuration was not properly persisted, for example on 5.7 servers. The fix ensures that the *Cluster*.rejoinInstance() operation updates the auto increment settings of the target instance. (Bug #30174191)
- The output of *Cluster*.status() now includes the replicationLag field. The value is displayed in HH:MM:SS format and shows the time difference between the last transaction commit timestamp and the last transaction applied timestamp. This enables you to monitor the amount of time between the most recent transaction being committed and being applied on an instance. (Bug #30003034)
- *Cluster*.addInstance() did not ensure that the MySQL Clone plugin was installed or loaded on all cluster instances, when available and not disabled. This meant that whenever a cluster was created using an older MySQL Shell version, on a target MySQL instance supporting clone, the instance would not have the clone plugin installed. The result was that any *Cluster*.addInstance() call that used clone would fail. The same issue happened if an instance was added to a cluster consisting of one instance using the incremental recovery type and afterwards the seed instance was removed. This resulted in all cluster instances not having the clone plugin installed and therefore any instance added

using the clone recovery method would fail. The fix ensures that the clone plugin is installed on all cluster members (if available and not disabled) at cluster creation time and also whenever an instance is added to a cluster. (Bug #29954085)

- The *`Cluster`*`.rejoinInstance()` operation was not checking the GTID consistency of an instance being rejoined to a cluster, which could result in data diverging. Now, the GTID consistency checks conducted as part of the *`Cluster`*`.rejoinInstance()` operation have been improved to check for irrecoverable or diverged data-sets and also for empty GTID sets. If an instance is found to not be consistent with the cluster, it is not rejoined and the operation fails with a descriptive error. You are also shown the list of errant transactions, possible outcomes and solutions. (Bug #29953812)

- *`Cluster`*`.describe()` was retrieving information about the cluster's topology and the MySQL version installed on instances directly from the current session. Now, the information is retrieved from the Metadata schema, and the MySQL version is not included in the information output by *`Cluster`*`.describe()`. (Bug #29648806)

- Using a password containing the ' character caused `dba.deploySandbox()` to fail. Now, all sensitive data is correctly wrapped to avoid such issues. (Bug #29637581)

- The *`Cluster`*`.addInstance()` operation creates internal recovery users which are required by the Group Replication recovery process. If the *`Cluster`*`.addInstance()` operation failed, for example because Group Replication could not start, the created recovery users were not removed. Now, in the event of a failure any internal users are removed. (Bug #25503159)

- When a cluster had lost quorum and the majority of the cluster instances were offline except the primary, after reestablishing quorum and adding a new instance to the cluster, it was not possible to remove and add the previous primary instance to the cluster. This was because the operation failed when trying to contact offline instances, which was because the feature to verify if a Group Replication protocol upgrade is required was not considering the possibility of some cluster instances being offline (not reachable). The fix improves the Group Replication protocol upgrade handling for the *`Cluster`*`.removeInstance()` operation, which now attempts to connect to other cluster instances and use the first reachable instance for this purpose. (Bug #25267603)

- The `dba.configureInstance()` operation was not setting the `binlog_checksum` option with the required value (NONE) in the option file for instances that did not support `SET PERSIST` (for example instances running MySQL 5.7), when the option file path was not provided as an input parameter but instead specified though the operation wizard in interactive mode. (Bug #96489, Bug #30171090)

Functionality Added or Changed

- MySQL Shell's upgrade checker utility (the `util.checkForServerUpgrade()` operation) includes the following new and extended checks:

  - The utility now checks for tablespace names containing the string “FTS”, which can be incorrectly identified as tablespaces of full-text index tables, preventing upgrade. The issue has been fixed in MySQL 8.0.18, but affects upgrades to earlier MySQL 8.0 releases.

  - The check for database objects with names that conflict with reserved keywords now covers the additional keywords `ARRAY`, `MEMBER`, and `LATERAL`.

  - - The checks for obsolete `sql_mode` flags now check the global `sql_mode` setting.

  Running the upgrade checker utility no longer alters the `gtid_executed` value, meaning that the utility can be used on Group Replication group members without affecting their synchronization with the group. The upgrade checker also now works correctly with the `ANSI_QUOTES` SQL mode. (Bug #30002732, Bug #30103683, Bug #96351, Bug #30103640, Bug #96350, WL #13376)

References: See also: Bug #29992589.

- MySQL Shell has two new built-in reports, which provide information drawn from various sources including MySQL's Performance Schema:

  - `threads` lists the current threads in the connected MySQL server which belong to the user account that is used to run the report. Using the report-specific options, you can choose to show foreground threads, background threads, or all threads. You can report a default set of information for each thread, or select specific information to include in the report from a larger number of available choices. You can filter, sort, and limit the output.

  - `thread` provides detailed information about a specific thread in the connected MySQL server. By default, the report shows information on the thread used by the current connection, or you can identify a thread by its ID or by the connection ID. You can select one or more categories of information, or view all of the available information about the thread.

  You can run the new reports using MySQL Shell's `\show` and `\watch` commands. The reports work with servers running all supported MySQL 5.7 and MySQL 8.0 versions. If any item of information is not available in the MySQL Server version of the target server, the reports leave it out. (WL #11651)

- MySQL Shell has two new control commands:

  - The `\edit` (`\e`) command opens a command in the default system editor for editing. If you specify an argument to the command, this text is placed in the editor, and if you do not, the last command in the MySQL Shell history is placed in the editor. When you have finished editing, MySQL Shell presents your edited text ready for you to execute or cancel. The command can also be invoked using the short form `\e` or the key combination **Ctrl-X Ctrl-E**.

  - The `\system` (`\!`) command runs the operating system command that you specify as an argument to the command, then displays the output from the command in MySQL Shell. MySQL Shell returns an error if it was unable to execute the command.

  (WL #12763)

- MySQL Shell now uses Python 3. For platforms that include a system supported installation of Python 3, MySQL Shell uses the most recent version available, with a minimum supported version of Python 3.4.3. For platforms where Python 3 is not included, MySQL Shell bundles Python 3.7.4. MySQL Shell maintains code compatibility with Python 2.6 and Python 2.7, so if you require one of these older versions, you can build MySQL Shell from source using the appropriate Python version. (WL #13184)

Bugs Fixed

- In debug mode, MySQL Shell raised an assertion when handling a character contained in SQL strings. (Bug #30286680)

- If a Python lambda was added as a member of a MySQL Shell extension object, the Python object was not released correctly when MySQL Shell shut down, causing a segmentation fault. (Bug #30156304)

- A memory leak could occur when Python code was executed in interactive mode. (Bug #30138755)

- Help information for a MySQL Shell report could not be displayed unless there was an active session. MySQL Shell now checks for an open session only before actually running the report. (Bug #30083371)

- If a default schema was set for the MySQL Shell connection, and a different default schema was set after the connection was made, MySQL Shell's `\reconnect` command attempted to use the default

schema from the original connection. The user's current default schema is now used for the reconnection attempt. (Bug #30059354)

- Due to a bug introduced by a change in MySQL Shell 8.0.16, the MSI file that is used by Windows Installer to install MySQL Shell overwrote the Windows PATH environment variable with the path to the application binary (mysqlsh), removing any other paths present. The issue has now been fixed. (Bug #29972020, Bug #95432)
- When the `\reconnect` command is used to attempt reconnection to a server, if the last active schema set by the user appears to be no longer available, MySQL Shell now attempts to connect with no schema set. (Bug #29954572)
- In interactive mode, MySQL Shell now handles multiline comments beginning with a slash and asterisk (/*) and ending with an asterisk and slash (*/). (Bug #29938424)
- The MySQL Shell `\source` command was not handled correctly when used in combination with SQL statements. (Bug #29889926)
- With MySQL Shell in SQL mode, if multiple SQL statements including a `USE` statement were issued on a single line with delimiters, the `USE` statement was not handled correctly. (Bug #29881615)
- If MySQL Shell's JSON import utility was used to send a large number of JSON documents to a server with insufficient processing capacity, the utility could fill up the write queue with batches of prepared documents, causing the connection to time out and the import to fail. The utility now waits to read the response from the server before sending the next batch of prepared documents to the server. (Bug #29878964)
- When MySQL Shell was built from source with a bundled OpenSSL package, the required linker flags were not set. (Bug #29862189)
- If a new query was executed in MySQL Shell while a result was still active, resulting in rows being cached, not all rows were returned by the old query. (Bug #29818714)

Changes in MySQL Shell 8.0.17 (2019-07-22, General Availability)

- AdminAPI Added or Changed Functionality
- AdminAPI Bugs Fixed
- Functionality Added or Changed
- Bugs Fixed

AdminAPI Added or Changed Functionality

- **Important Change:** The handling of internal recovery accounts created by InnoDB Cluster has been changed so that by default accounts are always created as “mysql_innodb_cluster_*`server_id`*@%”, where *`server_id`* is instance specific. This generated recovery account name is stored in the InnoDB Cluster metadata, to ensure the correct account is always removed if the instance is removed from the cluster.

  The previous behavior where multiple accounts would be created if `ipWhitelist` was given has been removed. In addition *`Cluster`*`.removeInstance()` no longer removes all recovery accounts on the instance being removed. It now removes the recovery account of the instance being removed on the primary and waits for the changes to be replicated before actually removing the instance from the group. Similarly, *`Cluster`*`.rejoinInstance()` no longer drops any recovery accounts. It only creates the recovery account of the instance being rejoined if it no longer exists on the primary

(which it should in normal circumstances). If the recovery account already exists, it is reused by *`Cluster`*`.rejoinInstance()`.

When a cluster is adopted from an existing Group Replication deployment, new recovery accounts are created and set for each member. Pre-existing accounts configured by the user are left unchanged and not dropped, unless they have the “mysql_innodb_cluster_” prefix.

As part of this work, the behavior of `dba.createCluster()` and `Cluster.rebootClusterFromCompleteOutage()` operations has been changed. Now, if these operations encounter an instance which has `super_read_only=ON`, it is disabled automatically. Therefore the `clearReadOnly` option has been deprecated for these operations. (WL #12773)

References: See also: Bug #29629121, Bug #29559303.

- The `dba.createCluster()` operation has been improved, and as part of this work the order in which some steps of the operation are executed was changed. Now, the creation of the recovery (replication) user and updates to the Metadata are performed after bootstrapping the Group Replication group. As part of this work, the `dba.createCluster()` operation has been updated to support the `interactive` option, which is a boolean value that controls the wizards provided. When `interactive` is true, prompts and confirmations are displayed by the operation. The default value of `interactive` is equal to `useWizards` option. (WL #12011)

- The compatibility policies that Group Replication implements for member versions in groups now consider the patch version of a member's MySQL Server release. Previously, when combining instances running different MySQL versions, only the major version was considered. InnoDB Cluster has been updated to support cluster operations where these compatibility policies have an impact. Using the patch version ensures better replication safety for mixed version groups during group reconfiguration and upgrade procedures. As part of this work the information provided about instances has been extended.

  The following InnoDB Cluster changes have been made to support the compatibility policies:

  - The *`Cluster`*`.addInstance()` operation now detects incompatibilities due to MySQL versions and in the event of an incompatibility aborts with an informative error.
  - The *`Cluster`*`.status()` attribute `mode` now considers the value of `super_read_only` and whether the cluster has quorum.
  - The *`Cluster`*`.status()` output now includes the boolean attribute `autoRejoinRunning`, which is displayed per instance belonging to the cluster and is true when automatic rejoin is running.
  - The `extended` option has been changed to accept integer or Boolean values. This makes the behavior similar to the `queryMembers` option, so that option has now been deprecated.

  (WL #13084)

  References: See also: Bug #29557250.

- InnoDB Cluster supports the new MySQL Clone plugin on instances running 8.0.17 and later. When an InnoDB Cluster is configured to use MySQL Clone, instances which join the cluster choose whether to use Group Replication's distributed recovery or MySQL Clone to recover the transactions processed by the cluster. You can optionally configure this behavior, for example to force cloning, which replaces any transactions already processed. You can also configure how *`Cluster`*`.addInstance()` behaves, letting cloning operations proceed in the background or showing different levels of progress in MySQL Shell. This enables you to automatically provision instances in the most efficient way. In addition, the output of *`Cluster`*`.status()` for members in `RECOVERING` state has been extended to include

recovery progress information to enable you to easily monitor recovery operations, whether they be using MySQL Clone or distributed recovery. (WL #13208)

AdminAPI Bugs Fixed

- **Important Change:** The sandboxes deployed using the AdminAPI did not support the `RESTART` statement. Now, the wrapper scripts call `mysqld` in a loop so that there is a monitoring process which ensures that `RESTART` is supported. (Bug #29725222)

- The *`Cluster`*`.addInstance()` operation did not validate if the `server_id` of the joining instance was not unique among all cluster members. Although the use of a unique `server_id` is not mandatory for Group Replication to work properly (because all internal replication channels use `--replicate-same-server-id=ON`), it was recommended that all instances in a replication stream have a unique `server_id`. Now, this recommendation is a requirement for InnoDB Cluster, and when you use the *`Cluster`*`.addInstance()` operation if the `server_id` is already used by an instance in the cluster then the operation fails with an error. (Bug #29809560)

- InnoDB Clusters do not support instances that have binary log filters configured, but replication filters were being allowed. Now, instances with replication filters are also blocked from InnoDB Cluster usage. (Bug #29756457)

  References: See also: Bug #28064729, Bug #29361352.

- On instances running version 8.0.16, the *`Cluster`*`.rejoinInstance()` operation failed when one or more cluster members were in `RECOVERING` state, because the Group Replication communication protocol could not be obtained. More specifically, the `group_replication_get_communication_protocol()` function failed because it could only be executed if all members were `ONLINE`. Now, in the event of the function failing when rejoining an instance a warning is displayed and AdminAPI proceeds with the execution of the operation.

  Starting from MySQL 8.0.17, the `group_replication_get_communication_protocol()` function no longer issues an error if a member is `RECOVERING`. (Bug #29754915)

- On Debian-based hosts, `hostname` resolves to the IP address 127.0.1.1 by default, which does not match a real network interface. This is not supported by Group Replication, which made sandboxes deployed on such hosts unusable unless a manual change to the configuration file was made. Now, the sandbox configuration files created by MySQL Shell contain the following additional line:

  ```
  report_host = 127.0.0.1
  ```

  In other words the `report_host` variable is set to the loopback IP address. This ensures that sandbox instances can be used on Debian-based hosts without any additional manual changes. (Bug #29634828)

- If the binary logs had been purged from all cluster instances, *`Cluster`*`.checkInstanceState()` lacked the ability to check the instance's state, resulting in erroneous output values. Now, *`Cluster`*`.checkInstanceState()` validates the value of `GTID_PURGED` on all cluster instances and provides the correct output and also an informative message mentioning the possible actions to be taken. In addition, *`Cluster`*`.addInstance()` and *`Cluster`*`.rejoinInstance()` were not using the checks performed by *`Cluster`*`.checkInstanceState()` in order to verify the GTID status of the target instance in relation to the cluster. In the event of all cluster instances having their binary logs purged, the *`Cluster`*`.addInstance()` command would succeed but the instance would never be able to join the cluster as distributed recovery failed to execute. Now, both operations make use of the checks performed by *`Cluster`*`.checkInstanceState()` and provide informative error messages. (Bug #29630591, Bug #29790569)

- When using the `dba.configureLocalInstance()` operation in interactive mode, if you provided the path to an option file it was ignored. (Bug #29554251)

- Calling `cluster.removeInstance()` on an instance that did not exist, for example due to a typo or because it was already removed, resulted in a prompt asking whether the instance should be removed anyway, and the operation then failing. (Bug #29540529)

- To add or rejoin an instance to an existing InnoDB Cluster, the instance must not already be operating as a replica in asynchronous replication. Previously, `dba.checkInstanceConfiguration()` incorrectly reported target instances operating as a replica as valid for InnoDB Cluster usage. As a consequence, attempting to use the instance which had been incorrectly validated with operations such as *`Cluster`*`.addInstance()` failed without informative errors.

  Now, `dba.checkInstanceConfiguration()` verifies if the target instance is already configured as a replica and generates a warning if that is the case. Similarly, the *`Cluster`*`.addInstance()` and *`Cluster`*`.rejoinInstance()` operations detect such instances and block them from InnoDB Cluster usage, failing with an error. Note that this does not prevent instances which belong to a cluster also operating as the source in asynchronous replication. (Bug #29305551)

- The `dba.createCluster()` operation was allowed on a target instance that already had a populated Metadata schema, when the instance was already in that Metadata. The Metadata present on the target instance was being overridden, which was unexpected. Now, in such a situation the `dba.createCluster()` throws an exception and you can choose to either drop the Metadata schema or reboot the cluster. (Bug #29271400)

- When a sandbox instance of MySQL had been successfully started from MySQL Shell using `dba.startSandboxInstance()`, pressing **Ctrl+C** in the same console window terminated the sandbox instance. Sandbox instances are now launched in a new process group so that they are not affected by the interrupt. (Bug #29270460)

- During the creation of a cluster using the AdminAPI, some internal replication users are created with user names which start with “mysql_innodb_cluster”. However, if the MySQL server had a global password expiration policy defined, for example if `default_password_lifetime` was set to a value other than zero, then the passwords for the internal users expired after reaching the specified period. Now, the internal user accounts are created by the AdminAPI with password expiration disabled. (Bug #28855764)

- The `dba.checkInstanceConfiguration()` and `dba.configureInstance()` operations were not checking the validity of persisted configurations, which can be different from the corresponding system variable value, in particular when changed with `SET PERSIST_ONLY`. This could lead these operations to report wrong or inaccurate results, for example reporting that the instance configuration is correct when in reality the persisted configuration was invalid and wrong settings could be applied after a restart of the server, or inaccurately reporting that a server update was needed when only a restart was required. (Bug #28727505)

  References: See also: Bug #29765093.

- When you removed an instance's metadata from a cluster without removing the metadata from the instance itself (for example because of wrong authentication or when the instance was unreachable) the instance could not be added again to the cluster. Now, another validation has been added to *`Cluster`*`.addInstance()` to verify if the instance already belongs to the cluster's underlying group but is not in the InnoDB Cluster metadata, issuing an error if it already belongs to the ReplicaSet. Similarly, an error is issued when the default port automatically set for the local address is invalid (out of range) instead of using a random port. (Bug #28056944)

- When issuing `dba.configureInstance()` in interactive mode and after selecting option number 2 “Create a new admin account for InnoDB cluster with minimal required grants” it was not possible to enter a password for the new user.

Functionality Added or Changed

- MySQL Shell has a new function for SQL query execution for X Protocol sessions that works in the same way as the function for SQL query execution in classic MySQL protocol sessions. The new function, `Session.runSql()`, can be used in MySQL Shell only as an alternative to X Protocol's `Session.sql()` to create a script that is independent of the protocol used for connecting to the MySQL server. Note that `Session.runSql()` is exclusive to MySQL Shell and is not part of the standard X DevAPI. As part of this change, the `ClassicSession.query` function for SQL query execution, which is a synonym of `ClassicSession.runSQL()`, is now deprecated.

  A new function `fetchOneObject()` is also provided for classic MySQL protocol and X Protocol sessions to return the next result as a scripting object. Column names are used as keys in the dictionary (and as object attributes if they are valid identifiers), and row values are used as attribute values in the dictionary. This function enables the query results to be browsed and used in protocol-independent scripts. Updates made to the returned object are not persisted on the database. (WL #12766)

- MySQL Shell's new parallel table import utility provides rapid data import to a MySQL relational table for large data files. The utility analyzes an input data file, divides it into chunks, and uploads the chunks to the target MySQL server using parallel connections. The utility is capable of completing a large data import many times faster than a standard single-threaded upload using a `LOAD DATA` statement.

  When you invoke the parallel table import utility, you specify the mapping between the fields in the data file and the columns in the MySQL table. You can set field- and line-handling options as for the `LOAD DATA` command to handle data files in arbitrary formats. The default dialect for the utility maps to a file created using a `SELECT ... INTO OUTFILE` statement with the default settings for that statement. The utility also has preset dialects that map to the standard data formats for CSV files (created on DOS or UNIX systems), TSV files, and JSON, and you can customize these using the field- and line-handling options as necessary. (WL #12193)

- MySQL Shell has a number of new display options for query results:

  - The `shell.dumpRows()` function can format a result set returned by a query in any of the output formats supported by MySQL Shell, and dump it to the console. Note that the result set is consumed by the function. This function can be used in MySQL Shell to display the results of queries run by scripts to the user in the same ways as the interactive SQL mode can.

  - The new MySQL Shell output format `json/array` produces raw JSON output wrapped in a JSON array. The output format `ndjson` is added as a synonym for `json/raw`, and both those output formats produce raw JSON output delimited by newlines. You can select MySQL Shell output formats by starting MySQL Shell with the `--result-format=[value]` command line option, or setting the MySQL Shell configuration option `resultFormat`.

  A new function `shell.unparseUri()` is also added, which converts a dictionary of URI components and connection options into a valid URI string for connecting to MySQL. (WL #13030)

- You can now extend MySQL Shell with plugins that are loaded at startup. MySQL Shell plugins can be written in either JavaScript or Python, and the functions they contain are available in MySQL Shell in both JavaScript and Python modes. The plugins can be used to contain functions that are registered as MySQL Shell reports, and functions that are members of extension objects that are made available as part of user-defined MySQL Shell global objects.

  You can create a MySQL Shell plugin by storing code in a subfolder of the `plugins` folder in the MySQL Shell user configuration path, with an initialization file that MySQL Shell locates and executes at startup. You can structure a plugin group, with a collection of related plugins that can share common code, by placing the subfolders for multiple plugins in a containing folder under the `plugins` folder. (WL #13051)

- You can now extend the base functionality of MySQL Shell by defining extension objects and making them available as part of additional MySQL Shell global objects. Extension objects can be written in JavaScript or Python. When you create and register an extension object, it is available in MySQL Shell in both JavaScript and Python modes. You construct and register extension objects using functions provided by the built-in global object `shell`. (WL #12625)

- You can now configure MySQL Shell to send logging information to the console, in addition to sending it to the application log. The `--verbose` command-line option and the `verbose` MySQL Shell configuration option activate this function. By default, when the option is set, internal error, error, warning, and informational messages are sent to the console, which is the equivalent to a logging level of 5 for the application log. You can add three further levels of debug messages, up to the highest level of detail. (WL #13047)

- MySQL Shell's upgrade checker utility (the `util.checkForServerUpgrade()` operation) carries out two new checks. When checking for upgrade from any MySQL 5.7 release to any MySQL 8.0 release, the utility identifies partitioned tables that use storage engines other than InnoDB or NDB and therefore rely on generic partitioning support from the MySQL server, which is no longer provided. When checking for upgrade from any release to MySQL 8.0.17, the utility identifies circular directory references in tablespace data file paths, which are no longer permitted. (WL #13140)

- X DevAPI now supports indexing array fields. A single index field description can contain a new member name `array` that takes a Boolean value. If set to true, the field is assumed to contain arrays of elements of the given type. In addition, the set of possible index field data types (used as values of member type in index field descriptions) is extended with type `CHAR(N)`, where the length N is mandatory. (WL #12254)

- MySQL Shell now supports the ability to send connection attributes (key-value pairs that application programs can pass to the server at connect time). MySQL Shell defines a default set of attributes, which can be disabled or enabled. In addition, applications can specify attributes to be passed in addition to the default attributes. The default behavior is to send the default attribute set.

  You specify connection attributes as a `connection-attributes` parameter in a connection string. The `connection-attributes` parameter value must be empty (the same as specifying `true`), a `Boolean` value (`true` or `false` to enable or disable the default attribute set), or a list or zero or more `key=value` specifiers separated by commas (to be sent in addition to the default attribute set). Within a list, a missing key value evaluates as an empty string. Examples:

  ```
  "mysqlx://user@host?connection-attributes"
  "mysqlx://user@host?connection-attributes=true"
  "mysqlx://user@host?connection-attributes=false"
  "mysqlx://user@host?connection-attributes=[attr1=val1,attr2,attr3=]"
  "mysqlx://user@host?connection-attributes=[]"
  ```

  You can specify connection attributes for both X Protocol connections and classic MySQL protocol connections. The default attributes set by MySQL Shell are:

  ```
  > \sql SELECT ATTR_NAME, ATTR_VALUE FROM performance_schema.session_account_connect_attrs;
  +-----------------+------------+
  | ATTR_NAME       | ATTR_VALUE |
  +-----------------+------------+
  | _pid            | 28451      |
  | _platform       | x86_64     |
  | _os             | Linux      |
  | _client_name    | libmysql   |
  | _client_version | 8.0.17     |
  | program_name    | mysqlsh    |
  +-----------------+------------+
  ```

  Application-defined attribute names cannot begin with _ because such names are reserved for internal attributes.

If connection attributes are not specified in a valid way, an error occurs and the connection attempt fails.

For general information about connection attributes, see Performance Schema Connection Attribute Tables. (WL #12446)

- MySQL Shell now supports the `OVERLAPS` and `NOT OVERLAPS` operators for expressions on JSON arrays or objects:

```
expr OVERLAPS expr
expr NOT OVERLAPS expr
```

These operators behave in a similar way to the `JSON_OVERLAPS()` function. Suppose that a collection has these contents:

```
mysql-js> myCollection.add([{ "_id": "1", "list": [1, 4] }, { "_id": "2", "list": [4, 7] }])
```

This operation:

```
mysql-js> var res = myCollection.find("[1, 2, 3] OVERLAPS $.list").fields("_id").execute();
mysql-js> res
```

Should return:

```
{
"_id": "1"
}
1 document in set (0.0046 sec)
```

This operation:

```
mysql-js> var res = myCollection.find("$.list OVERLAPS [4]").fields("_id").execute();
mysql-js> res
```

Should return:

```
{
"_id": "1"
}
{
"_id": "2"
}
2 documents in set (0.0031 sec)
```

An error occurs if an application uses either operator and the server does not support it. (WL #12767)

Bugs Fixed

- With MySQL Shell in Python mode, using auto-completion on a native MySQL Shell object caused informational messages about unknown attributes to be written to the application log file. (Bug #29907200)
- The execution time for statements issued in MySQL Shell in multiple-line mode has been reduced by reparsing the code only after the delimiter is found. (Bug #29864587)
- Python's `sys.argv` array was only initialized when MySQL Shell was started in batch mode, and was not initialized when MySQL Shell was started in interactive mode. (Bug #29811021)
- MySQL Shell incorrectly encoded the `CAST` operation as a function call rather than a binary operator, resulting in SQL syntax errors. (Bug #29807711)

- MySQL Shell now supports the unquoting extraction operator `->>` for JSON. (Bug #29794340)
- Handling of empty lines in scripts processed by MySQL Shell in batch mode has been improved. (Bug #29771369)
- On Windows, when a MySQL Shell report was displayed using the `\watch` command, pressing **Ctrl+C** to interrupt execution of the command did not take effect until the end of the refresh interval specified with the command. The interrupt now takes effect immediately. Also, any queries executed by reports run using the `\show` or `\watch` commands are now automatically canceled when **Ctrl+C** is pressed. (Bug #29707077)
- In Python mode, native dictionary objects created by MySQL Shell did not validate whether they contained a requested key, which could result in random values being returned or in a `SystemError` exception being thrown. Key validation has now been added, and a `KeyError` exception is thrown if an invalid key is requested. (Bug #29702627)
- When using MySQL Shell in interactive mode, if raw JSON output was being displayed from a source other than a terminal (for example a file or a pipe), in some circumstances the prompt was shown on the same line as the last line of the output. The issue has now been corrected, and a new line is printed before the prompt message if the last line of the output did not end with one. (Bug #29699640)
- The MySQL Shell `\sql` command, which executes a single SQL statement while another language is active, now supports the `\G` statement delimiter to print result sets vertically. (Bug #29693853)
- Some inconsistencies in MySQL Shell's choice of `stdout` or `stderr` for output have been corrected, so that only expected output that is intended to be processed by other programs goes to `stdout`, and all informational messages, warnings, and errors go to `stderr`. (Bug #29688637)
- When MySQL Shell was started with the option `--quiet-start=2` to print only error messages, warning messages about the operation of the upgrade checker utility `checkForServerUpgrade()` were still printed. (Bug #29620947)
- In Python mode, native dictionary objects created by MySQL Shell did not provide an iterator, so it was not possible to iterate over them or use them with the `in` keyword. Functionality to provide Python's iterator has now been added. (Bug #29599261)
- When a MySQL Shell report was displayed using the `\watch` command, the screen was cleared before the report was rerun. With a report that executed a slow query, this resulted in a blank screen being displayed for noticeable periods of time. The screen is now cleared just before the report generates its first text output. (Bug #29593246)
- MySQL Shell's upgrade checker utility `checkForServerUpgrade()` returned incorrect error text for each removed system variable that was detected in the configuration file. (Bug #29508599)
- MySQL Shell would hang when attempting to handle output from a stored procedure that produced results repeatedly from a single statement. The issues have now been corrected. (Bug #29451154, Bug #94577, Bug #28880081, Bug #93070)
- You can now specify the command line option `--json` to activate JSON wrapping when you start MySQL Shell to use the upgrade checker utility. In this case, JSON output is returned as the default, and you can choose raw JSON format by specifying `--json=raw`. Also, warning and error messages relating to running the utility have been removed from the JSON output. (Bug #29416162)
- In SQL mode, when MySQL Shell was configured to use an external pager tool to display output, the pager was invoked whether or not the query result was valid. For an invalid query, this resulted in the pager displaying an empty page, and the error message was only visible after quitting the pager. The pager tool is now only invoked when a query returns a valid result, otherwise the error message is displayed. (Bug #29408598, Bug #94393)

- MySQL Shell did not take the `ANSI_QUOTES` SQL mode into account when parsing quote characters. (Bug #27959072)
- Prompt theme files for MySQL Shell that were created on Windows could not be used on other platforms. The issue, which was caused by the parser handling the carriage return character incorrectly, has now been fixed. (Bug #26597468)
- The use of the `mysqlsh` command-line option `--execute` (`-e`) followed by `--file` (`-f`) when starting MySQL Shell is now disallowed, as these options are mutually exclusive. If the options are specified in that order, an error is returned. Note that if `--file` is specified first, `--execute` is treated as an argument of the processed file, so no error is returned. (Bug #25686324)
- Syntax errors returned by MySQL Shell's JavaScript expression parser have been improved to provide context and clarify the position of the error. (Bug #24916806)

Changes in MySQL Shell 8.0.16 (2019-04-25, General Availability)

- AdminAPI Added or Changed Functionality
- AdminAPI Bugs Fixed
- Functionality Added or Changed
- Bugs Fixed

AdminAPI Added or Changed Functionality

- The new `autoRejoinTries` option enables you to configure how many times an instance tries to rejoin a group after being expelled. In scenarios where network glitches happen but recover quickly, setting this option prevents you from having to manually add the expelled instance back to the group. The `autoRejoinTries` option accepts positive integer values between 0 and 2016 and the default value is 0, which means that instances do not try to automatically rejoin. Set the value to a valid integer to configure the number of attempts expelled instances should make to rejoin the group. You can pass the `autoRejoinTries` option to these AdminAPI operations:

  - `dba.createCluster()`
  - *`Cluster`*`.addInstance()`
  - *`Cluster`*`.setOption()`
  - *`Cluster`*`.setInstanceOption()`

  When you configure the `autoRejoinTries` option, it sets the `group_replication_autorejoin_tries` system variable. Passing the option to `dba.createCluster()`, *`Cluster`*`.addInstance()` or *`Cluster`*`.setInstanceOption()` configures the automatic rejoin for specific cluster instances. Passing the option to *`Cluster`*`.setOption()` configures the automatic rejoin for all cluster instances.

  For more information, see Responses to Failure Detection and Network Partitioning. (WL #12066)

- AdminAPI now reports information about the version of MySQL running on instances. This information is available from the following operations:

  - *`Cluster`*`.status()`
  - *`Cluster`*`.describe()`

- *Cluster*.rescan()

See Checking the MySQL Version on Instances for more information. (WL #12881)

- MySQL InnoDB cluster automatically and transparently manages the communication protocol versions of its members, whenever the cluster topology is changed using AdminAPI operations. An InnoDB cluster always uses the most recent communication protocol version that is supported by all instances that are part of the cluster or joining it.

  - When an instance is added to, removed from, or rejoins the cluster, or a rescan or reboot operation is carried out on the cluster, the communication protocol version is automatically set to a version supported by the instance that is now at the earliest MySQL Server version.

  - When you carry out a rolling upgrade by removing instances from the cluster, upgrading them, and adding them back into the cluster, the communication protocol version is automatically upgraded when the last remaining instance at the old MySQL Server version is removed from the cluster prior to its upgrade.

  To see the communication protocol version in use in an InnoDB cluster, use the *Cluster*.status() function with the 'extended' option enabled. The communication protocol version is returned in the 'GRProtocolVersion' field, provided that the cluster has quorum and no cluster members are unreachable. (WL #12765)

AdminAPI Bugs Fixed

- Removing an instance from a cluster when the instance to be removed had no user defined for the group_replication_recovery channel resulted in dropping users on the remaining instances of the cluster. (Bug #29617572)

- The failoverConsistency option has been deprecated and a new option named consistency has been added, to make it more consistent with the target Group Replication group_replication_consistency system variable name. The MySQL Shell online documentation now also correctly describes all of the values you can assign to the consistency option. (Bug #29356599)

- The dba.configureLocalInstance() operation would remove any section that did not start with mysqld from the provided option file. This could remove sections such as the client section from the option file. (Bug #29349014)

- When an instance with X Plugin disabled was added to an InnoDB Cluster, if the instance was later removed from the cluster using *Cluster*.removeInstance() the operation failed with LogicError "get_string(7): field is NULL". This was a regression introduced by the fix for Bug#27677227. (Bug #29304183)

- There was an inconsistency between the behavior of dba.checkInstanceConfiguration() and the commands to add instances to the cluster (dba.createCluster() and *Cluster*.addInstance()) regarding the localhost and loopback address validation. In particular, a simple error was printed by dba.checkInstanceConfiguration() but the execution of the operation continued showing that everything was correct at the end of the operation, while an error was issued and the execution stopped for dba.createCluster() and Cluster.addInstance().

  As part of fixing this issue, it was decided that the existing localhost and loopback address validations are no longer needed and should be removed. In particular, whatever address is specified for report_host, even if it is localhost or the loopback address (127.0.0.1), should be allowed, because it was explicitly specified by the user to use it. (Bug #29279941)

- The `dba.rebootClusterFromCompleteOutage()` operation was not preserving the existing Group Replication configurations previously set for the instances. In particular, the Group Replication local address and exit state action values were being changed. Now all settings are read at the time of rebooting the cluster. (Bug #29265869)
- Using either *`Cluster`*`.setOption()` or *`Cluster`*`.setInstanceOption()` to set an option which only exists in MySQL 8.0 on an instance running MySQL 5.7 was not being caught correctly. (Bug #29246657)
- On Debian-based platforms (such as Ubuntu), if the hostname resolved to 127.0.1.1 - which is the default on these platforms - it was not possible to create a cluster using the default settings. Now, in such situations a proper validation of the instance is performed before creating a cluster and adding instances to it. (Bug #29246110)
- The `dba.checkInstanceConfiguration()` operation did not validate host restrictions for the account provided for cluster administration, for example if the account could actually connect to all of the instances in the cluster. In particular, now an error is issued if the provided user account is only able to connect through localhost. (Bug #29018457)
- InnoDB Cluster configured `auto_increment_increment` and `auto_increment_offset` on instances for clusters running in multi-primary mode and consisting of up to 7 instances based on the logic described at InnoDB Cluster and Auto-increment. But Group Replication permits groups to contain up to 9 members, and *`Cluster`*`.addInstance()` and *`Cluster`*`.removeInstance()` were not following the logic used for other operations. Now, InnoDB Cluster uses the same logic for auto increment regardless of the operation used and correctly handles multi-primary clusters with more than 7 instances. (Bug #28812763)
- MySQL Shell uses the host value of the provided connection parameters as the target hostname used for AdminAPI operations, namely to register the instance in the metadata (for the `dba.createCluster()` and `cluster.addInstance()` operations). However, the host used for the connection parameters might not match the hostname that is used or reported by Group Replication, which uses the value of the `report_host` system variable when it is defined (in other words it is not `NULL`), otherwise the value of `hostname` is used. Therefore, AdminAPI now follows the same logic to register the target instance in the metadata and as the default value for the `group_replication_local_address` variable on instances, instead of using the host value from the instance connection parameters. During this fix it was detected that when the `report_host` variable was set to empty, Group Replication uses an empty value for the host but AdminAPI (for example in commands such as `dba.checkInstanceConfiguration()`, `dba.configureInstance()`, `dba.createCluster()`) reports the hostname as the value used which is inconsistent with the value reported by Group Replication. An error is now issued by AdminAPI if an empty value is set for the `report_host` system variable. (Bug #28285389)
- In the event that `dba.createCluster()` failed and a rollback was performed to remove the created replication (recovery) users, the account created at `localhost` and any of the `ipWhitelist` addresses were not being removed. The fix ensures that the replication accounts are removed whenever a rollback related to `dba.createCluster()` is performed. This work was based on a code contribution from Bin Hong. (Bug #94182, Bug #29308037)

Functionality Added or Changed

- **Important Change:** Attempting to connect to an X Protocol port, 33060 by default, using the classic MySQL protocol resulted in the following error: `ERROR 2013 (HY000): Lost connection to MySQL server at 'reading initial communication packet', system error: 0`

  This was because of differences in X Protocol and classic MySQL protocol clients expectations on how connections were initialized. Now, in such a situation the generated error message is `ERROR 2007`

`(HY000): Protocol mismatch; server version = 11, client version = 10`. If you encounter this error then you are probably trying to use the wrong port for the protocol your client is using.

As part of this improvement the `mysqlx_enable_hello_notice` system variable has been added, which controls messages sent to classic MySQL protocol clients that try to connect over X Protocol. When enabled, clients which do not support X Protocol that attempt to connect to the server X Protocol port receive an error explaining they are using the wrong protocol. Set `mysqlx_enable_hello_notice` to false to permit clients which do not recognize the hello message to still connect. (WL #12240)

- MySQL Shell's upgrade checker utility can now check the configuration file (`my.cnf` or `my.ini`) for the server instance. The utility checks for any system variables that are defined in the configuration file but have been removed in the target MySQL Server release, and also for any system variables that are not defined in the configuration file and will have a different default value in the target MySQL Server release. For these checks, when you invoke `checkForServerUpgrade()`, you must provide the file path to the configuration file. If you omit the file path and the upgrade checker utility needs to run a check that requires the configuration file, that check fails with a message informing you that you must specify the file path. (Bug #27801824, Bug #29222179, WL #12760)

- MySQL Shell now has a framework and commands that you can use to set up and run reports to display live information from a MySQL server, such as status and performance information. Reports can be run once using the MySQL Shell `\show` command, or run then refreshed continuously in a MySQL Shell session using the `\watch` command. They can also be accessed as API functions in the `shell.reports` object.

  The reporting facility supports both built-in reports and user-defined reports. User-defined reports can be created in the supported scripting languages JavaScript and Python, and can be run in any MySQL Shell mode (JavaScript, Python, or SQL), regardless of the language that the report was written in. Reports can be saved in a folder in the MySQL Shell configuration path and automatically loaded at startup. You can also create a report directly in the MySQL Shell prompt. You register a report to MySQL Shell using the `shell.registerReport` method to provide information about the report and the options and arguments that it supports.

  For more information, see Reporting with MySQL Shell. (WL #11263)

- When running MySQL Shell in interactive mode, you can now execute an SQL statement without switching to SQL mode and back again afterwards. This function enables you to conveniently issue some SQL statements in the context of a longer AdminAPI workflow in JavaScript or Python mode. Use the `\sql` command immediately followed by the SQL statement, for example:

  ```
  \sql select * from sakila.actor limit 3;
  ```

  The SQL statement does not need any additional quoting, and the statement delimiter is optional. With this format, MySQL Shell does not switch mode as it would if you entered the `\sql` command. After the SQL statement has been executed, MySQL Shell remains in JavaScript or Python mode.

  You cannot use multiple line mode when you use the `\sql` command with a query to execute single SQL statements while another language is active. The command only accepts a single SQL query on a single line. (WL #11155)

- MySQL Shell history is now split per active language which the command was issued under. This means that your history now matches the active language, for example when you are running in JavaScript mode having issued `\js`, the history contains the previous JavaScript statements you issued, and when you issue `\sql` to change to SQL mode your history contains the previous SQL statements you issued. Similarly, now any history related commands such as `\history clear` or `\history delete` are

performed on the history of the current active language. When you install this version, any existing MySQL Shell history files are duplicated to ensure that existing history is not lost. Subsequent operations are then added to the language specific history file. (WL #12761)

- When `resultFormat` was set to `json` or `json/raw`, every result was being returned as a JSON document. This behavior was expected when JSON wrapping is off (in other words the `--json` command option was not used when starting MySQL Shell). Now, for consistency reasons when JSON wrapping is off and `resultFormat` is set to `json` or `json/raw`, every record is printed in a separate document and statistics and warnings are printed in plain text.

  For example if MySQL Shell is started without `--json` and `resultFormat=json/raw`:

```
mysqlsh-sql> SHOW DATABASES;
{"Database":"information_schema"}
{"Database":"mysql"}
{"Database":"performance_schema"}
{"Database":"sys"}
4 rows in set (0.0035 sec)
```

  If MySQL Shell is started with `--json` and with `resultFormat=json/raw`:

```
mysqlsh-sql> SHOW DATABASES;
{
    "hasData": true,
    "rows": [
        {
            "Database": "information_schema"
        },
        {
            "Database": "mysql"
        },
        {
            "Database": "performance_schema"
        },
        {
            "Database": "sys"
        }
    ],
    "executionTime": "0.0018 sec",
    "affectedRowCount": 0,
    "affectedItemsCount": 0,
    "warningCount": 0,
    "warningsCount": 0,
    "warnings": [],
    "info": "",
    "autoIncrementValue": 0
}
```

  (WL #12764)

Bugs Fixed

- MySQL Shell could be installed in an environment where Python was not present, but the application has a dependency on many standard Python modules, resulting in error messages at startup. The RPM and Debian packages for MySQL Shell now explicitly specify the dependency on Python. (Bug #29469201)

- The MSI file that is used by Windows Installer to install MySQL Shell now adds the path to the application binary (`mysqlsh`) to the Windows `PATH` environment variable, so that the application can be started from a command prompt. (Bug #29457639)

- In the instructions to build MySQL Shell from source (the `INSTALL` document), the required version of the optional V8 dependency has been updated from 3.28.71.19 to 6.7.288.46. (Bug #29430049, Bug #94529)
- MySQL Shell's upgrade checker utility `checkForServerUpgrade()` could incorrectly report a schema inconsistency error for a table whose name included a special character such as a hyphen. (Bug #29346836, Bug #94303)
- On Windows, MySQL Shell's upgrade checker utility `checkForServerUpgrade()` incorrectly reported a schema inconsistency error for partitioned tables. (Bug #29256562)
- MySQL Shell stopped unexpectedly if Python code was running in interactive mode and threw exceptions from C++ libraries. These exceptions are now caught and translated to Python's built-in RuntimeError exceptions. (Bug #29057116)
- When a connection is specified using key-value pairs in MySQL Shell's `shell.connect()` method, the host name cannot be an empty string. MySQL Shell now handles this situation consistently and returns an error if the supplied host name is an empty string. (Bug #28899522)
- MySQL Shell's JSON import utility can now accept input from FIFO special files (named pipes) when you invoke the utility using the `util.importJSON` function, so you can carry out large imports by this method without needing to put the data into a file. (Bug #28785527)
- When you use the MySQL Shell command `\help` (or `\h`, or `\?`) with a search pattern to search for help on a specific subject, multiple help topic titles can match the pattern and be returned as a list, to be selected by entering the command again with an extended search pattern. With this system, it was possible for help topics with a single-word title to be inaccessible from such a list because there was nothing further to add to the search pattern. To avoid this situation, the handling of multiple matches has now been improved. If a topic title is found that matches the given search pattern exactly (case-sensitive in the event of multiple topic matches, and case-insensitive in the event of no case-sensitive matches), the topic is identified as the exact match and its help data is printed. The rest of the topics with pattern matches in their titles are listed in a “see also” section and can be selected by further pattern matching. (Bug #28393119)

Changes in MySQL Shell 8.0.15 (2019-02-01, General Availability)

This release contains no functional changes and is published to align version number with the MySQL Server 8.0.15 release.

Changes in MySQL Shell 8.0.14 (2019-01-21, General Availability)

- AdminAPI Added or Changed Functionality
- AdminAPI Bugs Fixed
- Functionality Added or Changed
- Bugs Fixed

AdminAPI Added or Changed Functionality

- The *`Cluster`*`.status()` operation has been extended to enable you to display information about the underlying Group Replication group used by the cluster. Now you can retrieve information from all members of a cluster without having to connect to each member individually.

  To see information about the `groupName` and `memberId`; and general statistics about the number of transactions checked, proposed, and rejected by members issue:

```
Cluster.status({extended:true})
```

To see information about recovery and regular transaction I/O, applier worker thread statistics and any lags; applier coordinator statistics, if parallel apply is enabled; error, and other information from I/O and applier threads issue:

```
Cluster.status({queryMembers:true})
```

In addition, in previous versions the URI-type string shown for `groupInformationSourceMember` in the output of *`Cluster`*`.status()` could be the cluster's MySQL Router address, rather than the address of the instance which provided the displayed group information. This has been improved to ensure `groupInformationSourceMember` always shows the correct `hostname`, or `report_host`, value and `port`, or `report_port`, value of the instance which provided the group information. (Bug #28636963, Bug #26519466, Bug #27824265, Bug #28366027, WL #11997)

- AdminAPI no longer relies on the mysqlprovision check command. This work has resulted in the following:

  - The `errors` field in the JSON returned by `dba.checkInstanceConfiguration()` has been removed, because it was only used to hold errors issued by mysqlprovision. Any errors are now reported directly, for example as RuntimeError.

  - The `dba.verbose` value no longer influences the amount of debug information displayed for `dba.checkInstanceConfiguration()`, `dba.configureInstance()`, and `dba.configureLocalInstance()` because it was only used to control the verbosity of the information displayed from mysqlprovision. Instead, the generic verbose value from MySQL Shell is used to control the verbosity level for those functions.

  - In addition, the messages returned have been generally improved to make them more accurate.

  (WL #12006)

  References: See also: Bug #28737777, Bug #27305806, Bug #28768627, Bug #27702439, Bug #28733883.

- When you create a cluster, you can set the timeout before instances are expelled from the cluster, for example when they become unreachable. Pass the new `expelTimeout` option to the `dba.createCluster()` operation, which configures the `group_replication_member_expel_timeout` variable on the seed instance. All instances running MySQL server 8.0.13 and later which are added to the cluster are automatically configured to have the same `group_replication_member_expel_timeout` value as configured on the seed instance. (WL #12050)

- You can now configure an InnoDB Cluster's mode while the cluster remains online. This enables you to configure the underlying Group Replication group to choose a specific instance as the new primary in single-primary mode, or to change between multi-primary and single-primary modes without taking the cluster offline. This uses the group coordinator and the functions added in WL#10378, see Configuring an Online Group. Use the following operations:

  - *`Cluster`*`.setPrimaryInstance(`*`instance`*`)`, which forces the election of *`instance`* as the new primary by overriding any election process.

  - *`Cluster`*`.switchToMultiPrimaryMode()`, which switches the cluster to multi-primary mode. All instances become primaries.

  - *Cluster*.switchToSinglePrimaryMode([*instance*]), which switches the cluster to single-primary mode. If *instance* is specified, it becomes the primary and all the other instances become secondaries. If *instance* is not specified, the new primary is the instance with the highest member weight (and the lowest UUID in case of a tie on member weight).

  (WL #12052)

- You can now check and modify the settings in place for an InnoDB Cluster while the instances are online. To check the current settings of a cluster, use the following operation:

  - *Cluster*.options(), which lists the cluster configuration options for its ReplicaSets and instances. A boolean option all can also be specified to include information about all Group Replication system variables in the output.

  This work also enables you to configure the InnoDB Cluster options at a cluster level or instance level, while instances remain online. This avoids the need to remove, reconfigure and then again add the instance to change InnoDB Cluster options. Use the following operations:

  - *Cluster*.setOption(option, value) to change the settings of all cluster instances globally
  - *Cluster*.setInstanceOption(instance, option, value) to change the settings of individual cluster instances

  The way which you use InnoDB Cluster options with the operations listed depends on whether the option can be changed to be the same on all instances or not. These options are changeable at both the cluster (all instances) and per instance level:

  - exitStateAction
  - memberWeight

  This option is changeable at the per instance level only:

  - label

  These options are changeable at the cluster level only:

  - failoverConsistency
  - expelTimeout
  - clusterName

  (WL #11465)

- The *Cluster*.rescan() operation has been extended to enable you to detect changes to the cluster's topology, and modify the cluster metadata, for example to remove old instance data. Now you can:

  - use the updateTopologyMode option to detect if the Group Replication mode (single-primary or multi-primary mode) registered in the metadata matches the current mode of the cluster, updating that information in the metadata if requested through a new option or by a prompt confirmation. You can use this option to update the metadata after using the *Cluster*.switchToSinglePrimaryMode([*instance*]) and *Cluster*.switchToMultiPrimaryMode() options added in WL#12052.

  - use the `addInstances` option to specify a list of new instances to add to the metadata, or the `removeInstances` option to specify a list of obsolete instances to remove from the metadata. Pass the `auto` value to these options to automatically add or remove instances from the metadata, without having to specify an explicit list of instances. This enables the function to update the metadata even in noninteractive mode, making it consistent with the other AdminAPI operations.

  - In addition, a new interactive option has been added to the *`cluster`*`.rescan()` operation, to enable or disable interactive mode prompts specifically for the *`cluster`*`.rescan()` command.

  (WL #10644)

  References: See also: Bug #28997465, Bug #28529362, Bug #28889563, Bug #25675665, Bug #28542904.

- In 8.0.14, Group Replication introduces the ability to specify the failover guarantees (eventual or “read your writes”) if a primary failover happens in single-primary mode (see WL#11123). Configure the failover guarantees of an InnoDB Cluster at creation by passing the new `failoverConsistency` option to the `dba.createCluster()` operation, which configures the `group_replication_consistency` system variable on the seed instance. This option defines the behavior of a new fencing mechanism used when a new primary is elected in a single-primary group. The fencing restricts connections from writing and reading from the new primary until it has applied any pending backlog of changes that came from the old primary (sometimes referred to as “read your writes”). While the fencing mechanism is in place, applications effectively do not see time going backward for a short period of time while any backlog is applied. This ensures that applications do not read stale information from the newly elected primary.

  The `failoverConsistency` option is only supported if the target MySQL server version is 8.0.14 or later, and instances added to a cluster which has been configured with the `failoverConsistency` option are automatically configured to have `group_replication_consistency` the same on all cluster members that have support for the option. The variable default value is controlled by Group Replication and is `EVENTUAL`, change the `failoverConsistency` option to `BEFORE_ON_PRIMARY_FAILOVER` to enable the fencing mechanism. Alternatively use `failoverConsistency=0` for `EVENTUAL` and `failoverConsistency=1` for `BEFORE_ON_PRIMARY_FAILOVER`.

  

  Note

  Using the `failoverConsistency` option on a multi-primary InnoDB Cluster has no effect but is allowed because the cluster can later be changed into single-primary mode with the *`Cluster`*`.switchToSinglePrimaryMode()` operation.

  (WL #12067)

AdminAPI Bugs Fixed

- The default value for `group_replication_exit_state_action` is `ABORT_SERVER`, but AdminAPI now overrides this and sets the default on instances to `READ_ONLY`. This ensures that instances which leave the group unexpectedly continue running and can be rejoined to the cluster. (Bug #28701263)

- When a cluster was created on a server that did not have the X Plugin enabled, a silent assumption was being made about the X Protocol port value. Now the value of an X Protocol port is only stored for instances on which X Plugin is enabled. (Bug #27677227)

- The `dba.checkInstanceConfiguration()` operation did not check if the Performance Schema was enabled on the target instance. This could result in a situation where you could create a cluster but

could not run several management operations on it, for example the `Cluster.status()` operation. Now, `dba.checkInstanceConfiguration()` checks that the Performance Schema is enabled on instances. (Bug #25867733)

- When `Cluster.checkInstanceState()` was executed on an instance which was already a member of the current cluster, the output indicated that the instance was fully recoverable. This was misleading and was caused by a missing validation to ensure the instance does not belong to a cluster. (Bug #24942875)

- The `dba.checkInstanceConfiguration()` operation did not recognize privileges when they were associated to a user through a role (available in MySQL server 8.0 and higher). In such a case, a missing privileges error was being incorrectly issued despite the user possessing all the required privileges. Now users with their privileges assigned by roles are recognized by AdminAPI operations correctly. (Bug #91394, Bug #28236922)

Functionality Added or Changed

- **X DevAPI:** The `Table` and `Collection` objects now support the `.count()` method, part of the X DevAPI. (WL #12447)

- When started from the command line, MySQL Shell prints information about the product, information about the session (such as the default schema and connection ID), warning messages, and any errors that are returned during startup and connection. You can now suppress printing of information that you do not need by using the `--quiet-start[=1|2]` `mysqlsh` command-line option. With a value of 1 (the default when the option is specified), information about the MySQL Shell product is not printed, but session information, warnings, and errors are printed. With a value of 2, only errors are printed.

  As part of this work, the printed information was tidied up so that the information about the MySQL Shell product is printed before the information about the session. Also, the handling of error printing was normalized to send diagnostic data to `stderr`, and errors to `stdout`. (Bug #28833718, Bug #28855291, WL #12631)

- MySQL Shell connections using classic MySQL protocol now support compression for information sent between the client and the server. You can specify compression when you start MySQL Shell and connect using command line options, or in a URI string or a key-value pair when you create a session using other interfaces. You can also use the MySQL Shell configuration option `defaultCompress` to enable compression for every global session.

  For MySQL Shell connections that use Unix socket files, the `--socket` command line option can now be specified with no argument to connect using the default Unix socket file for the protocol. (Bug #28730149, WL #12537)

- The MySQL Shell JSON import utility can now process BSON (binary JSON) data types that are represented in JSON documents. The data types used in BSON documents are not all natively supported by JSON, but can be represented using extensions to the JSON format. The import utility can process documents that use JSON extensions to represent BSON data types, convert them to an identical or compatible MySQL representation, and import the data value using that representation. The resulting converted data values can be used in expressions and indexes, and manipulated by SQL statements and X DevAPI functions.

  To convert JSON extensions for BSON types into MySQL types in this way, you must specify the `convertBsonTypes` option when you run the import utility. Additional options are available to control the mapping and conversion for specific BSON data types. If you import documents with JSON extensions for BSON types and do not use this option, the documents are imported in the same way as they are represented in the input file. (WL #12134)

- A MySQL Shell configuration option `showColumnTypeInfo` and command line option `--column-type-info` have been added to display metadata for each column in a returned result set, such as the column type and collation. The metadata is printed before the result set, and is only shown in SQL mode.

  In the metadata, the column type is returned as both the type used by MySQL Shell (`Type`), and the type used by the original database (`DBType`). For MySQL Shell connections using classic MySQL protocol, `DBType` is as returned by the protocol, and for X Protocol connections, `DBType` is inferred from the available information. The column length (`Length`) is returned in bytes. (WL #12426)

- The upgrade checker utility provided by MySQL Shell, which is the `checkForServerUpgrade()` function of the `util` global object, has several enhancements:

  - The utility can now select and provide advice and instructions for relevant checks that cannot be automated, and must be performed manually. The manual checks are rated as either warning or notice (informational) level, and are listed after the automated checks. In MySQL Shell 8.0.14, the utility provides advice where relevant about the change of default authentication plugin in MySQL 8.0.

  - A check has been added for the removed `log_syslog_*` system variables that previously configured error logging to the system log (the Event Log on Windows, and `syslog` on Unix and Unix-like systems).

  - A check has been added for specific schema inconsistencies that can be caused by the deletion or corruption of a file, including the removal of the directory for a schema and the removal of a `.frm` file for a table.

  You can access the upgrade checker utility from within MySQL Shell or start it from the command line. For instructions and further information, see MySQL Shell Utilities. (WL #12506)

- MySQL Shell can print results in table, tabbed, or vertical format, or as pretty or raw JSON output. From MySQL Shell 8.0.14, the new MySQL Shell configuration option `resultFormat` can be used to specify any of these output formats as a persistent default for all sessions, or just for the current session. Changing this option takes effect immediately. Alternatively, the new command line option `--result-format` can be used at startup to specify the output format for a session. The existing command line options `--table`, `--tabbed`, and `--vertical` are now aliases for the `--result-format` option given with the corresponding value.

  The existing command line option `--json` controls JSON wrapping for all MySQL Shell output from a session. Specifying `--json` or `--json=pretty` turns on JSON wrapping and generates pretty-printed JSON. Specifying `--json=raw` turns on JSON wrapping and generates raw JSON. With any of these options, the value of the `resultFormat` MySQL Shell configuration option is ignored. Specifying `--json=off` or not specifying the `--json` option turns off JSON wrapping, and result sets are output as normal in the format specified by the `resultFormat` configuration option.

  The `outputFormat` MySQL Shell configuration option is now deprecated. This option combined the JSON wrapping and result printing functions, which have now been separated. If this option is still specified in your MySQL Shell configuration file or scripts, the behavior is as follows:

  - With the `json` or `json/raw` value, `outputFormat` activates JSON wrapping with pretty or raw JSON respectively.

  - With the `table`, `tabbed`, or `vertical` value, `outputFormat` turns off JSON wrapping and sets the `resultFormat` MySQL Shell configuration option for the session to the appropriate value.

  (WL #12141)

- The V8 library used by MySQL Shell has been updated to version 6.7.288.46. (WL #12264)

Bugs Fixed

- The TAR build of MySQL Shell comes with Python 2.7. When attempting to include the site package, an error was emitted because of missing build files needed by the include. (Bug #28973138)
- Handling procedures for user-supplied data in MySQL Shell were refactored to ensure correct cleanup after use. (Bug #28915716)
- The exception type and error messages returned by MySQL Shell functions for parameter errors have been standardized across the different functions. (Bug #28838958)
- MySQL Shell stopped unexpectedly if the `shell.setCurrentSchema()` method was called to set the default schema before an active session had been established. MySQL Shell now validates that there is an active session when the operation takes place. (Bug #28814112)
- The MySQL Shell JSON import utility no longer requires an empty dictionary to be supplied if there are no import options. (Bug #28768585)
- In SQL mode, MySQL Shell does not add statements to the history if they include the strings `IDENTIFIED` or `PASSWORD`, or other strings that you configure using the `--histignore` command option or `shell.options["history.sql.ignorePattern"]`. However, this previously meant that filtered-out statements were not available to be corrected immediately after entry, and had to be re-typed in case of any errors. MySQL Shell now always makes the last executed statement available to be recalled by pressing the Up arrow, regardless of the filters set in the history ignore list. If filtering applies to the last executed statement, it is removed from the history as soon as another statement is entered, or if you exit MySQL Shell immediately after executing the statement. (Bug #28749037)
- The result printing logic in MySQL Shell has been refactored to use back-end rather than high-level result data, delivering performance improvements for all types of result data and more accurate representation for JSON data. (Bug #28710831)
- A memory leak was fixed that occurred when the new MySQL Shell command-line syntax was used. (Bug #28705373)
- The check for partitioned tables in shared tablespaces in the upgrade checker utility provided by MySQL Shell (the `util.checkForServerUpgrade()` operation) did not return correct results for the 8.0.11 and 8.0.12 target versions. The check now uses alternative Information Schema tables that are populated with the required information in these versions. (Bug #28701423)
- The MySQL Shell command `\option` ignored additional arguments separated by spaces that were specified for an option after the initial value. (Bug #28658632)
- MySQL Shell permitted newline characters (line feed and carriage return) in passwords to be passed to a Secret Store Helper using the `shell.storeCredential` method, resulting in an error in the Secret Store Helper. MySQL Shell now returns an exception if newline characters are used in supplied passwords for the `shell.storeCredential` method, and does not pass them to the Secret Store Helper. (Bug #28597766)
- On the Windows platform, UTF-8 encoded strings were printed to the console using the `cout` object, which transfers a byte at a time. This resulted in multi-byte Unicode characters, such as a single quotation mark, being displayed and handled incorrectly. MySQL Shell now uses alternative functions for printing, and verifies that multi-byte UTF-8 characters are emitted as a complete unit. (Bug #28596692)
- When executing an SQL script in MySQL Shell, an inaccurate line number was reported for the location of syntax errors in the script. The number referenced the current SQL block rather than the line number in the script. The error message now uses the global line number. (Bug #28545982)

- The SQL statement splitting logic in MySQL Shell has been refactored to fix a number of issues and to match behaviors of the MySQL command-line tool `mysql`:

  - The backslash character (`\`) is no longer accepted in the delimiter string.
  - The use of the word “delimiter” in contexts other than as a command is now handled correctly.
  - In scripts, comments are not discarded, and groups of comments and statements are now split in the same way as `mysql` would split them.
  - Large scripts can now be successfully split into incremental chunks even when some tokens span across more than one chunk.
  - Scripts can now be parsed in the `ANSI_QUOTES` SQL mode.
  - Multi-line strings and comments that contain quotes are now parsed correctly.
  - Inline commands are handled in the same way as by `mysql`, as follows:
    - A `\` character appearing at the beginning of a statement is interpreted as the start of a multi-letter MySQL Shell command.
    - A `\` character appearing within a statement is interpreted as the start of a single-letter command. The command is executed immediately, then stripped out of the input statement.
    - A `\` character appearing after the end of a statement is interpreted as the start of a single-letter command.

  (Bug #27959016, Bug #25689071)

- The handling of Windows named pipe connections by MySQL Shell has been improved and systematized. Now, if you specify the host name as a period (.) on Windows, MySQL Shell connects using a named pipe.

  - If you are connecting using a URI type string, specify `user@.`
  - If you are connecting using a data dictionary, specify `{"host": "."}`
  - If you are connecting using individual parameters, specify `--host=.` or `-h .`

  By default, the pipe name `MySQL` is used. You can specify an alternative named pipe using the `--socket` option or as part of the URI type string. If a URI type string is used, the named pipe must be prepended with the characters `\\.\` as well as being either encoded using percent encoding or surrounded with parentheses, as shown in the following examples:

  ```
  (\\.\named:pipe)
  \\.\named%3Apipe
  ```

  (Bug #27381738)

- When JSON format output was enabled for MySQL Shell, the properties of the Shell API Options class (`shell.options`) and AdminAPI Cluster class (`dba.getCluster`) were not printed, only the class name. (Bug #25027181)

Changes in MySQL Shell 8.0.13 (2018-10-22, General Availability)

- Deprecation and Removal Notes

- AdminAPI Added or Changed Functionality
- AdminAPI Bugs Fixed
- Functionality Added or Changed
- Bugs Fixed

Deprecation and Removal Notes

- If you do not specify a protocol with the `\connect` command or when starting MySQL Shell, MySQL Shell automatically attempts to use X Protocol for the session's connection, and falls back to classic MySQL protocol if X Protocol is unavailable. The connection type option `-ma`, which specified that behavior explicitly, is now deprecated.

  The use of a single dash with the connection type options `-mx` and `-mc`, for an X Protocol and classic MySQL protocol connection respectively, is also deprecated. These options must now be specified with a double dash (that is, `--mx` and `--mc`) with the `\connect` command or when starting MySQL Shell. They are now defined as aliases of the long form `--mysql` (`--mc`) and `--mysqlx` (`--mx`) connection type options. (Bug #27363459)

- The mysqlsh command-line options `--dbpassword[=`*`password`*`]` and `--dbuser=`*`user_name`* are now deprecated. Use the options `--password` (`-p`) and `--user` (`-u`) instead. (Bug #26049681)

AdminAPI Added or Changed Functionality

- The behavior of *`Cluster`*`.dissolve()` has been updated to make it more consistent with other AdminAPI commands. Now you do not have to pass in the `force` option to start the command, and there is an interactive prompt available. When all instances belonging to the cluster are online, if MySQL Shell is running in interactive mode then you are prompted to confirm the operation of dissolving the cluster. If MySQL Shell is running in noninteractive mode, when all instances are reachable, or online, then the command removes the instances from the cluster. In the case that instances are not reachable an error is thrown. Pass in the `force` option to remove instances which are not reachable. (WL #11889)

  References: See also: Bug #27833605, Bug #27837231.

- A new optional parameter `exitStateAction` can be used with the *`dba`*`.createCluster()` and *`cluster`*`.addInstance()` commands, which enables you to configure the `group_replication_exit_state_action` variable of an InnoDB Cluster member. The `group_replication_exit_state_action` variable enables you to specify what action is taken if a member involuntarily leaves the group. When `group_replication_exit_state_action` is set to `ABORT_SERVER` (the default value), the instance shuts itself down, and when `group_replication_exit_state_action` is set to `READ_ONLY` the instance switches itself to super read only mode instead and goes into the Group Replication `ERROR` state. (WL #12049)

- The new optional `memberWeight` option can be used with the `dba.createCluster()` and *`Cluster`*`.addInstance()` functions to enable you to set the `group_replication_member_weight` system variable of an InnoDB Cluster server instances in a single-primary cluster. The default value is 50, in other words the system variable default. Set the `memberWeight` option to an integer between 0 and 100 to configure a member's weight in the failover election process. The value determines the chance of the instance being elected as the primary in the event of a failover. See Single-Primary Mode for more information. (WL #11032)

AdminAPI Bugs Fixed

- The `dba.deploySandboxInstance()` function in version 8.0.12 deploys the sandbox and includes `log_syslog=OFF` in the instance's configuration file. This variable was deprecated in MySQL 8.0.12

and was removed in MySQL 8.0.13. Now, the variable has been updated to include the `loose_` prefix which makes the server ignore it for a MySQL 8.0.13 sandbox, while maintaining compatibility with earlier version sandboxes. (Bug #28543536)

- Occasionally, when adding an instance to an existing cluster the instance got stuck in the distributed recovery phase resulting in an immutable reported status of `RECOVERING`. This issue was related to the automatically generated password for the internal replication users created by InnoDB Cluster. (Bug #28219398, Bug #91348)

- In the default interactive mode, whenever using the function `dba.rebootClusterFromCompleteOutage()` without any parameter, the function failed with an error specifying the cluster name does not exist. Now the default cluster is assumed when the function is issued without a parameter. (Bug #28207565)

- The handling of metadata server changes related to the *`Cluster`*`.addInstance()` has been improved, resulting in the following changes:

  - the correct session is now used for metadata and group operations

  - the *`Cluster`*`.addInstance()` operation aborts and recommends the use of *`Cluster`*`.rescan()` if the instance is in the group but not the InnoDB Cluster metadata

  - the unnecessary parameter `super_user_password` has been removed

  (Bug #28200661)

- The Windows scripts generated by `dba.deploySandboxInstance()` incorrectly displayed user output messages in quotes. Additionally, the scripts have been improved and they no longer display executed commands. (Bug #28199954)

- The `dba.createCluster()` AdminAPI operation always created replication users, even when the `adoptFromGR` option was used. However, when adopting an already existing Group Replication group no additional users need to be created. (Bug #28054500)

- Error messages issued by the AdminAPI relating to an invalid number of arguments for a function did not include the relevant object and method prior to the message text. These messages have now been standardized. (Bug #27832594)

- When using `dba.createCluster()` or *`Cluster`*`.addInstance()`, the AdminAPI was setting the values of `auto_increment_offset` and `auto_increment_increment` incorrectly. Now the variables are set according to the following logic:

  - for a cluster running in single-primary mode:

    - `auto_increment_offset=2`

    - `auto_increment_increment=1`

  - for a cluster running in multi-primary mode:

    - `auto_increment_offset=1` and `server_id` % 7

    - `auto_increment_increment=7`

  (Bug #27084767)

- AdminAPI was using the incorrect terms for Group Replication. Now clusters are described as single-primary and multi-primary, the `multiMasterp` option has been deprecated, and the `multiPrimary` option has been added. (Bug #25926603)

- The result of calling `dba.get_cluster().status()` when quorum was lost could not be converted to a JSON object, because the string representation of the resultant object contained escape sequences. This issue was not limited to the `Cluster.status()` method, but affected all arrays and dictionaries returned by the Shell API in Python mode. The internal representation of arrays and dictionaries has been fixed. (Bug #91304, Bug #28200499)

Functionality Added or Changed

- **X DevAPI:** The `connect-timeout` connection path parameter (see Connecting to the Server Using URI-Like Strings or Key-Value Pairs) has been added to the X DevAPI, which enables you to specify the number of seconds clients such as MySQL Shell wait until the client stops trying to connect to an unresponsive MySQL server. The value of `connect-timeout` must be a non-negative integer that defines a time frame in milliseconds. The timeout default value is 10000 milliseconds, or 10 seconds. For example:

  ```
  // Decrease the timeout to 2 seconds.
  mysqlx.getSession('user@example.com?connect-timeout=2000');

  // Increase the timeout to 20 seconds
  mysqlx.getSession('user@example.com?connect-timeout=20000');
  ```

  To disable the timeout set the value to 0, meaning that the client waits until the underlying socket times out, which is platform dependent. (WL #12015, WL #12210)

- The upgrade checker utility provided by MySQL Shell, which is the `checkForServerUpgrade()` function of the `util` global object, has several enhancements:

  - You can now use the upgrade checker utility to check servers at earlier MySQL 8.0.x releases, as well as MySQL 5.7 servers, for compatibility errors and issues for upgrading.

  - You can now specify a target MySQL Server version to which you plan to upgrade. In MySQL Shell 8.0.13, you can specify release 8.0.11 (the MySQL Server 8.0 GA release), 8.0.12, or 8.0.13. The upgrade checker utility carries out the checks that are relevant for the specified target release. If you specify the short form version number 8.0, or omit the `targetVersion` option, the utility checks for upgrade to the MySQL Server release number that matches the current MySQL Shell release number, currently 8.0.13.

  - A check has been added for the removed syntax `GROUP BY ASC/DESC`, returning an error message if this syntax is found in a trigger, event, view, stored procedure, or function.

  - A check has been added for columns defined as `ENUM` or `SET` that contain elements longer than 255 characters, returning an error message if any such columns are found.

  - The upgrade checker utility no longer returns a value, making its output easier to parse and process when automation is used.

  You can access the upgrade checker utility from within MySQL Shell or start it from the command line. For instructions and further information, see MySQL Shell Utilities. (WL #12173)

- MySQL Shell onscreen output can now be displayed using a pager such as `less` or `more`. You can configure the pager you want to use with the `shell.options[pager]` option, the `\pager` command, or the `--pager` command option. This improves how you work with longer text output in MySQL Shell, specifically the online help and the results of SQL operations. See Using a Pager. (WL #10775)

- The integration of MySQL Shell into command-line environments has been improved. Use the `mysqlsh [options] -- shell_object object_method [method_arguments]` syntax to pass operations directly to MySQL Shell global objects, bypassing the REPL interface. For example:

  ```
  mysqlsh -- util check-for-server-upgrade user@example --output-format=JSON
  ```

  which executes the equivalent `util.checkForServerUpgrade(user@example, {"outputFormat": "JSON"})` with MySQL Shell and returns the output in JSON format. This makes it easy to integrate MySQL Shell into your automation scripts. To get help for this interface, use the MySQL Shell command `\help cmdline`. See API Command Line Integration. (WL #10607)

- MySQL Shell has a new JSON import utility that enables you to import JSON documents from a file or standard input to a MySQL Server collection or relational table. The utility parses and validates the supplied JSON documents automatically and inserts them into the target database, removing the need to use multiple INSERT statements or write scripts to achieve this task. The utility can be started in a MySQL Shell session with the `util.importJson()` method in JavaScript or the `util.import_json()` method in Python. From the command line, you can use the `-- util importJson` syntax or the `--import` command to invoke the utility. (WL #10606)

Bugs Fixed

- The upgrade checker utility provided by MySQL Shell (the `util.checkForServerUpgrade()` operation) did not report removed functions if they were used in views or events. (Bug #28642534)

- MySQL Shell incorrectly labeled warning messages as error messages in JSON output. (Bug #28546510)

- When MySQL Shell server connection passwords are persisted using a Secret Store, if a classic MySQL protocol connection was made without specifying a port or socket, the saved password could not be retrieved for a subsequent connection. The password storage and retrieval process now ensures that the server URL used to store the password matches subsequent queries with user-provided connection options, even if defaults were used for the original connection. (Bug #28544628)

- A number of improvements have been made to the MySQL Shell prompt, including handling of overlength text, statement splitting, and support for multiple-line prompts. New sample prompt theme files are provided for double-line prompts that use one line for information display and a new line for the input prompt itself, so that additional information can be shown without detracting from the space available for text entry. (Bug #28515394, Bug #92048)

- The `--table` command line option did not produce the appropriate output format in noninteractive mode. (Bug #28511408)

- Extra spaces before or after the parameter used with the `\help` command are now trimmed. Previously, the presence of extra spaces made MySQL Shell unable to find the relevant help topic. (Bug #28508724, Bug #92030)

- Some native MySQL Shell objects were not properly wrapped into JavaScript objects, causing memory leaks. The memory-handling mechanism has been corrected. (Bug #28473341)

- If an empty string was provided as the argument for a MySQL Shell command-line option that expects a non-null argument and has no default defined, MySQL Shell did not return an appropriate error. The error handling for command-line arguments has now been improved so that a suitable error is issued in this situation and execution of the command terminates. (Bug #28378553)

- If a session was explicitly closed by the user without closing MySQL Shell, the prompt continued to display the details of the closed session. (Bug #28314383)

- User credentials stored by MySQL Shell could not be automatically retrieved for hosts identified by IPv6 addresses. (Bug #28261301)
- MySQL Shell now displays the build type (commercial or Community Edition) as part of the product version information displayed at startup and when the `--help` argument is used. (Bug #28242573)
- If a MySQL Shell session was disconnected without closing MySQL Shell (for example, using the `session.close()` method), a subsequent query in SQL mode did not return a "Not connected" error. MySQL Shell now checks not only that the global session object exists, but also that it has a valid connection to the MySQL server instance. (Bug #28240437)
- On the Windows platform, the background color was not reset in the MySQL Shell window when the terminal was cleared using **Ctrl+L**. (Bug #28235701, Bug #91102)
- The commercial MySQL Shell package could not be installed on Debian or Ubuntu if the equivalent Community Edition package had already been installed. (Bug #28223781)
- MySQL Shell was using some deprecated functions and properties internally, which caused warning messages to appear in the MySQL Shell log file, although the functions were not executed by users. The deprecated functions and properties have now been removed from the internal code. (Bug #28216558)
- If an invalid value was specified for the MySQL Shell option `credentialStore.helper`, the resulting error message at MySQL Shell startup was displayed incorrectly. (Bug #28216485)
- The upgrade checker utility provided by MySQL Shell (the `util.checkForServerUpgrade()` operation) now correctly handles account names with spaces and other blank characters, and skips the permissions check when the server was started with the `--skip-grants` option. (Bug #28212899, Bug #91326)
- When deleting history entries using the `\history` command in MySQL Shell, you can now specify a number of history entries to be deleted from the tail of the history, using the format `\history delete -number`. The handling of history entry numbers for deleted entries has been improved so that when the tail of the history is deleted, those history entry numbers are reused for new entries, and there is no gap. If a `\history delete` command empties the history, the numbering of history entries now resets as it does when the `\history clear` command is used. Also, the error message issued if you specify an invalid range of history entries to be deleted (using the format `\history delete firstnumber-lastnumber`) has been improved. (Bug #28199513)
- On the Windows platform, using the **Ctrl+C** key combination in MySQL Shell caused MySQL Shell to close, even if a command was in progress. (Bug #27894642)
- The MySQL Shell code for identifying whether a given IP address is a loopback address did not account for additional IP addresses (besides the 127.0.0.0/8 address block) that had been added by the user to the loopback interface. All IP addresses assigned to the loopback interface are now checked. (Bug #27703779)
- When running MySQL Shell in batch mode, tab separated format is the default for output. In some situations, table format was used for output instead. This issue has now been fixed. A command line option `--tabbed` has also been added to switch to the tab separated format for output when MySQL Shell is in interactive mode, where the default is table format. (Bug #27546082, Bug #89514)
- Zone IDs in IPv6 addresses are now supported for MySQL Shell connections. A zone identifier is suffixed to the IPv6 address with a percent character (%) as a separator. For example:

  ```
  2001:0db8:3c4d:0015::1a2f:1a2b%14
  ```

  If an IPv6 address with a zone ID is provided as a URI type string, URL encoding (percent-encoding) must be used for the percent character. For example:

```
mysqlsh --uri=user@[2001:0db8:3c4d:0015::1a2f:1a2b%2514]:33060
    shell.connect("user@[2001:0db8:3c4d:0015::1a2f:1a2b%2514]:33060")
```

If an IPv6 address with a zone ID is provided using individual parameters or a data dictionary, URL encoding does not need to be used for the percent character, and the IPv6 address can be supplied as seen. For example:

```
mysqlsh --user=user --host=2001:0db8:3c4d:0015::1a2f:1a2b%14 --port=33060
```

See Connecting to the Server Using URI-Like Strings or Key-Value Pairs. (Bug #27539702)

- When MySQL Shell log entries were output to `stderr` by prepending @ (at sign) to the value of the `--log-level` option, and the JSON output format was selected for MySQL Shell, some log entries were not being output in JSON format. The logger now checks and uses the current value of the MySQL Shell `outputFormat` configuration option as the output format when writing log entries to `stderr`. (Bug #27480887)
- In SQL mode, MySQL Shell erroneously entered multi-line mode if an unknown command was executed, or if multiple consecutive SQL statement delimiters were used. Now, an appropriate error is returned in these situations. (Bug #27411526)
- On the Windows platform, when a long command was accessed from the MySQL Shell command history and edited at the right edge of the console window, a cursor positioning error caused the command to move up one line and overwrite the output of previous commands. The issue has now been fixed. (Bug #27068352)
- The result printer in MySQL Shell was refactored to improve handling of binary data based on the output format, handling of multi-byte characters, and alignment of table formatting when multi-line characters are present. (Bug #24912154, Bug #24967872)

Changes in MySQL Shell 8.0.12 (2018-07-27, General Availability)

- AdminAPI Added or Changed Functionality
- AdminAPI Bugs Fixed
- Functionality Added or Changed
- Bugs Fixed

AdminAPI Added or Changed Functionality

- The *`Cluster`*`.removeInstance()` command has been improved, with the following changes:
  - A new interactive option has been added to enable or disable interactive mode for the command. The output displayed in interactive mode has been improved, displaying more useful information. In interactive mode, you are prompted to continue with the removal of the instance (or not) in case it is not reachable.
  - The operation now ensures that the instance is removed from the metadata of all the cluster members and itself. This only applies to `ONLINE` members.
  - A new global option `dba.gtidWaitTimeout` is available to define the timeout to wait for transactions (GTIDs) to be applied when required by AdminAPI commands. If the timeout value defined by `dba.gtidWaitTimeout` is reached when waiting for the cluster transactions to be applied for *`cluster`*`.removeInstance()` and `force: false` (or not defined) then an error is issued and the

operation is canceled. When `force: true` then the operation continues and does not generate an error.

(WL #11862)

References: See also: Bug #27817894.

- When using the `ipWhitelist` to define which servers could access the cluster, the internal user accounts were not matching the whitelist. Now AdminAPI applies the same filtering logic from `ipWhitelist` for the internal administrative accounts. (WL #10587)

  References: See also: Bug #26140094, Bug #28165891.

AdminAPI Bugs Fixed

- The `cluster.forceQuorumUsingPartitionOf()` operation sets the `group_replication_force_members` variable on the target instance to force a new group membership and restore the quorum, but it did not reset the value of the variable at the end of the process. Consequently, if Group Replication later needed to be restarted on the target instance it failed because the `group_replication_force_members` variable was still set. Now, the `group_replication_force_members` variable is reset to an empty string at the end of the `cluster.forceQuorumUsingPartitionOf()` operation. (Bug #28064621)
- Some messages displayed by MySQL Shell were showing a MySQL server version that does not exist. (Bug #27924694)
- It was possible to use AdminAPI operations on server instances running an incompatible version of MySQL. (Bug #27765769)
- The setting of the `bind_address` variable is no longer a requirement. (Bug #27765484)
- When creating a cluster or adding an instance, if the `localAddress` option is not specified, the port used for `group_replication_local_address` is automatically assigned with the value: `port` * 10 + 1. However, if the resulting port determined by the previous rule was already in use then a random port was generated and used. Now MySQL Shell checks that the `group_replication_local_address` port is available, and fails if it is not. (Bug #27758041)
- The `dbPassword` option is no longer valid in the options dictionary of all AdminAPI commands. (Bug #27745106)
- It was possible to use the `dba.forceQuorumUsingPartition()` operation on a cluster which had not lost quorum. (Bug #27508698)
- The help message for `dba.rebootClusterFromCompleteOutage()` operation was incorrectly suggesting to use `dba.forceQuorumUsingPartition()`. (Bug #27508627)
- The `dba.rebootClusterFromCompleteOutage()` operation was creating a new user on the target instances, which could lead to the existence of an increasing number of users. The fix ensures that these users are not created by the `dba.rebootClusterFromCompleteOutage()` operation. (Bug #27344040)
- Now when you issue `dba.getCluster()` and retrieve a cluster without quorum a warning is issued in addition to the log message. (Bug #27148943)
- The `memberSslMode` option could be used with *`cluster`*`.addInstance()` and *`cluster`*`.rejoinInstance()` operations but if you specified a different value than the one used at cluster creation an error was thrown. Now set the SSL mode at the cluster level only, in other

words when issuing `dba.createCluster()`. The `memberSslMode` option has been removed from *`cluster`*`.addInstance()` and *`cluster`*`.rejoinInstance().` (Bug #27062122)

- When you issued `dba.configureLocalInsance()` on an instance, it configured the `disabled_storage_engines` variable with the `MyISAM`, `BLACKHOLE`, `FEDERATED`, `CSV`, and `ARCHIVE` storage engines to ensure that the storage engine was set to `InnoDB`, as required by Group Replication. The change to this option was not being reported correctly by AdminAPI, and hence the required restart after changing the `disabled_storage_engines` variable was not clear. This change was deemed a recommendation, rather than a requirement, hence `dba.configureLocalInsance()` no longer configures `disabled_storage_engines`. (Bug #26754410)
- Creating a cluster using an account which was missing the global grant option failed with an ambiguous error message, even though `dba.checkInstanceConfiguration()` did not return any errors. Now when you create a cluster, the account being used to administer the cluster is checked to ensure that it has the global grant option. (Bug #25966235)
- MySQL Shell is able to automatically reconnect global session when running in the interactive mode, but AdminAPI methods lacked this feature. This resulted in you having to reconnect manually. Now, the AdminAPI methods which utilize the global session object have been improved in order to detect an interrupted session and trigger the reconnection mechanism. The Cluster object uses its own internal session instance, which does not support automatic reconnection. If connection to the cluster is lost, you need to manually recreate the Cluster object. (Bug #24702489)
- In the event of a whole cluster stopping unexpectedly, upon reboot the `memberSslMode` was not preserved. In a cluster where SSL had been disabled, upon issuing `dba.rebootClusterFromCompleteOutage()` this could prevent instances from rejoining the cluster. (Bug #90793, Bug #27986413)

Functionality Added or Changed

- **Important Change:** An RPM package for installing ARM 64-bit (aarch64) binaries of MySQL Shell on Oracle Linux 7 is now available in the MySQL Yum Repository and for direct download.

  Known Limitation for this ARM release: You must enable the Oracle Linux 7 Software Collections Repository (ol7_software_collections) to install this package, and must also adjust the libstdc++7 path. See Yum's Platform Specific Notes for additional details.

- **X DevAPI:** In order to be compliant with the X DevAPI specification, the following changes have been made:

  - `Collection.modify(condition).arrayDelete()` and `Collection.modify(condition).merge()` have been deprecated.
  - `Collection.find().limit(x).skip(y)` has been renamed to `Collection.find().limit(x).offset(y)`.
  - `Collection.find().limit(x).skip(y)` has been deprecated.
  - `Collection.find().limit(x).offset(y)` has been implemented.
  - `BaseResult.getAffectedItemsCount()` has been implemented.
  - `BaseResult.getWarningCount()` has been deprecated.
  - `BaseResult.getWarningsCount()` has been implemented.
  - `Result.getAffectedItemCount()` has been deprecated.

  - `SqlResult.getAffectedRowCount()` has been deprecated.
  - `SqlResult.nextDataSet()` has been renamed to `SqlResult.nextResult()`.
  - `SqlResult.nextDataSet()` has been deprecated.
  - `SqlResult.nextResult()` has been implemented.

  (WL #11920)

- MySQL Shell now enables you to store user credentials in an operating system specific secret store. You can then enter a MySQL user's password during connection and store it for future connections. Currently the following secret stores are supported:
  - MySQL login-path
  - MacOS keychain
  - Windows API

  (Bug #23304789, Bug #81484, WL #11745)

- The way you access the online Shell help has been standardized. Use the `\help` *`pattern`* command to search the help. The scope of the command has been increased to support retrieving help for the following categories:
  - Class and function help for the Admin API, X DevAPI and Shell API. Previously, to retrieve help for API objects, you had to create an instance of the object and use the *`object`*`.help()` method.
  - SQL syntax help, provided that a global session object exists.

  Wildcards can now be used to search for help. A number of additional bugs relating to incomplete help information have also been fixed. (Bug #23255291, Bug #81277, Bug #24963435, Bug #25732663, Bug #85481, Bug #25739522, Bug #85511, Bug #25739664, Bug #85514, Bug #26393155, Bug #86950, Bug #24943074, Bug #26429399, Bug #87037, Bug #27870491, Bug #90455, Bug #27870503, Bug #90456, Bug #27875150, Bug #90474, Bug #24948933, Bug #83527, WL #11219)

- The `util.checkForServerUpgrade()` operation has an additional `outputFormat` parameter that you can specify when running the utility. The utility can now generate output in two formats:
  - TEXT format, which is the default. This option provides output suitable for humans, as previously returned by the utility.
  - JSON format. This option provides output suitable for machines, which can be parsed and processed for various further use cases.

  (WL #11892)

Bugs Fixed

- The sample prompt theme files for MySQL Shell were deployed to an incorrect location on the Windows platform, in the root install folder. The files are now correctly deployed in the `\share\mysqlsh\prompt` sub-folder. (Bug #28188761)
- The upgrade checker utility provided by MySQL Shell (the `util.checkForServerUpgrade()` operation) has been enhanced with a summary count of the errors, warnings, and information level

issues found by the tool, and with links to documentation with further information where this is available. (Bug #28171814)

- When upgrading from version 1.0.11 to version 8.0.11 of MySQL Shell on Linux, the upgrade failed if the original package was the community edition and the new package was the commercial edition, or vice versa. Upgrading from one edition to the other edition is now enabled. (Bug #28037407)
- The `util.checkForServerUpgrade()` operation can now use either an X Protocol connection or a classic MySQL protocol connection. (Bug #28027707)
- The `checkForServerUpgrade()` operation to verify upgrade prerequisites included an unnecessary check relating to `ZEROFILL` and display length attributes in columns. The check has now been removed. (Bug #27927641, Bug #90634)
- For sessions using the classic MySQL protocol, if the `session_track_gtids` system variable is set on the server to capture and return GTIDs to the client, MySQL Shell now displays the GTIDs for successfully committed transactions. The returned GTID values are also now recorded in tracing information. (Bug #27871148)
- When the `defaultMode` MySQL Shell configuration option had been set with the `--persist` option, batch code execution from a file was always attempted using the specified default language, even if the file extension indicated a different supported language. Now when a file is loaded for batch processing using the `--file` or `-f` option, files with the extensions `.js`, `.py`, and `.sql` are processed in the appropriate language mode, regardless of the set default language. (Bug #27861407)
- The methods provided in the `shell.options` configuration interface to set and save persistent option values used underscores in JavaScript as well as in Python mode. The methods have now been changed to `shell.options.setPersist()` and `shell.options.unsetPersist()` in JavaScript to follow the appropriate naming convention. (Bug #27861141)
- When executing a SQL script using MySQL Shell, delimiters (such as the default semi-colon character) present in multi-line comments caused execution to fail. Delimiters are now ignored inside multi-line comments. (Bug #27841719)
- MySQL Shell returned an error when querying timestamp values that were zero, because a zero value for the month or day in a date was not accepted. Zero timestamp values can now be used without producing an error. (Bug #27833822, Bug #90355)
- The `shell.getSession()` function returns a reference to the `session` global object representing the already established connection between MySQL Shell and a MySQL server, known as a global session. MySQL Shell now gracefully handles the situation where the function is called when no global session has yet been established. (Bug #27809310)
- The MySQL Shell application icon on Microsoft Windows was not being displayed for the MySQL Shell 8.0 GA release, due to an incorrect association introduced for the icon during code refactoring. The icon is now displayed correctly. (Bug #27746532)
- The `\status` (`\s`) command in MySQL Shell now displays full information about the version and build of the connected MySQL server. (Bug #27740420)
- The check for reserved keywords carried out by the `util.checkForServerUpgrade()` operation was updated to match the list of reserved keywords for the MySQL 8.0 GA release. (Bug #27724201)
- When handling escape sequences, MySQL Shell now identifies and skips over SQL comments and string literals within quotation marks. (Bug #27665229)
- Python's mapping type has been added to MySQL Shell, so that dictionary syntax can be used to interact with data in Python mode. (Bug #27614110)

- When a file was redirected to standard input for execution in MySQL Shell, on Unix, the first part of the file was taken as being the password. The password prompt now looks for user input first before resorting to standard input. (Bug #27572380)
- If **Ctrl+C** was entered or an unexpected error occurred at a password prompt in MySQL Shell, the terminal state was not restored correctly afterwards. (Bug #27379834)

Changes in MySQL Shell 8.0.11 (2018-04-19, General Availability)

- AdminAPI Added or Changed Functionality
- AdminAPI Bugs Fixed
- Functionality Added or Changed
- Bugs Fixed

AdminAPI Added or Changed Functionality

- InnoDB Cluster instances running MySQL 8.0.11 and higher can now be configured for InnoDB Cluster usage remotely, without the need to log in to the instance and run `dba.configureLocalInstance()` locally. Use the new `dba.configureInstance()` operation, which enables remote automatic configuration of compatible instances for InnoDB Cluster usage and if necessary also supports remote restart of compatible instances after configuration.

  Similarly, compatible instances support having any changes to their settings automatically persisted by AdminAPI after any cluster topology changes. Now, when working with remote instances if you create a cluster, add an instance to a cluster, or remove an instance from a cluster supporting this functionality, there is no need to log in to the instance and run `dba.configureLocalInstance()` to persist the changes to the instance's option file. This makes working with production clusters, which are built on networked instances, much easier. This also ensures that instances automatically rejoin a cluster in the case of a restart.

  As part of this feature the following improvements were made:

  - Improved display of messages.
  - Improved detection of loopback hostnames on Debian and Ubuntu, and local instances based on the hostname.
  - Improved display of required configuration changes.
  - Replaced automatic configuration of instances after issuing `dba.createCluster` and `dba.addInstance` with the configuration check `dba.checkInstance()`, aborting the command if the check fails.
  - When specifying an account in interactive mode, if the user's hostname is local, for example localhost, you are given the option to recreate the account configured for remote usage by using the hostname `%`, or to create a new InnoDB Cluster administrator account.

  (WL #10434, WL #11344)

  References: See also: Bug #27608299, Bug #27112727, Bug #27629803.

AdminAPI Bugs Fixed

- After creating a cluster administrative user using `dba.createCluster()`, MySQL Shell attempted to reconnect using the new user but still using localhost, which failed causing the whole configuration to fail. Now MySQL Shell no longer switches accounts between administrative account creation and instance configuration. (Bug #27673816)
- The changes introduced by Bug#27545850 mean that now it is not possible to modify Group Replication related options while the plugin is starting or stopping. The AdminAPI operations have been modified to ensure that they respect this change, and attempting to issue a statement which would change a Group Replication operation while the plugin is starting results in an error. There is no check for when the plugin is stopping and so you should ensure you do not attempt to configure an instance if there is a chance the plugin is stopping. (Bug #27545850)
- When rejoining an instance to a cluster the displayed output has been improved. (Bug #27437389)
- The `dba.createCluster()` function fails if used on an instance with `innodb_page_size=4k`, because the `instance_name` column of the instances table in the InnoDB Cluster metadata uses a `VARCHAR` of size 256, which is too large for the `innodb_page_size=4k`. However, the size of this column cannot be changed because it is used to hold hostnames of the cluster members which can have a length up to 255 bytes. This limitation is now validated when using `dba.createCluster()`, `dba.checkInstanceConfiguration()`, `dba.configureInstance()`, and `dba.configureLocalInstance()`. (Bug #27329079)
- The AdminAPI commands which manage sandboxes failed with an error when using a sandbox directory with non-ASCII characters in the path. This occurred when the user name had a non-ASCII character because the default sandbox directory is a subdirectory of the `$HOME` or `%userprofile%` path, which is usually based on the current user name. The fix adds Unicode support to the internal provision tool used by AdminAPI. (Bug #27181177)
- MySQL Shell now sets the value of `group_replication_local_address` based on `port` by calculating the result of `((port * 10) + 1)`. The `port` variable defaults to 3306, in which case MySQL Shell sets `group_replication_local_address` to `((3306 * 10) +1)`, resulting in port 33061 instead of the previous default of 13306. (Bug #27146799)
- The online help for all AdminAPI commands which require a connection displayed detailed information on connection data, which cluttered up the details of each command. Now, the help displays an overview of connection details and information about where else to get more help. (Bug #27146290)
- When creating a cluster on a Debian type platform with the default system configuration, the` `cluster.addInstance()` command failed with an error if the instance hostname was used in the connection parameter. This happened because on these platforms the hostname is resolved to the IP address 127.0.1.1 by default, but this IP address is not supported by the Group Replication Group Communication Service (GCS). The fix adds a validation to the `dba.createCluster()` and `cluster.addInstance()` functions to verify if the connection hostname resolves to 127.0.1.1 and issue an error in that case. (Bug #27095984)
- The `Cluster.forceQuorumUsingPartitionOf()` operation was not working with a non-root user, even if that user had all the required privileges. The fix ensures that the specified user is used to connect to all instances, instead of the root user. (Bug #27089930)
- Issuing `dba.createCluster()` in interactive mode as a user which did not have all the required privileges resulted in an unexpected halt. The error message generated when issuing `dba.configureLocalInstance()` and `dba.checkInstanceConfiguration()` if the account being used did not have the required privileges has also been improved. (Bug #27076753, Bug #27324699)

- When using `adoptFromGR` to convert a Group Replication group into an InnoDB Cluster, the output recommended adding instances. This message has now been improved to show that the instances were already added. (Bug #27061615)
- In interactive mode the `cluster.removeInstance()` AdminAPI function was not accepting the second parameter with the options dictionary. This prevented use of the `force` option, which failed with an error. (Bug #26986141)

  References: See also: Bug #27572618.
- The `cluster.addInstance()` function was not checking the validity of the `server_uuid` being added to the cluster. Now, the `server_uuid` of an instance joining the cluster is checked to be unique, and if it is not an error is generated and the instance is prevented from joining the cluster. (Bug #26962715)
- When setting up a cluster a replication user is created to enable distributed recovery. If MySQL Shell logging was enabled, the `CREATE USER` statements were not being correctly added to the log. (Bug #26938488)
- The `dba.checkInstanceConfiguration()` and `dba.configureLocalInstance()` operations were not correctly reporting any errors related to incorrect configuration of the instance's `server_id`. Now, when an error is reported it is displayed in the list of variables not meeting the InnoDB Cluster requirements.

  In addition, the X Plugin load has been removed from the `my.cnf` created by AdminAPI for sandbox instances running MySQL version 8.0.11 and later because X Plugin is installed by default for those versions. (Bug #26836230)
- When using `dba.configureLocalInstance()` with the `clusterAdmin` option to create a user which can administer the cluster, the created account had too many privileges. (Bug #26737608)
- When you create a cluster and add instances to it internal users are created, which are required by Group Replication for distributed recovery when instances join the cluster. These automatically generated replication users were not being removed from instances after removing them from the cluster or after dissolving the cluster. (Bug #26395608)
- If an instance contained InnoDB Cluster metadata but was stand alone, in other words it did not belong to a cluster, it was not possible to use `dba.dropMetadataSchema()`. (Bug #26315635)
- If you issued `dba.configureLocalInstance()` against an instance which was already valid for InnoDB Cluster usage and specified the `myCnf` option, but the `my.cnf` file was not readable, the operation would print that there are issues that need to be fixed. Now when you set the `myCnf` option, using either `dba.configureInstance()` and `dba.configureLocalInstance()`, the operations only use it if necessary. In other words, if the target instance is not valid for InnoDB Cluster usage and settings must be changed. (Bug #25702994)
- The `cluster.describe()` and `cluster.status()` AdminAPI methods return JSON objects containing the same information but some fields were identified by different tags. To make the tags consistent, the object returned from `cluster.describe()` has changed so that `instances` has been replaced with `topology`, and `host` with `address`. (Bug #25247515)
- Creating a cluster from an existing Group Replication group by using the `adoptFromGR:true` option on an instance which already belonged to cluster was not being correctly detected. Now such a situation is detected and generates an error. (Bug #25061891, Bug #25664766)
- MySQL Shell is able to create a session to an IPv6 address, but it failed to create an InnoDB Cluster using such connections, reporting an URI-related error. This was related to the encoding of the URI containing an IPv6 address, which has been fixed. The core issue, IPv6 support for AdminAPI is a

known issue as Group replication requires an IPv4 network, see Group Replication Requirements. The implementation of InnoDB Cluster related operations has been modified in order to check if operations are executed using an IPv4 based session and IPv4 connection data. An exception is generated if these conditions are not met. (Bug #25042407)

- Attempting to add two different instances with the same label to an InnoDB Cluster correctly resulted in an error, however the instance with the duplicated label was being left with a partially initialized configuration. (Bug #24761416)

Functionality Added or Changed

- **Important Change; Microsoft Windows:** Before installing MySQL Shell, make sure you have the Visual C++ Redistributable for Visual Studio 2015 (available at the Microsoft Download Center) installed on your Windows system. This now applies for both Community and Commercial versions of MySQL Shell.

- **Important Change; X DevAPI:** The way which document IDs are generated has been changed. Now document IDs are generated by the server, rather than by the client. As a result the `getLastDocumentID()`, `getDocumentId` and `getDocumentIDs()` methods have been removed. To get a list of the document IDs automatically generated by an 8.0.11 and later server, use `Result.getGeneratedIDs()`. The type of column generated for the document ID in a collection has changed from `VARCHAR(32)` to `VARBINARY(32)`. The generated document ID can be overridden manually by including an ID but you must respect the server generated IDs to avoid conflicts. If you are using InnoDB Cluster, use the `mysqlx_document_id_unique_prefix` variable to ensure your documents can be moved between ReplicaSets.

  Now, if you are adding documents to a collection on a server running a version of MySQL earlier than 8.0.11, you must manually include a document ID as these versions do not add the ID automatically. (WL #10955, WL #11390)

- **X DevAPI:** Support for the `NOWAIT` and `SKIP LOCKED InnoDB` locking modes has been added to the lock operations. Now you can use these locking modes with the `lockShared()` and `lockExclusive()` methods, for example:

  - `Table.select().lockShared([LockContention])`
  - `Table.select().lockExclusive([LockContention])`
  - `Collection.find().lockExclusive([LockContention])`
  - `Collection.find().lockExclusive([LockContention])`

  where `LockContention` can be one of:

  - `DEFAULT` - if the function encounters a row lock it waits until there is no lock
  - `NOWAIT` - if the function encounters a row lock it aborts and generates an `ER_LOCK_NOWAIT` error
  - `SKIP_LOCKED` - if the function encounters a row lock it skips the row and continues

  For more information see Locking Read Concurrency with NOWAIT and SKIP LOCKED. (WL #11327, WL #11417, WL #11242)

- The indexing of collections has been improved to make large collections of documents more efficient to navigate. Now, you can create an index based on one or more fields found in the documents in the collection, using a JSON document which maps fields from the collection's documents to MySQL types.

The majority of MySQL types are supported, in addition to spatial indexes and GeoJSON data. (WL #9876, WL #10858)

- MySQL Shell can now connect to a MySQL server with an account that uses the `caching_sha2_password` authentication plugin. Assuming the server is configured for encrypted connections, you can use such accounts over both X Protocol and the classic MySQL protocol. See Using Encrypted Connections.

  

  Important

  If you are not using encrypted connections, to connect over X Protocol with an account that uses the `caching_sha2_password` authentication plugin, the user's password must be stored in the cache. Currently there is no way to store the password over X Protocol if not using encrypted connections.

  When using the classic MySQL protocol for connections with such an account and unencrypted connections, you can configure MySQL for password exchange using an RSA key pair. MySQL Shell supports such connections and the following command options have been added:

  - Use the `--server-public-key-path` option to specify the RSA public key file.
  - Use the `--get-server-public-key` option to request the public key from the server.

  For more information, see Caching SHA-2 Pluggable Authentication. (WL #11591, WL #11555)

- MySQL Shell now has a configuration file which stores configuration changes across sessions. Use the new `\option` MySQL Shell command for querying and changing configuration options. Alternatively use the following methods with the `shell.options` object:

  ```
  shell.options.set_persist(optionName, value)
  shell.options.unset_persist(optionName, value)
  ```

  In addition the new `defaultMode` option has been added, which enables you to configure the programming language which MySQL Shell starts up in. You can override the default mode using the command options. (WL #10783)

- The `util.checkForServerUpgrade([uri])` operation has been extended to check for the following incompatible features:

  - obsolete `sql_mode`
  - partitioned tables in shared tablespaces
  - removed functions

  (WL #11622)

Bugs Fixed

- **X DevAPI:** The handling of SQL wildcard characters was corrected for the X DevAPI `Schema.getTable()`, `Schema.getCollection()` and `Session.getSchema()` functions. (Bug #26392984)
- When the `\quit` command was used to exit MySQL Shell, this event could be noted in the error log as a lost connection. The issue has now been fixed. (Bug #27821045, Bug #90281)

- When the MySQL Shell command-line option `--json=raw` was used, the output was actually provided in pretty-printed format, and an empty string was displayed in place of error messages. These issues have now been corrected. (Bug #27733996, Bug #26737357)

- When MySQL Shell commands were executed from a script, interactive prompting for passwords and confirmations was not available. Now, interactive prompting is enabled by default when the commands are used in a script, as it is when they are used on the command line. The `--no-wizard` command line option disables interactive prompting for MySQL Shell commands used in both ways. (Bug #27702250)

- The `util.checkForServerUpgrade()` operation now prompts the user for a password interactively if the required password is not provided along with the command. If the `--no-wizard` option has been used to disable the connection wizard, missing credentials instead result in an error and the function is not executed. (Bug #27514395)

- The `util.checkForServerUpgrade()` operation was requiring the wrong privileges for the user passed to the function. The user now requires `ALL` privileges, and does not require the `GRANT OPTION` privilege. (Bug #27506702)

- The `util.checkForServerUpgrade()` operation was rejecting host names that included the % (percent) symbol or were specified as a numeric IP address. (Bug #27506079, Bug #27513260)

- By default, MySQL Shell connections are assumed to require a password, which is requested at the login prompt. A new MySQL Shell command-line option `--no-password` is provided to explicitly specify that no password is used, and to disable the password prompt. The `--no-password` option can be used if socket peer-credential authentication is in use (for Unix socket connections), or for any authentication method where the user has a passwordless account (though note that this situation is insecure and not recommended).

  The methods previously provided by MySQL Shell for specifying that no password is used for the connection are still valid, and can be used instead of the `--no-password` option. These methods are as follows:

  - If you are connecting using a URI type string, place a `:` after the user name in the URI type string but do not specify a password after it.

  - If you are connecting using individual parameters, specify the `--password=` option with an empty value.

  (Bug #26986360)

- When an error occurred while returning a result set, the fetch operation was interrupted but the associated error was not reported. The error is now reported correctly. (Bug #26906527)

- Support for microseconds has been added to the `mysqlx.dateValue()` function and the `Date` object. (Bug #26429497)

- Argument validation and error messages were improved for the `mysqlx.dateValue()` function. (Bug #26429426, Bug #26429377)

- The `db` global object is not available for connections in SQL mode. This reference has been removed from the message returned by the `\connect` command in that situation. (Bug #26428665)

- A shortcut to uninstall MySQL Shell is no longer provided in Microsoft Windows menus in the group of installed MySQL programs. Uninstallation of MySQL Shell should be handled through MySQL Installer. (Bug #26317449)

- The runtime timer for MySQL Shell, which reports the time taken for each query execution, has been refactored to provide increased precision of 4 digits for fractional seconds. (Bug #25976636, Bug #86135)
- In the event of a disconnection, MySQL Shell did not reconnect to the schema that was in use before the connection was lost. The last active schema set by the user is now restored during the automatic reconnection process, and during a manually triggered reconnection using the `\reconnect` command. (Bug #25974003, Bug #86115)
- MySQL Shell no longer attempts to reconnect automatically when the connection to the server is lost. A new MySQL Shell command `\reconnect` is provided, which makes MySQL Shell try several reconnection attempts for the current global session with the existing connection parameters. If those attempts are unsuccessful, you can make a fresh connection using the `\connect` command and specifying the connection parameters. (Bug #25105307)
- A memory leak was fixed that occurred if MySQL Shell was started and a connection was made to the MySQL server, then the user exited MySQL Shell without executing any commands. (Bug #24794589)

Changes in MySQL Shell 8.0.5 - 8.0.10 (Skipped version numbers)

There are no release notes for these skipped version numbers.

Changes in MySQL Shell 8.0.4 (2018-01-25, Release Candidate)

- Deprecation and Removal Notes
- AdminAPI Added or Changed Functionality
- AdminAPI Bugs Fixed
- Functionality Added or Changed
- Bugs Fixed

Deprecation and Removal Notes

- The command line options `--classic`, `--node`, `--sqln`, and `--ssl` were stated as deprecated, but actually had been removed. MySQL Shell now handles the options again, but prints a warning message when they are used. The deprecated options are processed as their replacement options, as follows:
  - `--classic` is processed as `--mysql`
  - `--node` is processed as `--mysqlx`
  - `--sqln` is processed as `--sqlx`
  - `--ssl` is processed as `--ssl-mode`

  (Bug #27012385)

AdminAPI Added or Changed Functionality

- When you create clusters or add instances you can now override the default group name, local addresses, and group seeds. This makes it easier to customize your clusters. The following options were added to the `dba.createCluster()` and `cluster.addInstance()` commands:
  - use `groupName` with `dba.createCluster()` to set the name of the cluster

  - use `localAddress` to set the address which an instance provides to communicate with other instances
  - use `groupSeeds` to set the instances used as seeds when instances join the cluster

  For more information, see Customizing InnoDB Cluster Member Servers. (Bug #26485254, Bug #26838005, WL #11162)

- Connections to an InnoDB cluster have been simplified. Now, when you issue `dba.getCluster()` and the active Shell session is not a connection to the primary, the cluster is queried for the primary information and a connection to the primary is automatically opened. This ensures that configuration changes can be written to the metadata. As part of this improvement you can now configure the type of MySQL Shell connection to an InnoDB Cluster with the following options which have been added:

  - `--redirect-primary`
  - `--redirect-secondary`
  - `--cluster`

  Additionally the following changes were made:

  - The `dba.resetSession()` method has been removed.
  - A new `disconnect()` method has been added to the `cluster` object, which closes all internal sessions opened by the cluster. InnoDB Cluster operations on a disconnected cluster object result in an error.
  - The output of the `Cluster.status()` method now includes the `groupInformationSourceMember` field, which shows the URI of the internal connection used by the cluster object to obtain information about the cluster.

  (WL #11251)

  References: See also: Bug #25091586.

AdminAPI Bugs Fixed

- Account validation did not work correctly unless the session account existed. Now, validation is done using the account that was authenticated by the server. (Bug #26979375)
- The AdminAPI in MySQL Shell for working with InnoDB cluster only supports TCP connections to server instances. The AdminAPI now checks that a TCP connection is in use before starting an operation that requires database access, instead of attempting the operation with another connection type and not succeeding. (Bug #26970629)
- If you issued `STOP GROUP REPLICATION` on an instance that belonged to a cluster, attempting to rejoin the instance to the cluster failed because the wrong Group Replication seeds were being used. Now, `Cluster.rejoinInstance()` correctly sets `group_replication_group_seeds` based on the `group_replication_local_address` of all currently active instances in the cluster. (Bug #26861636)
- Sometimes the `dba.addInstance()` command failed with an error indicating that the server was in `RECOVERING` state despite being `ONLINE`. The fix ensures the correct instance state is returned. (Bug #26834542)

- If the user running MySQL Shell did not have write permissions to the option file configured by AdminAPI, no error was displayed. (Bug #26830224)
- With the addition of WL#10470, the default value of `server_id` has changed to 1. As the server ID has to be unique for each instance, this caused issues with AdminAPI. Now, server instances with `server_id=1` are correctly identified as incorrectly configured for InnoDB Cluster use. (Bug #26818744)
- AdminAPI now supports Python 2.6 in addition to Python 2.7, removing the need to manually install on Oracle Linux 6. (Bug #26809748)
- After removing an instance from a cluster using `Cluster.removeInstance()`, the instance silently rejoined the Group Replication group after it restarted. This happened because `group_replication_start_on_boot` was set to `ON` by default. Now, for instances running MySQL version 8.0.4 and later, the fix sets `group_replication_start_on_boot` to `OFF` in the option file. For instances running a MySQL version earlier than 8.0.4, a warning is issued to tell you to manually edit `group_replication_start_on_boot` in the instance's option file to avoid the issue. (Bug #26796118)
- Using AdminAPI commands on Windows that required SSL resulted in an error due to the Python version being used. (Bug #26636911)
- Creating an InnoDB Cluster from an existing Group Replication deployment, by using the `adoptFromGR` option with the `dba.createCluster()` command, would fail with an error stating that the instance was already part of a replication group. The issue was only present in the MySQL Shell default wizard mode. The fix ensures that the interactive layer of the `dba.createCluster()` command allows the use of the `adoptFromGR` option. (Bug #26485316)
- The warnings generated when creating and adding sandbox instances have been improved. (Bug #26393614)
- When working with instances that had `require_secure_transport=ON`, AdminAPI commands that required a connection to the instance failed. (Bug #26248116)
- The `Cluster.dissolve()` command was trying to stop Group Replication on all of the instances registered in the metadata which lead to connection errors if any of those instances were not contactable, in other words with the state `(MISSING)`. The fix ensures that only instances which can be contacted, in other words with the state `ONLINE`, are stopped. (Bug #26001653)
- When adding instances to an InnoDB Cluster using the appropriate AdminAPI operations, checks are performed to verify the compatibility of any existing tables. If incompatible tables (for example using `MyISAM`) are detected then an error is issued. However the error message was referring to an option not available for the AdminAPI operations: `--allow-non-compatible-tables`. (Bug #25966731)
- The `cluster.rejoinInstance()` command attempted to rejoin an instance even if was already part of the cluster. Now, only instances in the `MISSING` state are accepted by `cluster.rejoinInstance()`. Attempting to rejoin an instance in any other state fails with an error. (Bug #87873, Bug #26870329)
- On Unix, if Python 3 was installed AdminAPI commands failed. (Bug #87731, Bug #26785584)
- When using the `dba.checkInstanceConfiguration()` and `dba.configurelocalinstance()` commands, the account being used was not being checked if it had enough privileges to actually execute the command. The fix ensures that account has the required privileges before proceeding. This also required a change of the privileges given to `clusterAdmin` users. (Bug #87300, Bug #26609909)

Functionality Added or Changed

- **X DevAPI:** A new `patch()` operation has been added to the `modify()` function which enables you to modify a document by merging in a set of changes. For more information see `JSON_MERGE_PATCH()`. (WL #10856)

- **X DevAPI:** MySQL Shell performs automatic `_id` generation on collection add operations when no `_id` is specified on the documents being added. The autogenerated _id is created using the UUID() function. Now, the order of the tokens used has changed from `aaaaaaaa-bbbb-cccc-dddd-eeeeeeeeeeee`, where `eeeeeeeeeeee` is the MAC Address, to `eeeeeeeeeeee-dddd-cccc-bbbb-aaaaaaaa`. (WL #10005)

- **X DevAPI:** Sessions created by X DevAPI using either `mysql.getClassicSession(connection_data)` or `mysqlx.getSession(connection_data)` now use `ssl-mode=REQUIRED` as the default if no `ssl-mode` is provided, and neither `ssl-ca` nor `ssl-capath` is provided. If no `ssl-mode` is provided and any of `ssl-ca` or `ssl`-capath is provided, all Sessions created by MySQL Shell now default to `ssl-mode=VERIFY_CA`. (WL #10706)

- **X DevAPI:** In addition to the existing CRUD commands which work on documents in a collection by matching a filter, the following operations have been added to enable you to work with single documents:

  - `Collection.replaceOne(string id, Document doc)` updates (or replaces) the document identified by `id` with the provided one, if it exists.

  - `Collection.addOrReplaceOne(string id, Document doc)` add the given document. If the `id` or any other field that has a unique index on it already exists in the collection, the operations updates the matching document instead.

  - `Document Collection.getOne(string id)` returns the document with the given `id`. This is a shortcut for `Collection.find("_id = :id").bind("id", id).execute().fetchOne()`.

  - `Result Collection.removeOne(string id)` removes the document with the given `id`. This is a shortcut for `Collection.remove("_id = :id").bind("id", id).execute()`.

  Using these operations you can reference documents by ID, making operations on single documents simpler by following a load, modify and save pattern such as:

  ```
  doc = collection.getOne(id);
  doc["address"] = "123 Long Street";
  collection.replaceOne(id, doc);
  ```

  (WL #10849)

- **X DevAPI:** You can now use savepoints with MySQL Shell sessions. The following methods have been added to the `Session` object:

  - Use `setSavepoint()` to generate a savepoint. The server executes the `SAVEPOINT` SQL statement and returns the generated savepoint name.

  - Use `setSavepoint(name)` to specify the name used by the `SAVEPOINT` SQL statement.

  - Use `releaseSavepoint(name)` to execute the `RELEASE` SQL statement.

  - Use `rollbackTo(name)` to execute the `ROLLBACK TO name` SQL statement.

  Any names passed to these functions are checked to make sure that the name is not null or an empty string. Names such as `''`, `""`, ``` `` ```, and so on are not allowed even though they are allowed

by the server. For more information, see SAVEPOINT, ROLLBACK TO SAVEPOINT, and RELEASE SAVEPOINT Statements. (WL #10868, WL #10869)

- **X DevAPI:** X DevAPI now supports row-locking for CRUD operations on tables and collections. Use the `lockShared()` method for write locks and the `lockExclusive()` method for read locks, which have been added to `find()` and `select()`. Either method can be called any number of times, including none, in any combination. But the locking type to be used is always the last one called. Once you call a lock method you can only then execute the operation, calling `execute()` or `bind()`. For more information, see InnoDB Locking. (WL #10850, WL #10641)

- **X DevAPI:** The X DevAPI drop operations have been improved. Now, the `drop()` methods are at the same level as the `create()` methods and they all return nothing. There is now no need to use `execute()` after the drop method. If a drop method is called on an object which no longer exists in the database there is now no error.

  This modifies `Session` objects so that:

  - `dropSchema()` returns Nothing
  - `dropTable(String schema, String name)` is removed
  - `dropCollection(String schema, String name)` is removed
  - `dropView(String schema, String name)` is removed

  This modifies Schema objects so that:

  - Nothing `dropCollection(String name)` is added

  This modifies Collection objects so that:

  - `dropIndex(String name)` is a direct operation, meaning `.execute()` is no longer needed
  - `dropIndex(String name)` returns Nothing

  (WL #10712)

- **X DevAPI:** The `mysql` module, used with classic sessions for SQL connections to instances, has been streamlined. This has resulted in the following changes:

  - The new `getSession()` function has been added to the `mysql` module to create a classic session. This function works in the same way as the existing `mysql.getClassicSession()` function.
  - The `runSql()` method of `ClassicSession` has been extended to take a list of arguments to replace with placeholders in the query. For example, now you can issue:

    ```
    session.runSql("select * from tbl where name = ? and age > ?", ["Joe", "20"])
    ```

    Additionally a `query()` function has been added to `ClassicSession` as an alias to `runSql()`, making it consistent with `Session.query()`.

  - The following functions have been removed from `ClassicSession`:

    - `createSchema()`
    - `getSchema()`
    - `getDefaultSchema()`

    - `getCurrentSchema()`
    - `setCurrentSchema()`
    - `ClassicSchema()`
  - The `ClassicTable` object has been removed.
  - The following `Table` and View DDL functions have been removed from `Schema` objects:
    - `dropTable()`
    - `dropView()`
  - The behavior of the global `db` variable has been modified. When a global connection is created and the connection data includes a default schema, the global `db` variable is set to the corresponding `Schema` object. Now, this is not done for connections through the classic MySQL protocol, such as those created by `ClassicSession`.

  (WL #11180)

- MySQL Shell now supports autocompletion of text preceding the cursor by pressing the Tab key. Autocompletion is available for:
  - Built-in MySQL Shell commands, for example typing **`\con`** followed by the Tab key completes to `\connect`.
  - SQL, JavaScript and Python language keywords depending on the current MySQL Shell mode.
  - Table names, column names, and active schema names in SQL mode, based on the current default schema.

  Autocompletion can be configured using the following command options:

  - `--name-cache`
  - `--no-name-cache`

  (WL #10447)

- You can now use Unix sockets for X Protocol connections. Socket file paths in URI type strings should be either percent encoded, such as `root@/home%2Fuser%2Fmysql-sandboxes%2F3310%2Fsandboxdata%2Fmysqlx.sock`, or surrounded by parenthesis such as `root@(/home/user/mysql-sandboxes/3310/sandboxdata/mysqlx.sock)`. The `--socket` option cannot be combined with the `--port` option. See Connecting to the Server Using URI-Like Strings or Key-Value Pairs. (WL #10503)

- Use `util.checkForServerUpgrade([uri])` to check if a server can be upgraded from the version it is currently running to the next major series release. For example, you can verify that a server instance running MySQL 5.7 satisfies the upgrade prerequisites for MySQL 8.0, see Preparing Your Installation for Upgrade. To verify a server instance at a URI type string such as `user@example.com:3306` issue:

  ```
  util.checkForServerUpgrade(['user@example.com:3306'])
  ```

  MySQL Shell provides a report on anything in the table space which would cause a problem when upgrading the instance to MySQL 8.0. (WL #11051)

- MySQL Shell builds against MySQL server version 8.0.4 and OpenSSL. (WL #11343)
- MySQL 8.0.4 uses the `caching_sha2_password` authentication plugin and encrypted connections by default. This means any new accounts created in MySQL use these features. The new authentication method is completely transparent if the connection is made with encrypted connections enabled, which is the default, and the preferred mode of connection. If encrypted connections are explicitly disabled, the following limitations apply:
  - X Protocol sessions require encrypted connections.
  - If the caching_sha2_password plugin reports an error such as `Authentication requires a secure connection`, it is not possible to connect to the account with MySQL Shell, until either an encrypted connection is made or the `mysql` client is used to clear the error.

  See Using Encrypted Connections and Caching SHA-2 Pluggable Authentication. (WL #11513)

Bugs Fixed

- Attempting a server connection without specifying the port or socket, when using an expired or temporary password, failed with an error requiring a password reset. The default connection data is now set at an appropriate point in connection processing, so the connection succeeds. (Bug #27266648)
- MySQL Shell required the option name `ssl-ciphers` instead of the standard MySQL option name `--ssl-cipher` to specify the list of permitted ciphers for connection encryption. The standard option name `--ssl-cipher` is now required in MySQL Shell. (Bug #27185275)
- The `--show-warnings` option was not working in MySQL Shell. (Bug #27036716)
- The output of `\status` when using a Unix socket for connections was showing a relative path. Now the absolute path of the socket is shown. (Bug #26983193)
- MySQL Shell now automatically pre-loads the built-in `mysql` and `mysqlx` API modules when it is invoked in batch mode, as well as when it is used in interactive mode. (Bug #26174373)
- The user configuration path for MySQL Shell defaults to `%AppData%\MySQL\mysqlsh\` on Windows, and `~/.mysqlsh/` on Unix. This directory is used for the MySQL Shell history file (`history`), log file (`mysqlsh.log`), and theme file (`prompt.json`). It is also the final location that MySQL Shell searches for startup scripts (`mysqlshrc.js` or `mysqlshrc.py`).

  You can now override the user configuration path by specifying an alternative path using the `MYSQLSH_USER_CONFIG_HOME` environment variable. A directory specified by that environment variable is used by MySQL Shell for the user configuration data in place of `%AppData%\MySQL\mysqlsh\` on Windows, and `~/.mysqlsh/` on Unix. (Bug #26025157)
- For file processing, MySQL Shell now expands a leading tilde in a file path to the appropriate home directory. MySQL Shell identifies the home directory using a relevant environment variable, or looks it up for the logged-in user. (Bug #25676405)
- MySQL Shell now provides a `program_name` connection attribute to the server at connect time, with the value `mysqlsh`. Connection attributes are displayed in the Performance Schema connection attribute tables. (Bug #24735491, Bug #82771)
- Running `help()` in MySQL Shell in Python mode caused an AttributeError. (Bug #24554329, Bug #82767)
- Arrays and Objects now accept the `IN` operator. For example:

```
collection.find("'Fred' IN username")
```

(WL #10848, WL #10610)

Changes in MySQL Shell 8.0.3 (2017-09-29, Development Milestone)

MySQL Shell now synchronizes the first digit of its version number with the (highest) MySQL server version it supports. This change makes it easy and intuitive to decide which client version to use for which server version. MySQL Shell now uses the same version number as MySQL Server.

MySQL Shell 8.0.3 is the first release to use the new numbering. It is the successor to MySQL Shell 8.0.0.

- Deprecation and Removal Notes
- AdminAPI Added or Changed Functionality
- AdminAPI Bugs Fixed
- Functionality Added or Changed
- Bugs Fixed

Deprecation and Removal Notes

- **X DevAPI:** The types of session available have been simplified. `XSession` and `NodeSession` have been consolidated into `Session`. This has caused the following changes:
  - The following command options have been deprecated: `--node`, `--sqln`, `--classic`
  - The following command options have been introduced to replace the deprecated ones: `-ma`, `--mysqlx` (`-mx`), `--mysql` (`-mc`), `--sqlx`
  - The `\connect` MySQL Shell command no longer supports the arguments `-c`, and `-n`. Now the `\connect` command supports the argument `--mysqlx`(`-mx`) for creating X Protocol connections, and `--mysql`(`-mc`) for creating classic MySQL protocol connections.

  (WL #10823)
- The `--ssl` option has been deprecated, use the `--ssl-mode` option. Now, if you use the `--ssl` option, a deprecation warning is generated and the `--ssl-mode` option is set to either `DISABLED` or `REQUIRED` based on the value used with the `--ssl` option. (Bug #25403945)

AdminAPI Added or Changed Functionality

- With the addition of WL#10611 and WL#10960 , it was not possible to add or rejoin instances that belonged to a cluster (or a replication group) because `super_read_only=ON` was being set by Group Replication when stopping. To ensure that AdminAPI supports instances running MySQL 8.0.2 and later, the following functions have been modified:
  - `dba.configureLocalInstance()`
  - `dba.createCluster()`
  - `dba.rebootClusterFromCompleteOutage()`
  - `dba.dropMetadataSchema()`

  Now, if any of these functions is issued against an instance which has `super_read_only=ON`, in interactive mode you are given the choice to set `super_read_only=OFF`. To force the function to

set `super_read_only=OFF` in a script, pass the `clearReadOnly` option set to `true`. For example `dba.configureLocalInstance({clearReadOnly: true}).` For more information see Instance Configuration in Super Read-only Mode. (Bug #26422638, WL #11054)

AdminAPI Bugs Fixed

- When using the `clusterAdmin` option, the created account did not have all of the correct privileges. (Bug #26523629)
- When using the `multiMaster` option with `dba.createCluster()`, the warning displayed in interactive mode was not being logged. (Bug #26385634)
- When making cluster topology or membership changes, AdminAPI was not taking into consideration the value of `group_replication_group_name`, which could lead to incorrect, non-deterministic results in scenarios such as a split brain. Now, the following commands validate the InnoDB cluster Metadata and the corresponding instance's `group_replication_group_name` value:

  - `dba.getCluster()`
  - `Cluster.rejoinInstance()`
  - `Cluster.forceQuorumUsingPartitionOf()`

  If the values of `group_replication_group_name` do not match, the commands abort with an error.

  `dba.rebootClusterFromCompleteOutage()` was also updated to ensure that the `group_replication_group_name` variable has not been changed before rejoining the instance. (Bug #26159339)
- AdminAPI now always uses the active user value for the current `mysqlsh` session, whether the value was explicitly specified by the user or is the result of an implicit default used by `mysqlsh`. (Bug #26132527)
- The checks performed by the AdminAPI upon issuing `dba.rebootClusterFromCompleteOutage()` were more strict than those required by Group Replication. Now, the AdminAPI considers tables with a Primary Key Equivalent (such as a Non Null Unique Key) as compatible, matching the current requirement for Group Replication. (Bug #25974689)
- The randomly generated passwords used by internal users were not compatible with instances running the Password Validation plugin. (Bug #25714751)
- It is no longer possible to use the `adoptFromGr` option with the `multiMaster` option. When adopting an existing group to an InnoDB Cluster, the group is adopted based on whether it is running as multi-primary or single-primary. Therefore there is no use for the `multiMaster` option when adapting a group. (Bug #25664700)
- Issuing `configureLocalInstance()` when using a URI that contained a user without the correct privileges resulted in an incorrect new user being created. Now, if the user in `configureLocalInstance()` URI does not have enough privileges to grant all the necessary privileges for the new user chosen during the interactive wizard configuration the user is not created. (Bug #25614855)
- Issuing `Cluster.rescan()` resulted in non-deterministic behavior which could produce incorrect JSON output, showing an instance that was already part of the cluster as belonging to the `newlyDiscoveredInstances[]` list and to the `unavailableInstances`[] list. This also resulted in MySQL Shell prompting to add or remove the instance from the cluster. (Bug #25534693)

- AdminAPI functions now accept the standard connection parameters as used by `shell.connect`. New validations have been added for when `require_secure_transport` is `ON`, now it is not possible to create a cluster with `memberSslMode:DISABLED` or to add an instance with `require_secure_transport=ON` to a cluster where `memberSslMode:DISABLED`. (Bug #25532298)
- The parsing of account names, for example when passing the `clusterAdmin` option to `dba.configureLocalInstance()` has been improved. (Bug #25528695)
- The file permissions of option files created by AdminAPI did not match those of options files created by MySQL install. (Bug #25526248)
- Issuing `configureLocalInstance()` twice could fail. (Bug #25519190)
- When passing the `rejoinInstances[]` option to `dba.rebootClusterFromCompleteOutage()`, if no `rejoinInstances[]` option was specified then members were being incorrectly handled during the rebuild. Now, instances that are eligible to be added to the `rejoinInstances[]` list but that are specified in the `removeInstances[]` list are skipped by the interactive wizard that tries to automatically build a `rejoinInstances[]` list if one was not provided. This fix also ensures that both interactive and noninteractive use of MySQL Shell correctly verify the `rejoinInstances[]` list does not contain a unreachable instance. (Bug #25516390)
- The error messages issued when the SSL mode used by the cluster and the one specified when issuing `addInstance()` command do not match have been improved. (Bug #25495056)
- When creating a sandbox instance using the `dba.deploySandboxInstance()` function in MySQL Shell, pressing **Ctrl+C** at the prompt for a MySQL root password for the instance did not cancel the deployment. (Bug #25316811)
- Issuing `removeInstance`() on the last member of a cluster, and particularly the seed member, was resulting in a cluster that could not be dissolved. Now, issuing `removeInstance()` on the last member of a cluster results in an error, and you must use `dissolve()` on that instance to ensure the cluster is correctly dissolved. (Bug #25226130)
- The output of `cluster.status()` now includes the `ssl` parameter, which shows whether secure connections are required by the cluster or disabled. (Bug #25226117)
- Attempting to create a multi-primary cluster in interactive mode failed unless you passed in the `{force: true}` option. Now when you confirm that you understand the impact of using multi-primary mode the command correctly creates a multi-primary cluster. (Bug #25034951)
- The `removeInstance()` was not working on stopped instances and it was not possible to remove an unavailable instance from the cluster. The fix adds a new option `force` to the removeInstance() command to enable you to remove instances from the metadata that are permanently not available, avoiding obsolete data from being kept in the metadata of the cluster. In addition the error message provided when not using the force option has been improved and the online help for the `removeInstance()` was also updated accordingly. (Bug #24916064)
- The error messages generated by issuing `dba.deployLocalInstance()` against an unsuitable or incompatible instance have been improved. (Bug #24598272)
- The `dba.createCluster()`, `dba.getCluster()`, and `dba.rebootClusterFromCompleteOutage()` functions have been updated to validate the cluster name, using the following rules:
  - Name must start with a letter or the _ character
  - Name can only contain alphanumeric characters and the _ character

- Cannot be longer than 40 characters
- Cannot be empty

The `Cluster.addInstance()` function has been updated to validate the label used on an instance in the cluster, using the following rules:

- Label can only contain alphanumerics or the _ character
- Cannot be longer than 256 characters
- Cannot be empty

(Bug #24565242)

Functionality Added or Changed

- MySQL Shell now handles user interrupts, such as SIGINT, correctly. For example on Linux pressing Control-C when MySQL Shell is not executing anything exits the application. In SQL mode, interruption sends a `KILL QUERY` statement to the active MySQL Shell session from a new temporary session, resulting in the server interrupting the query and returning an error (or in an early return with no error in some cases, like the `sleep()` function). In JavaScript or Python scripting modes, how interruption behaves depends on the specific function being executed. If what is being executed is language code (such as a while loop and other normal script code), an exception is generated in the active language, which causes the code to stop executing. The exception may be caught by the script, but if not, the execution control returns to MySQL Shell. (Bug #24757361, WL #10568)
- MySQL Shell now includes a history function that stores the code which you issue. The history can be saved, searched, and filtered. A new mechanism to customize the MySQL Shell prompt has been added. Information such as the current mode (SQL, JavaScript, or Python), session information (host, URI, port and so on), the current active schema and others can be included in the prompt through variables. The customization information is self-contained in JSON theme files, which can be shared between users. MySQL Shell now supports unicode if the terminal used to run MySQL Shell supports it. Similarly if the terminal supports color, MySQL Shell can be configured to use colors in the theme. (WL #10446)
- The connection options passed to MySQL Shell, such as `sslMode` and so on, have been changed to use dashes and no longer be case sensitive. The options are now:
  - `sslMode` is now `ssl-mode`
  - `sslCa` is now `ssl-ca`
  - `sslCaPath` is now `ssl-capath`
  - `sslCert` is now `ssl-cert`
  - `sslKey` is now `ssl-key`
  - `sslCrl` is now `ssl-crl`
  - `sslCrlPath` is now `ssl-crlpath`
  - `sslCiphers` is now `ssl-ciphers`
  - `sslTlsVersion` is now `tls-version`

  - `authMethod` is now `auth-method`

  (WL #10912)

- The interpretation of the `document_path` field in operations such as `modify()` has been changed. Now, when the `document_path` is not set, operations apply to the whole document. All operations always preserve a document's `_id` field. (WL #10682)

Bugs Fixed

- **X DevAPI:** Unsigned data could be incorrectly read from the database. (Bug #24912358)
- In MySQL Shell, the `Schema.getCollectionAsTable()` function and the `select()` method could not be used in the same Python statement. (Bug #26552804)
- MySQL Shell returned some elements of DATE and DATETIME values incorrectly, including month values and fractional seconds. (Bug #26428636)
- The month was incorrectly incremented on insertion of a timestamp in a table using MySQL Shell. (Bug #26423177)
- For columns with the `ZEROFILL` attribute, `NULL` was also returned padded with zeroes. (Bug #26406214)
- The output of the MySQL Shell `\status` command was enhanced with additional information. (Bug #26403909)
- The MySQL Shell help for the `\connect` command indicated that a connection name could be used instead of a URI string, which was incorrect. (Bug #26392676)
- The MySQL Shell command `\use` did not attempt to reconnect if the connection to the global session was lost. (Bug #25974014, Bug #86118)
- The short form `-?` can now be used as an alias for the `--help` command-line option in MySQL Shell. (Bug #25813228)
- The MySQL Shell command history displayed the commands that were used to automatically import the `mysql` and `mysqlx` API modules when MySQL Shell started. (Bug #25739185)
- The MySQL Shell command history displayed the contents of scripts that were run using the `\source` MySQL Shell command. (Bug #25676495)
- The `mysqlx.getNodeSession()` function in MySQL Shell now returns an error if an unrecognized connection option is provided. (Bug #25552033)
- MySQL Shell did not exit gracefully when the user did not have a valid and accessible home directory. (Bug #25298480)
- MySQL Shell created a logger but did not deallocate it on exiting the shell. (Bug #25238576)
- MySQL Shell could hang when **Ctrl+C** was used to exit the shell. (Bug #25180850, Bug #84022)
- The parsing of Unix sockets provided as part of a URI has been improved. (Bug #24905066)
- MySQL Shell now accepts Unicode characters as input. (Bug #23151666, Bug #81176)

Changes in MySQL Shell 8.0.2 (Not released)

Version 8.0.2 has no release notes, or they have not been published because the product version has not been released.

Changes in MySQL Shell 8.0.1 (Not released)

Version 8.0.1 has no release notes, or they have not been published because the product version has not been released.

Changes in MySQL Shell 8.0.0 (2017-07-14, Development Milestone)

- AdminAPI Bugs Fixed
- Functionality Added or Changed
- Bugs Fixed

AdminAPI Bugs Fixed

- Executing AdminAPI commands on a server with a version of Python lower than 2.7 was failing without the correct error message. (Bug #25975317)
- When using MySQL Shell on Windows any files created or opened, for example those used during `dba.createSandboxInstance()`, could not be deleted. (Bug #25789094)
- The help for `dba.configureLocalInstance(instance[, options])` has been improved to describe the returned JSON object. (Bug #25703028)
- When using `dba.deploySandboxInstance()` and passing in `sandboxDir`, the specified path must not exceed 89 characters. (Bug #25485035)
- `removeInstance()` resulted in unexpected behavior in some cases, for example when an empty password was passed as part of the URI to the instance. (Bug #25111911)
- Creating Classic sessions that connect using Unix sockets now uses the correct defaults such as hostname. This resolves the previous limitation of using Unix sockets to connect to InnoDB cluster instances. See MySQL Shell Connections for information on how the defaults are applied to socket connections. (Bug #24848763, Bug #26036466)

  References: See also: Bug #24911068.
- On an instance configured as a multithreaded replica, in other words `slave_parallel_workers` set to greater than 0, and with `slave_parallel_type=DATABASE`, `dba.checkInstanceConfiguration()` was not detecting that the instance was not correctly configured for InnoDB cluster usage.
- If `removeInstance()` failed due to a connection error, an error was reported but the instance was incorrectly removed from the InnoDB cluster metadata, and remained part of the replication group. The fix ensures the metadata is correctly updated according to the result of `removeInstance`().
- In a situation where a new primary instance was elected, adding a new instance to the cluster resulted in an error due to a failed connection to the previous primary instance.
- The functions that modify server variables, such as `dba.createCluster()` and `dba.validateInstance()` now provide more information in interactive mode output and log output about server variables which are changed when executed.

- Deploying instances to paths with directories that contained spaces was failing without error. Use double backslash to specify such paths, for example `D:\\Cluster\\foo bar`.
- The Cluster object obtained from functions such as `dba.createCluster()` or `dba.getCluster()` became unusable once the Shell session in which the object was created is was connected to a different server. The fix modifies the Cluster object so that:
  - The Cluster object holds an internal reference to the Session from which it was created or retrieved.
  - AdminAPI functions that modify the Cluster are made using the session referenced by the object.

Functionality Added or Changed

- Calling the `modify()` or `remove()` function without a parameter caused the function to be executed against the whole collection, which could cause unexpected results such as deleting all rows in a table. To avoid this and make the behavior consistent with `update()` and `delete()`, a client-side exception is now thrown if the `modify()` or `remove()` function is called without a parameter. Now, to execute the `modify()` or `remove()` function against a collection call them with an expression that evaluates to `true`, for example `remove('true')` or `modify('true')`. (WL #10710)

Bugs Fixed

- The options in the MySQL Shell options dictionary are now fully documented. (Bug #25701345)
- `shell.connect()` did not report an error if an invalid argument was used. An `ArgumentError` is now issued for any invalid argument.

  The following mutually exclusive pairs of options are now checked, and an error is issued if both are specified:

  - `--password` and `--dbPassword`
  - `--user` and `--dbUser`
  - `--port` and `--socket`

  (Bug #25268670)

  References: See also: Bug #24911173.
- A number of issues with the output of `shell.help("prompt")` have been corrected. (Bug #25026855, Bug #25242638, Bug #25676343, Bug #25176769)
- MySQL Shell now displays an invalid year as `0000`, matching the behavior of the MySQL prompt, rather than as `0`. (Bug #24912061)
- MySQL Shell did not display fractional seconds for values in DATETIME columns. (Bug #24911885)
- Some issues with the MySQL Shell command line help output were fixed. (Bug #24841749, Bug #24841493, Bug #24910540)
- URIs were incorrectly parsed in MySQL Shell when passwords were hidden. (Bug #24793956)
- `mysqlsh` stopped responding if the `\source` command was given a directory (rather than file) argument. (Bug #23097932, Bug #81060)